



REPUBLIC OF TUNISIA
Ministry of Higher Education and Scientific Research
University of Carthage
National Institute of Applied Sciences and Technology



Graduation Project

To obtain the

ENGINEERING DEGREE

In Applied Sciences and Technology

Specialty: Software Engineering

CLMPilot: an integration-first Contract Lifecycle Management platform, built on a CQRS and Event Sourcing Architecture

Presented by

Idris SADDI

Hosted by



INSAT Supervisor : **Aymen SELLAOUTI**
Company Supervisor : **Artur SUSDORF**

Presented on: **24/09/2026**

JURY

Rabaa YOUSSEF (President)
Lilia SFAXI (Reviewer)

Academic Year: 2025/2026



REPUBLIC OF TUNISIA
Ministry of Higher Education and Scientific Research
University of Carthage
National Institute of Applied Sciences and Technology



Graduation Project

To obtain the

ENGINEERING DEGREE

In Applied Sciences and Technology

Specialty: Software Engineering

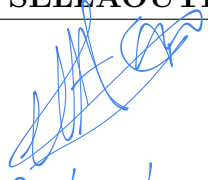
CLMPilot: an integration-first Contract Lifecycle Management platform, built on a CQRS and Event Sourcing Architecture

Presented by

Idris SADDI

Hosted by



Supervisor at the Company Artur SUSDORF	Supervisor at INSAT Aymen SELLAOUTI
 Date: 10.10.2026	 Date: 12/9/2026

Academic year: 2025/2026

Dedication

الحمد لله الذي بنعمته تتم الصالحات

All praise be to God, by whose grace all good things are accomplished.

To my loving parents, my twin pillars and my role models, for their everlasting love and unwavering sacrifices, who gave everything to help me become the best version of myself. Thank you for everything you have given me and for always being there. None of this would have been possible without your wonderful support and endless sacrifice.

To my brother and sisters, for their endless encouragement and their trust in me.

To all my friends, for having sparked my journey from the very beginning, and for the constant encouragement that made all of this possible.

To the INSAT community, which shaped parts of who I am and gave me countless unforgettable memories in the last five years.

To the JCI INSAT family: thanks to you, I discovered new potentials in myself and developed values, experiences and friendships that shaped my professional and personal identity. You helped bring out the best in me, and I will always be grateful.

Thank you for everything.

Acknowledgements

As I conclude this chapter of my academic journey, I want to thank those who have supported, guided, and inspired me along the way. This accomplishment is not mine alone; it reflects the encouragement, wisdom, and patience of many remarkable people.

I am profoundly grateful to Mr. Aymen Sellaouti, my academic supervisor, who has been an exceptional mentor throughout my five years at the university. Beyond guiding my academic work, he has shared professional lessons that have shaped my growth in ways that extend well past the classroom. Thank you for your guidance, your patience, and your steady support.

My sincere thanks go to my supervisor, Artur Susdorf, for believing in me from day one, and for his advice and help throughout the internship. I am equally grateful to the Gradient Zero team, who welcomed me warmly and offered their support, insight, and friendship.

I would also like to thank the members of the jury for taking the time to evaluate my work. Your feedback and suggestions are greatly appreciated and will undoubtedly contribute to my continued growth in this field.

Table of Contents

List of Figures	vii
List of Tables	ix
Abstract	x
General Introduction	1
I General Context and Project Scope	3
I.1 Hosting company: Gradient Zero	3
I.1.1 General presentation	3
I.1.2 Areas of expertise	4
I.2 Project context: CLM in the DACH Mittelstand	4
I.2.1 The Mittelstand segment	4
I.2.2 Market and compliance pressure	5
I.2.3 Competitive landscape	6
I.3 Problem statement	6
I.4 Proposed solution	8
I.5 Methodology	8
I.5.1 Team and supervision	9
I.5.2 The Extreme Programming methodology	9
I.5.3 Base practices applied in this project	9
I.5.4 Extreme Programming for a solo developer	10
I.5.5 Release planning and traceability	10
II Analysis and Requirements Specification	13
II.1 Actors	13
II.1.1 Actor catalogue	13
II.2 Functional requirements	14
II.2.1 Contract registry	14
II.2.2 Document linking	14
II.2.3 Deadline and renewal management	15
II.2.4 Approval workflows	15
II.2.5 Compliance and audit trail	15

II.2.6	Dashboard and reporting	15
II.2.7	Identity and access management	16
II.2.8	Advanced capabilities extending the core registry	16
II.3	Non-functional requirements	16
II.3.1	Security	17
II.3.2	Compliance and data residency	17
II.3.3	Availability and reliability	17
II.3.4	Performance	18
II.3.5	Auditability	18
II.3.6	Internationalisation	18
II.3.7	Maintainability	19
II.4	Use cases	19
II.4.1	Use-case diagram	19
II.4.2	Authentication use cases	20
II.4.3	Use cases by functional area	21
II.4.4	Detailed use-case descriptions	23
III	Architecture and Design	26
III.1	Architectural drivers	26
III.2	Why CQRS and Event Sourcing for CLM?	27
III.2.1	Architectural choice	27
III.2.2	CQRS and Event Sourcing	28
III.3	The comby framework	30
III.3.1	Role in the architecture	30
III.3.2	Justification of choice	32
III.4	Domain model overview	33
III.4.1	Bounded contexts	33
III.4.2	Aggregate responsibilities	34
III.4.3	Behavioural views: lifecycle and workflow	34
III.4.4	Extension contexts	36
III.5	System topology	37
III.5.1	Component view	37
III.5.2	Deployment topology	38
III.6	Security and compliance model	39
III.6.1	Identity and access model	39

III.6.2 Authentication and the request security pipeline	39
III.6.3 Defence in depth and tenant isolation	41
III.6.4 Compliance positioning	42
III.7 Internationalisation strategy	42
IV Technical realisation of the platform	44
IV.1 Engineering foundations and the delivery process	44
IV.2 Backend service architecture on comby	46
IV.3 The contract domain and its lifecycle	48
IV.4 Workflow and temporal automation	50
IV.5 Document intelligence	51
IV.6 Electronic signature	53
IV.7 The web application and its internationalisation	54
IV.8 Identity, access control and multi-tenancy in practice	56
IV.9 Production deployment and release operations	58
V The delivered platform	61
V.1 Access and onboarding	61
V.2 Reporting and the dashboard	62
V.3 The contract registry and its audit trail	63
V.4 Fitting the platform to the organisation	64
V.5 Approvals, deadlines and obligations	65
V.6 Reading and signing the document	67
V.7 The product in 2 languages	68
V.8 The platform from the operator's side	68
V.9 Measured performance in production	69
V.9.1 The runtime instrumentation	69
V.9.2 Traffic, response times and reliability	70
V.9.3 Against the non-functional requirements	71
Conclusion and Perspectives	73
References	75
Appendices	77
A.1 Glossary	77

A.2	Comby framework reference	77
A.3	Configuration and delivery pipeline	78
A.3.1	Production Docker Compose stack	79
A.3.2	Host nginx reverse proxy	80
A.3.3	systemd service unit	81
A.3.4	Continuous-integration pipeline	81
A.3.5	Release script and the lockstep bypass	82
A.3.6	Packaging caveat: the bundled help documentation	82

List of Figures

I.1	System context: CLMPilot manages the contract lifecycle	8
I.2	The release-train delivery model	11
I.3	Project timeline: GANTT chart	11
II.1	High-level use-case diagram for CLMPilot	20
II.2	Detailed use cases: authentication	21
II.3	Detailed use cases: identity and access administration	21
II.4	Detailed use cases: contracts and documents	22
II.5	Detailed use cases: approvals and deadlines	22
II.6	Detailed use cases: dashboard, reporting and audit	23
III.1	The CQRS and Event Sourcing flow	29
III.2	The command path and its reactions, in time	29
III.3	Layering of CLMPilot on comby.	31
III.4	Packages structure of the backend	32
III.5	The 4 bounded contexts, within the tenant boundary	34
III.6	State machine of the Contract aggregate	35
III.7	Activity diagram of the approval workflow	36
III.8	CLMPilot component view	37
III.9	CLMPilot deployment topology	38
III.10	The identity and access model	39
III.11	The request security pipeline	40
III.12	Authorisation process of commands and queries	41
III.13	Defence in depth	41
IV.1	The 4 repositories by concern	45
IV.2	The realised command path and query path, side by side	47
IV.3	The Contract and DocumentLink model with its events	49
IV.4	Temporal automation in the Deadline context	50
IV.5	The enrichment pipeline	52
IV.6	Component view of the e-signature webhook flow	53
IV.7	Package structure of the single-page application	55
IV.8	The authentication journey across the stack	57

IV.9	The production topology	58
V.1	Two-step sign-in	61
V.2	The dashboard	62
V.3	The report builder	63
V.4	The contract registry	63
V.5	The audit-log view of a contract	64
V.6	The contract form with custom fields	64
V.7	The integrations settings of a tenant	65
V.8	The approval inbox	65
V.9	The deadline view	66
V.10	Obligation tracking	66
V.11	Enrichment suggestions on a contract	67
V.12	The signature request on a contract	67
V.13	The bundled help area in German	68
V.14	The cross-tenant administration area	69
V.15	The runtime metrics dashboard	70

List of Tables

I.1	CLM market segmentation as it appears to a DACH Mittelstand buyer.	6
II.1	Actors recognised by CLMPilot, with their primary use cases.	14
II.2	Performance test metrics: the measured operation, the metric observed, the test condition, and the MVP target.	18
II.3	Use-case description: register a contract.	23
II.4	Use-case description: approve a contract.	24
II.5	Use-case description: reconstruct past contract state for an audit.	25
III.1	Established architectural style opposed on each dimension of the foundational choice.	27
III.2	CQRS and Event Sourcing platforms considered for CLMPilot, assessed against the project's decisive criteria.	32
V.1	Dispatch measurements from the production pilot.	70
V.2	The delivery against each non-functional requirement.	71
A.1	Command and query catalogue of the CLMPilot contexts.	78

Abstract

In the European mid-sized enterprises that form the economic backbone of the DACH region (Germany, Austria and Switzerland), contracts are still managed by hand, scattered across email inboxes, shared drives and filing cabinets. Renewal and cancellation windows pass unnoticed, approvals leave no auditable record, and the contract portfolio can be surveyed only through a manual exercise that is stale the moment it is finished. This happens at a time when GoBD, the European data-protection regulation and ISO 27001 all expect business records to be retained immutably and traced to an actor and a moment. The Contract Lifecycle Management (CLM) platforms that could close this gap are scoped and priced for the large enterprise, or require the customer to migrate every document into the vendor's own storage.

This report presents **CLMPilot**, an integration-first contract lifecycle management platform for the DACH *Mittelstand*, designed and built during an end-of-studies internship at Gradient Zero. Its defining decision is to manage the lifecycle of a contract (its metadata, deadlines, approvals and audit trail) while the document itself stays in the customer's own storage, so adoption carries no migration cost. The platform is built on *comby*, an in-house Go framework providing Command-Query Responsibility Segregation and Event Sourcing; that choice makes the compliance-grade audit trail a property of the architecture itself rather than a bolt-on feature.

Keywords: Contract Lifecycle Management, CQRS, Event Sourcing, Domain-Driven Design, regulation, audit trail, DACH *Mittelstand*.

Zusammenfassung

In den mittelständischen Unternehmen, die das wirtschaftliche Rückgrat der DACH-Region (Deutschland, Österreich und die Schweiz) bilden, werden Verträge nach wie vor von Hand verwaltet und verteilen sich über E-Mail-Postfächer, gemeinsame Laufwerke und Aktenschränke. Verlängerungs- und Kündigungsfristen verstreichen unbemerkt, Freigaben hinterlassen keine prüfbare Spur, und das Vertragsportfolio lässt sich nur durch eine manuelle Erhebung überblicken, die bereits veraltet ist, sobald sie abgeschlossen wurde. Dies geschieht zu einer Zeit, in der die GoBD, die europäische Datenschutz-Grundverordnung und ISO 27001 gleichermaßen verlangen, dass geschäftsrelevante Aufzeichnungen unveränderbar aufbewahrt und einem Akteur sowie einem Zeitpunkt zugeordnet werden. Die Plattformen für Contract Lifecycle Management (CLM), die diese Lücke schließen könnten, sind auf Großunternehmen zugeschnitten und entsprechend bepreist, oder sie zwingen den Kunden, sämtliche Dokumente in den Speicher des Anbieters zu migrieren.

Dieser Bericht stellt **CLMPilot** vor, eine integrationsorientierte Plattform für das Vertragslebenszyklus-Management für den DACH-*Mittelstand*, die im Rahmen eines Abschlusspraktikums bei Gradient Zero entworfen und umgesetzt wurde. Ihre bestimmende Entscheidung besteht darin, den Lebenszyklus eines Vertrags (seine Metadaten, Fristen, Freigaben und Prüfspur) zu verwalten, während das Dokument selbst im Speicher des Kunden verbleibt, sodass die Einführung keine Migrationskosten verursacht. Die Plattform baut auf `comby` auf, einem internen Go-Framework, das Command-Query Responsibility Segregation und Event Sourcing bereitstellt; diese Entscheidung macht die revisionssichere Prüfspur zu einer Eigenschaft der Architektur selbst und nicht zu einer nachträglich ergänzten Funktion.

Schlüsselwörter: Vertragslebenszyklus-Management, CQRS, Event Sourcing, Domain-Driven Design, Regulierung, Prüfspur, DACH-*Mittelstand*.

Résumé

Dans les moyennes entreprises qui constituent l'épine dorsale économique de la région DACH (l'Allemagne, l'Autriche et la Suisse), les contrats sont encore gérés manuellement et dispersés entre boîtes de messagerie, disques partagés et armoires d'archives. Les fenêtres de résiliation et de renouvellement passent inaperçues, les validations ne laissent aucune trace auditable, et le portefeuille contractuel ne peut être recensé que par un exercice manuel, périmé dès qu'il est achevé. Or le GoBD, le règlement européen sur la protection des données et la norme ISO 27001 exigent désormais que les documents d'activité soient conservés de façon immuable et rattachés à un acteur et à un instant. Les plateformes de gestion du cycle de vie des contrats (CLM) susceptibles de combler ce manque sont dimensionnées et tarifées pour les grands comptes, ou imposent au client de migrer l'ensemble de ses documents vers le stockage de l'éditeur.

Ce rapport présente **CLMPilot**, une plateforme de gestion du cycle de vie des contrats orientée intégration et destinée au *Mittelstand* de la région DACH, conçue et réalisée au cours d'un stage de fin d'études chez Gradient Zero. Le but fondamental de la plateforme est de gérer le cycle de vie du contrat (ses métadonnées, ses échéances, ses validations et sa piste d'audit) tandis que le document lui-même reste dans le stockage du client : l'adoption n'entraîne donc aucun coût de migration. La plateforme repose sur *comby*, un framework Go interne mettant en œuvre la séparation des responsabilités entre commandes et requêtes ainsi que l'*Event Sourcing* ; ce choix fait de la piste d'audit conforme aux exigences réglementaires une propriété de l'architecture elle-même, et non une fonctionnalité ajoutée.

Mots-clés : gestion du cycle de vie des contrats, CQRS, Event Sourcing, conception pilotée par le domaine, règlement, piste d'audit, *Mittelstand* de la région DACH.

ملخص

في الشركات الأوروبية المتوسطة الحجم التي تمثل العمود الفقري لاقتصاد منطقة DACH (ألمانيا والنمسا وسويسرا)، لا تزال العقود تدار يدويًا وتتوزع بين صناديق البريد الإلكتروني والأقراص المشتركة وخزائن الملفات. ونتيجة لذلك، تفوت مواعيد التجديد والإلغاء دون ملاحظتها، ولا تترك الموافقات أي أثر قابل للتدقيق، كما تعتري النظرة الشاملة على محفظة العقود صعوبة بالغة إذ تتطلب جهدًا يدويًا يفقد صلاحيته بمجرد إنجازها. يحدث هذا في وقت تشترط فيه قواعد GoBD واللائحة الأوروبية لحماية البيانات ومعياري ISO 27001 حفظ سجلات الأعمال بصورة غير قابلة للتعديل وتتبعها إلى المستخدم واللحظة المحددة. أما منصات إدارة دورة حياة العقود (CLM) القادرة على سد هذه الفجوة، فإما أن تكون مصممة ومسعرة للمؤسسات الكبرى، أو تلزم العميل بترحيل جميع وثائقه إلى بيئة التخزين الخاصة بالزود.

يقدم هذا التقرير منصة CLMPilot، وهي منصة لإدارة دورة حياة العقود قائمة على أولوية دمج الخصائص والوثائق وموجهة للشركات المتوسطة (Mittelstand) في منطقة DACH تم تصميمها وتطويرها خلال تدريب نهاية الدراسة في شركة Gradient Zero. ويكمن الهدف الأساسي للمنصة في إدارة دورة حياة العقد (بياناته الوصفية، وأجاله، وموافقاته، وسجل تدقيقه) مع بقاء الوثيقة ذاتها في بيئة التخزين الخاصة بالعمل، مما يلغي تكاليف الترحيل عند اعتمادها. تعتمد المنصة على comby، وهو إطار عمل داخلي بلغة Go يطبق نمط فصل مسؤوليات الأوامر والاستعلامات (CQRS) ونمط مصادر الأحداث (Event Sourcing)؛ مما يجعل سجل التدقيق المطابق للمتطلبات التنظيمية خاصة أصيلة في بنية النظام ذاته وليست مجرد ميزة إضافية.

الكلمات المفتاحية: إدارة دورة حياة العقود، CQRS، Event Sourcing، التصميم الموجه بالجال، لوائح تنظيمية، سجل التدقيق، الشركات المتوسطة في منطقة DACH.

General Introduction

Contracts record what an organisation has committed to, with whom, on what terms and until when. For the European *Mittelstand* – the mid-sized enterprises that form the economic backbone of Germany, Austria and Switzerland – managing those commitments is still largely manual, even as the rest of the back office has moved online. At the same time, the regulatory bar has risen: GoBD¹, the European data-protection regulation (GDPR) and the ISO 27001 security standard now expect business records to be retained immutably, traced to an actor and a moment, and kept within controlled boundaries. The discipline that addresses this, Contract Lifecycle Management (CLM), is mature in the large enterprise, but the tools that serve it there assume budgets, IT departments and rollout timelines the mid-market does not have.

The consequence is a recurring and costly gap. In a company without a dedicated system, contracts are scattered across email inboxes, shared drives and filing cabinets; renewal and cancellation windows pass unnoticed, so agreements renew silently and decisions default to the counterparty; there is no auditable record of who approved which version of a contract; and the simple portfolio questions (how much value expires next quarter, where the renegotiation leverage lies) can be answered only by a manual exercise that is stale the moment it is finished. The platforms that could close this gap either price and scope themselves for the enterprise, or, in the compliance-aware DACH niche, require the customer to migrate every document into the vendor’s own storage, an entry fee that pushes mid-market buyers back to spreadsheets and email.

This report presents **CLMPilot**, an integration-first contract lifecycle management platform for the DACH *Mittelstand*, built during an end-of-studies internship at Gradient Zero. Its defining decision is to manage the lifecycle of a contract (its metadata, deadlines, approvals and audit trail) while the document itself stays in the customer’s own storage, so adoption carries no migration fee. The platform is designed so that the compliance-grade audit trail is a property of its storage model itself. CLMPilot was delivered as a release train, and the realised pilot covers foundations, a full-stack minimum viable product, and a tier of enterprise features.

The report is organised in 4 chapters. Chapter I sets the general context and the scope of the project: the hosting company, the market, the problem, the solution we propose in response, and the methodology under which the work was carried out. Chapter II analyses and specifies the requirements: the actors, then the functional and non-functional requirements, and finally

¹GoBD stands for *Grundsätze zur ordnungsmäßigen Führung und Aufbewahrung von Büchern, Aufzeichnungen und Unterlagen in elektronischer Form sowie zum Datenzugriff*. It is a German administrative directive that defines exactly how companies must handle, process and store tax-relevant digital documents, to ensure auditability and prevent fraud.

the use cases through which they are exercised. Chapter [III](#) presents the architecture and design, justifying the choice of CQRS and Event Sourcing, deriving the domain model and the system topology, and setting out the security and compliance model. Chapter [IV](#) presents how the platform was built, consolidating the engineering work delivered across the realised phases, and Chapter [V](#) presents the result: the platform as its users and operators meet it, and the performance measured on it in production. A general conclusion then recaps the contribution and sets out the perspectives for future work, from the evolution of the event-sourced framework beneath the platform to the further phases of the product roadmap.

Chapter I

General Context and Project Scope

Introduction

This chapter establishes the context in which CLMPilot was built and the scope the project set out to deliver. We introduce Gradient Zero, the hosting company that initiated the project, and characterise the market the product addresses (contract lifecycle management in the DACH Mittelstand) together with the regulatory, operational and commercial pressures that make this market worth addressing now. From that context we derive the 4 operational pain points the product responds to, and we state, at the level of scope rather than implementation, the solution we propose in answer to them. The chapter closes with the methodology under which the work was organised: Extreme Programming, adapted to a solo build and planned as a release train of product phases.

I.1 Hosting company: Gradient Zero

I.1.1 General presentation

Gradient Zero¹, registered as *Gradient Zero Softwareentwicklungs GmbH*, is a software and machine-learning company founded in 2018 and headquartered in Vienna, Austria, whose motto is *Machine Learning Excellence*. Its core business is privacy-first artificial intelligence: it builds platforms and bespoke solutions that let organisations extract value from highly sensitive data without exposing it, and it develops parts of that work into standalone products and spin-off ventures.

The company's technology portfolio reflects that focus. *DQ0* (Detabord) provides secure data management built on differential privacy, so analysis can run over sensitive data in a GDPR-compliant way; *Daiki* is an AI-governance platform for managing the risks of trustworthy AI; and an in-house private large-language-model and retrieval-augmented-generation stack, a Git-based data-versioning platform certified by TÜV Austria, and a library of pre-trained machine-learning methods complete the portfolio. These products are deployed chiefly in highly

¹<https://gradient0.com/>

regulated sectors (healthcare and medical research, banking and financial technology, agriculture and industrial automation) where data-protection law is a hard constraint, not an afterthought.

I.1.2 Areas of expertise

Of Gradient Zero’s technology assets, the one CLMPilot is built on is **comby**, an internal Go framework that provides the building blocks of a Command-Query Responsibility Segregation (CQRS) and Event Sourcing application [1–3]: command and query dispatch, an event-store abstraction, projection runners, and a reactor scaffold. Comby is the company’s response to the build-versus-buy question: where each new product would otherwise reimplement, or import from an open-source project, the same infrastructure for audit trails, approval workflows, and temporal queries, comby makes that infrastructure a shared asset reused across products.

3 engineering principles recur across the privacy-first portfolio, and CLMPilot inherits all 3. The first is *compliance by architecture*: immutable event logs satisfy GoBD and ISO 27001 requirements without bolt-on audit features. The second is to *overlay existing systems rather than replace them*: the products are not built to become the system of record for customer data, but to layer workflow, governance and metadata on top of the storage and applications the customer already uses. The pattern runs through the company’s privacy and data-governance products, and CLMPilot applies it to contract documents specifically. The third is a *DACH-first product design*: German-first user experience, German tax-law constraints, and EU-hosted infrastructure as defaults, not premium add-ons.

I.2 Project context: CLM in the DACH Mittelstand

Contract Lifecycle Management (CLM) names the set of practices and tools through which an organisation manages its contracts from inception through negotiation, execution, performance, renewal and termination. While the discipline is mature in large enterprises, where dedicated CLM platforms from vendors such as SAP, ServiceNow or Icertis are commonplace, its application to the mid-market has historically been thin. This section sketches the market segment that CLMPilot targets, the regulatory pressure that frames the opportunity, and the competitive landscape against which the product’s positioning was designed.

I.2.1 The Mittelstand segment

The German term *Mittelstand* names the family of mid-sized enterprises that form the economic backbone of Germany, Austria and Switzerland (collectively the DACH region). Definitions

vary: the Institut für Mittelstandsforschung classifies companies with up to 500 employees and revenues below 50 million euros as Mittelstand by its qualitative criterion [4], while the European Commission’s SME definition stops at 250 employees [5]. For the purposes of CLMPilot, the operative segment is broader: companies of approximately 100 to 5,000 employees, generating between 20 million and 1 billion euros in annual revenue, and operating in industries where contracts are central to the business: finance, healthcare, manufacturing and professional services.

This segment has 2 properties that drive product design. Contract volume per company is substantial: typical customers manage between 200 and 10,000 active contracts at any time, with churn driven by renewal cycles. And the operating model is mixed: most companies run on Microsoft 365 or Google Workspace for daily work, retain a small in-house IT function, and have neither the budget nor the appetite for a 6-month enterprise software rollout.

I.2.2 Market and compliance pressure

3 structural pressures make CLM a present concern for Mittelstand decision-makers.

The first pressure is **regulatory**. The German *Grundsätze zur ordnungsmäßigen Führung und Aufbewahrung von Büchern, Aufzeichnungen und Unterlagen in elektronischer Form sowie zum Datenzugriff* (GoBD) requires that electronic business records, contracts among them, be retained immutably, with a verifiable audit trail, for periods of up to 10 years. ISO 27001 certification, increasingly demanded by customers in B2B sales, imposes parallel requirements on access control and change tracking. The European General Data Protection Regulation (GDPR) adds data-residency and processing-record obligations on top of both.

The second pressure is **operational**. In the absence of a dedicated CLM, contracts are scattered across email inboxes, network drives and physical filing cabinets. Auto-renewal clauses trigger silently when cancellation windows pass unnoticed, costing companies thousands of euros per incident. Consolidated reporting (“*how much contract volume expires in Q3?*”, “*which vendors have we exceeded the agreed thresholds with?*”) requires manual aggregation across these scattered sources, producing snapshots that are out of date as soon as they are delivered.

The third pressure is **commercial**. The global CLM software market is projected to grow from USD 2.07 billion in 2026 to USD 5.09 billion by 2034, an 11.9% CAGR over the forecast period, with North America holding a 35% share in 2025 [6]. Penetration in the DACH Mittelstand, however, remains low: the majority of companies still operate on file-share-plus-spreadsheet processes.

These pressures compound: companies that handled the issue informally 5 years ago are increasingly forced to formalise their contract operations, and the tools to do so are needed now.

I.2.3 Competitive landscape

The CLM market separates into 4 bands that map cleanly onto the size and geography of the buyer. Table I.1 summarises the segmentation as it appears to a DACH Mittelstand customer.

Table I.1 – CLM market segmentation as it appears to a DACH Mittelstand buyer.

Band	Representative players	Gap from a Mittelstand buyer's standpoint
Enterprise	SAP CLM, ServiceNow CLM, Icertis	Pricing assumes an enterprise IT budget; deployment timelines measured in quarters; functional scope much wider than required.
Mid-market (US)	Ironclad, Juro, ContractWorks	US-first user experience and integrations; weak coverage of DACH compliance base-lines (GoBD); English-only at the lower tiers.
DACH niche	ContractHero, Vertbase, TopLegal	Compliance fit is good, but adoption requires migrating documents into the vendor's storage, a multi-month project that consumes the very budget the product is meant to free up.
DIY	Excel + Outlook + file shares	Nominally free; no automation, no compliance tracking, no portfolio visibility.

The structural gap that CLMPilot is designed to address is in the third row: the DACH-niche category gets the regulatory fit right but pays for it with an up-front document-migration cost that pushes mid-market customers back toward DIY. The positioning chosen for CLMPilot, to manage the lifecycle of contracts whose documents *stay in their original storage*, avoids that cost, and drives many of the architectural choices presented in Chapter III.

I.3 Problem statement

From customer-discovery interviews with target Mittelstand companies, 4 pain points recur consistently, each with a distinct operational and financial cost. The platform this report presents can be read as a structured response to these 4 problems.

Missed deadlines

The default outcome for a contract whose renewal window is unmonitored is silent auto-renewal. A typical multi-year vendor agreement triggers a 1-year extension if it is not formally cancelled in a 30 to 90 day window before its expiry; in practice these windows pass unnoticed because no system is responsible for flagging them. The financial cost per incident varies, from a few

hundred euros for a low-value subscription to tens of thousands for a vendor lock-in that should have been renegotiated, but the structural cost is constant: a critical business decision is taken not by the people responsible for it, but by the default behaviour of an external counterparty.

No audit trail

When an external auditor or an internal compliance function asks who approved a given contract, when, and on which version of the document, the answer in the absence of a CLM is *“let me search my email”*. Email threads are not designed as auditable artefacts: they are mutable, they degrade through deleted accounts and expired retention windows, and they offer no guarantee that the version of the document approved is the version currently on file. This gap is enough to fail a GoBD audit or delay an ISO 27001 certification.

No portfolio overview

A common executive question (*“how much contract volume expires in Q3?”*, *“what is our renegotiation leverage with vendor X over the next 6 months?”*, *“which suppliers will we exceed the agreed thresholds with?”*) has no good answer in a file-share-based contract operation. The data exists, but its retrieval is a manual exercise that consumes a person-week of effort and produces a snapshot that is out of date the moment it is produced. Strategic decisions on supplier consolidation, departmental budgets and M&A diligence are routinely taken with this information missing.

Unstructured approvals

Contracts that should pass through legal review, financial review and management sign-off, in that order, are in practice approved through hallway conversations and forwarded emails. There is no enforceable workflow, no record of who approved what, and no way to verify after the fact that the approval policy was followed. The compliance exposure here is severe: in a regulated industry, the absence of a documented approval chain is, by itself, sufficient grounds for fines or for the invalidation of the contract concerned.

These 4 pain points are not independent. They share a common cause (the lack of a system that treats the contract lifecycle as a managed entity with its own metadata, deadlines, approvals and audit log) and they share a common solution, which the next section states as the scope of this project.

I.4 Proposed solution

We propose an integration-first contract lifecycle management platform that answers the 4 pain points at their common root, by treating the contract lifecycle as a first-class object. At its centre is a contract registry that carries each agreement’s metadata, deadlines, approvals and audit history, and thereby gives the organisation the consolidated portfolio view it lacks today. The documents stay in the customer’s own storage: the platform links to files there rather than ingesting them, so adoption carries no document-migration project. Figure I.1 draws this positioning as a system boundary: CLMPilot owns the contract’s metadata, deadlines, approvals and audit trail, while the document itself never leaves the customer’s storage.

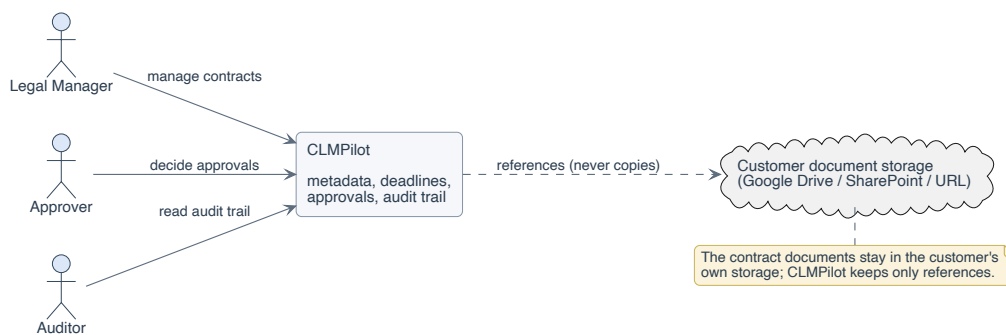


Figure I.1 – System context: CLMPilot manages the contract lifecycle

Around that registry, 3 capabilities address the remaining pain points directly. Automated reminders flag every cancellation, renewal and expiry window before it passes, returning renewal decisions to the people responsible for them. Enforceable approval workflows replace forwarded emails with an ordered sequence of recorded decisions. And a compliance-grade audit trail records every state-changing action immutably, so that who approved what, when, and on which version of a contract can be answered years after the fact.

I.5 Methodology

The methodology under which CLMPilot was built was governed by a single operating constraint: the work was carried out by one developer against a fixed set of phase milestones, with a scope that customer-discovery findings were still refining during the first weeks. That same scope bounds this report. The internship delivered a working pilot across 3 phases (foundations, the MVP, and a tier of enterprise features). For the build itself we adopted *Extreme Programming* (XP) [7], an agile methodology suited to exactly these conditions, in the solo variant described for a single developer [8]. The subsections that follow describe the team and supervision under

which the work ran, define the methodology, state how its base practices were applied, explain how its team-oriented practices were adapted to a solo build, and describe the release planning and traceability that organised the work.

I.5.1 Team and supervision

The CLMPilot project was carried out as a solo build by the author of this report, supervised by Mr. Artur Susdorf, Chief Technology Officer of Gradient Zero, through a weekly synchronisation meeting at the start of each week. The release-train cadence detailed at the end of this section – Phase 0 (foundations), Phase 1 (the MVP) and Phase 2 (enterprise features) – was set jointly with the company supervisor at the start of the internship and adjusted in light of customer-discovery findings during the first weeks.

Academic supervision was provided by Dr. Aymen SELLAOUTI, whose role focused on the academic framing of the project: the soundness of the architectural choices, the rigour of the methodology, and the structure of this report.

I.5.2 The Extreme Programming methodology

Extreme Programming is an agile methodology introduced by Kent Beck, who defines it as a lightweight discipline of software development for small teams facing vague or rapidly changing requirements [7]. Its purpose is to lower the cost of change: instead of fixing every requirement up front, an XP project advances in short cycles, each of which leaves the product working, tested and deliverable, so that feedback arrives while it can still alter the plan. The methodology rests on 5 values (communication, simplicity, feedback, courage and respect) and makes them operational through a set of concrete practices, among them the planning game, small releases, testing, refactoring, simple design, coding standards, continuous integration, pair programming and an on-site customer [7].

I.5.3 Base practices applied in this project

Rather than survey the practices, we state how the decisive ones were applied. *Small releases* set the delivery rhythm: every release leaves the platform in a deployable state, and the pilot reached production early and was then re-released repeatedly, rather than delivered once at the end of the internship. *Continuous integration* is enforced mechanically: each repository carries a pipeline that lints, tests and builds every change before it can reach a protected branch. *Testing* is layered (unit, integration and end-to-end), and the pipeline runs these suites on every change. *Simple design* translates into the deliberately narrow technology stack motivated by

the maintainability requirement, and *refactoring* keeps the design aligned with the domain as that understanding improves. *Coding standards*, finally, are held uniform by linters and by conventional commit messages, which also carry the traceability described at the end of this section.

I.5.4 Extreme Programming for a solo developer

Extreme Programming is written for teams, and some of its practices presuppose a second person; the literature nonetheless describes its use by a single developer [8], and parts of the discipline become simpler in that setting: continuous integration, for one, proceeds without merge conflicts. 2 practices required adaptation. *Pair programming* is replaced by asynchronous review: every change reaches the protected branches through a pull request reviewed by the Gradient Zero engineering team, and the continuous-integration gate provides the constant verification a pairing partner would. The role of the *on-site customer* is filled by the company supervisor: each week opens with a synchronisation meeting, the project's planning game, in which the prior week's commits are reviewed against the roadmap and the next items are selected, and architectural decisions are not on a fixed schedule but raised as they arise. There is no fixed sprint length, since a solo developer is best served by a continuous flow with explicit checkpoints; the checkpoints are the release gates described next.

I.5.5 Release planning and traceability

In Extreme Programming, release planning fixes what each release contains and when it ships. Stretched over the internship, our release plan took the form of a release train of 3 product phases, each with an explicit entry gate, exit milestone and demonstrable artefact: *Phase 0* – foundations (repository, CI/CD, development environment, planning); *Phase 1* – the MVP itself, covering the contract registry, document linking, deadlines, approvals, dashboard and IAM; and *Phase 2* – enterprise features on top of the MVP (intelligence-assisted extraction, electronic signature, per-tenant custom fields, obligation tracking and advanced reporting). Each of these phases advances the product to a state in which it can be deployed and used; no phase is internal-only, and all 3 were realised and shipped to production. Figure I.2 shows the 3 realised phases and the gates between them.

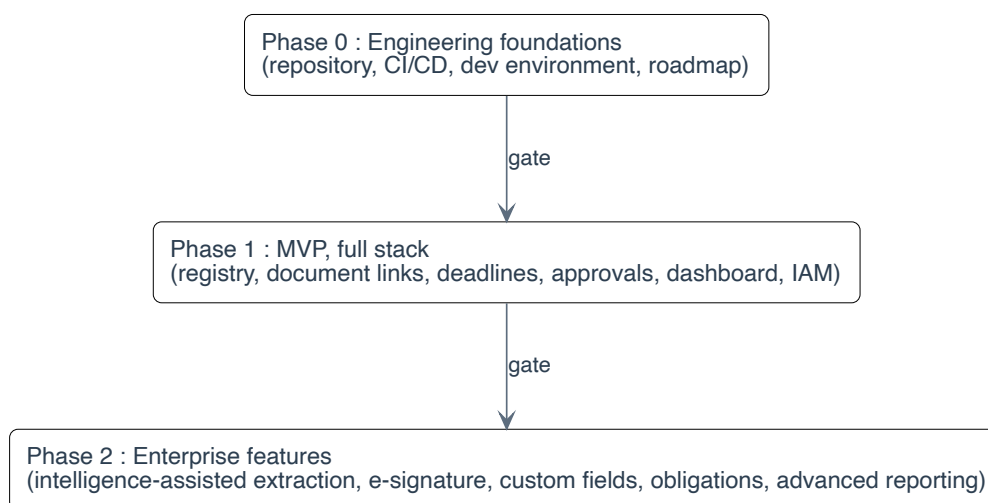


Figure I.2 – The release-train delivery model

Figure I.3 places the release train on the calendar of the internship, together with the sub-phases each phase carried. 2 milestones mark the gates between the phases after which the remaining weeks carry the audit and the maintenance of the running platform alongside the writing of this report.

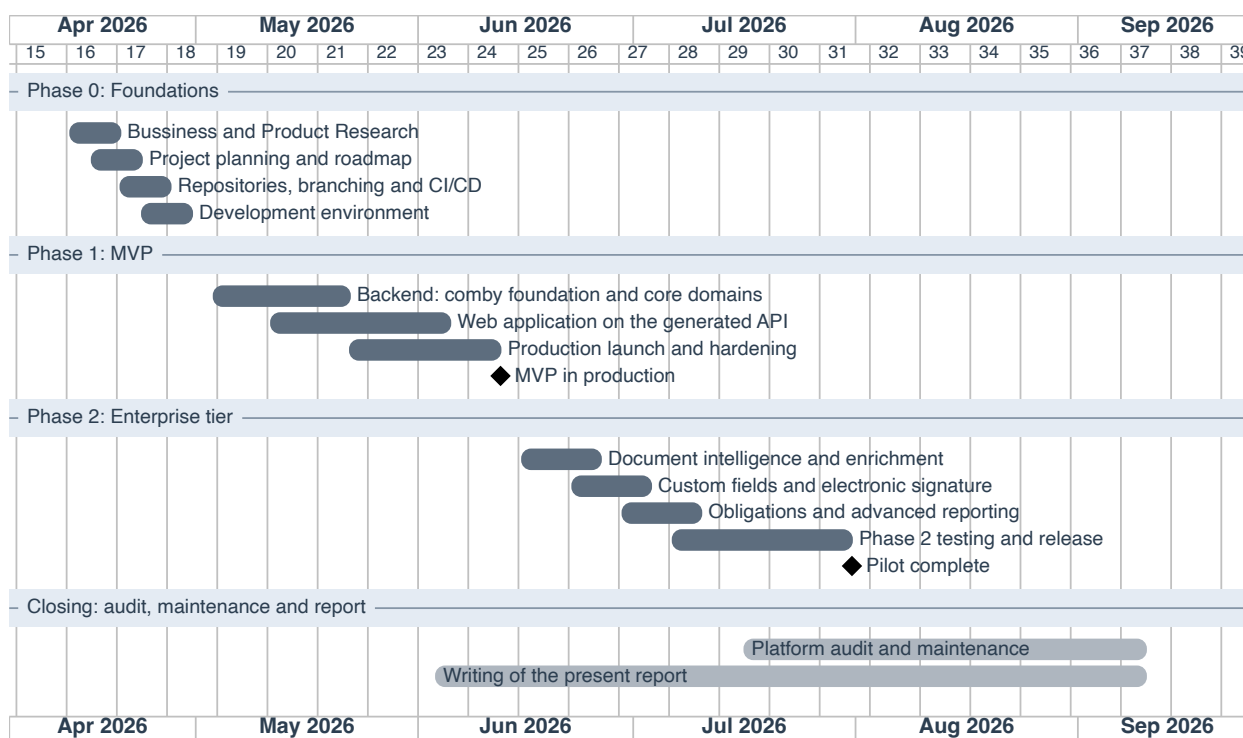


Figure I.3 – Project timeline: GANTT chart

Traceability is enforced by 3 artefacts. The roadmap is the single source of truth and is held in a Markdown document under version control. A spreadsheet derived from that document tracks progress at phase, section and subsection granularity, and is automatically regenerated from the Markdown to keep the two in lock-step. The Git history uses conventional commit messages, so that every change to the code is attached to the roadmap item that motivated it by a uniform prefix and a free-form description. Together these artefacts make it possible to answer, for any line of code in the repository, what it does, why it is there, and which phase of the project introduced it.

Conclusion

This chapter has framed the work that the remainder of the report describes. We placed CLMPilot inside Gradient Zero's portfolio of European mid-market products, characterised the DACH Mittelstand as a segment, and showed that the regulatory, operational and commercial pressures on that segment have turned contract lifecycle management into a pressing need. From that context we derived 4 operational pain points (missed deadlines, missing audit trails, absent portfolio overview, and unstructured approval flows) and answered them with the scope of an integration-first platform that manages a contract's lifecycle while its documents stay in the customer's own storage. Finally, we set out the Extreme Programming methodology under which the work was carried out, its adaptation to a solo build, and the release planning and traceability that supported it.

With the context and the scope established, Chapter [II](#) refines them into a specification: the actors the platform serves, the functional and non-functional requirements the solution must satisfy, and the use cases through which they are exercised.

Chapter II

Analysis and Requirements Specification

Introduction

This chapter specifies what CLMPilot is required to do. We start from the users: we first catalogue the actors who interact with the platform. From the problem statement and the actors we then derive the functional requirements, stated as capabilities, not implementation detail, and the non-functional requirements: the qualities the platform must exhibit while delivering them. We close by expanding the use cases the actors initiate, from a single high-level overview down to the individual cases of each functional area, with the most important scenarios described step by step.

II.1 Actors

The actor model is intentionally narrow. CLMPilot is a multi-tenant platform: each customer organisation is an isolated tenant, so the catalogue below centres on the roles *within* a single tenant, mapping onto the functions that already exist in a Mittelstand legal-operations team. Outside the tenant, a platform-level *Super Admin* provisions and isolates the tenants themselves; that role belongs to the operator rather than the customer, and lies outside the scope of this project's contribution, but it is catalogued here so the actor model is complete.

II.1.1 Actor catalogue

Table [II.1](#) catalogues the 4 per-tenant actors, together with the platform-level Super Admin, and the use cases each one initiates.

2 non-human actors complete the catalogue. The system clock is the trigger for the scheduled use cases of the deadline-management subsystem; we treat it separately to make the provenance of system-initiated actions explicit in the audit log. And the LLM (the intelligence service) is an external secondary actor: it supports the intelligence-assisted capture of contract metadata and clauses when a document is linked.

Table II.1 – Actors recognised by CLMPilot, with their primary use cases.

Actor	Primary use cases
Admin	Provision and decommission users; assign roles; configure organisation-wide policies; review the system audit log.
Legal Manager	Create and edit contract records; attach document links; initiate approval workflows; consult the dashboard and reports; receive deadline reminders for owned contracts.
Approver	Receive approval requests; approve, reject or request changes; consult the history of past approval decisions in scope.
Auditor	Read-only access to the full event log; export the audit trail; reconstruct the state of any contract at any past date within the retention window.
Super Admin	Provision and archive tenants; issue the first administrator invitation of a tenant; operate the platform across tenants. A platform-operator role, outside the scope of this project's contribution.

II.2 Functional requirements

The functional requirements that follow flow directly from the 4 pain points of the problem statement. They are stated as outcomes, not mechanisms: each subsection identifies the actor that initiates the use case, the outcome the platform must produce, and the constraints the surrounding context imposes. The architectural decomposition into domains, commands and events is deferred to Chapter III.

II.2.1 Contract registry

The contract registry is the platform's central object and the entry point for every other capability. The Legal Manager creates a contract record by supplying the counterparty, the contract category, the start and end dates, the notice window and the contract value. The same actor edits these attributes over the contract's lifetime and decommissions the record by archiving rather than deleting it: the immutability requirement of Section II.3.5 forbids hard deletion. The registry supports filtering by status, category, business unit and counterparty, full-text search on free-form fields, and pagination over the result set. Performance targets for these operations are stated in Section II.3.4.

II.2.2 Document linking

A contract record is connected to the document that embodies it through a *link* – a typed reference to a file held in the customer's own document store (Google Drive, SharePoint, or any HTTP-addressable location). The Legal Manager attaches and detaches links; the platform resolves them at display time, showing the file name, last-modified timestamp and a click-through

to the original location. The platform never copies the document contents into its own storage: the integration-first stance of Section [I.2.3](#) is enforced at this boundary.

II.2.3 Deadline and renewal management

Every contract carries up to 3 temporal markers: its expiry date, the start of its notice window, and, where applicable, its auto-renewal date. The platform must highlight each of these markers before it passes, through a reminder workflow that emits notifications at 90, 60 and 30 days before the notice window opens, and again on the day the window closes. The notification target is the contract's designated owner, by default the Legal Manager who created the record. The reminder schedule is generated by the platform itself: the actor who triggers the use case is the system clock, not a human user.

II.2.4 Approval workflows

A contract under negotiation moves through a sequence of approvals before it is signed. The Legal Manager initiates an approval request and selects a workflow template: a sequence of approval steps, each addressed to one or several Approvers. Each Approver receives a request and answers with one of 3 decisions: approve, reject, or request changes. A reject or request-changes outcome interrupts the workflow and returns the contract to the initiator; an approval advances it to the next step. The platform records every decision, its author, its timestamp and any comment, and these records are not editable after the fact.

II.2.5 Compliance and audit trail

Every action that changes the state of a contract (creation, edit, link attachment, approval decision, archiving) is recorded in an immutable audit log. The Auditor accesses the log read-only and exports it in a format independent of the application: a tabular file usable in spreadsheet software, with one row per recorded action, and columns for the actor, the timestamp, the contract identifier and the action type. The log must satisfy the GoBD retention requirement of 10 years and must be reproducible at any date within that window, even after the underlying business state has changed.

II.2.6 Dashboard and reporting

The dashboard is the Legal Manager's daily view. It shows the portfolio at a glance: counts of active, expired and pending contracts; the contracts whose notice windows open in the next 30,

60 and 90 days; the approval requests awaiting the user's decision; and the aggregate value of the active portfolio. Beyond the standing dashboard, the platform supports parametric reports (contracts by counterparty, spend by quarter, renewals by business unit) to answer the portfolio questions raised in the problem statement.

II.2.7 Identity and access management

Identity and access management (IAM) gates every other capability. The Admin creates user accounts, assigns roles and revokes access; users authenticate with an email address, a password and a second factor delivered as a one-time code by email. The platform enforces role-based access control along the actor catalogue of Section II.1.1: the same query can return different projections depending on the caller's role, and any command that exceeds the caller's authority is refused before it reaches the domain. The reset, invitation and logout flows are required at this layer because they are the flows through which non-Admin users actually exercise IAM.

II.2.8 Advanced capabilities extending the core registry

Beyond the core registry, the platform is required to support a tier of enterprise capabilities that address the same 4 pain points more fully. Document linking is expected to trigger *intelligence-assisted capture*: when a contract's document is attached, the platform proposes counterparty, dates, value and detected clauses for the Legal Manager to confirm, rather than leaving every field to manual entry. The platform must take a contract through *electronic signature* with an external counterparty and reflect the signed outcome on the record, so that execution is part of the tracked lifecycle. *Per-tenant custom fields* let each organisation extend the contract schema to its own taxonomy without a code change. *Post-signature obligation tracking* extends standard deadline management so the organisation tracks the commitments a signed contract creates (payments, reviews, renewals) for as long as it runs. And *advanced, schedulable reporting* answers the portfolio questions of the problem statement on a recurring basis rather than on demand. These capabilities are stated here as requirements; how they were built is the subject of Chapter IV, and Chapter V shows them in the delivered product.

II.3 Non-functional requirements

Non-functional requirements describe the qualities the platform must exhibit while delivering the functional capabilities of the previous section. We group them along 7 axes (security, compliance, availability, performance, auditability, internationalisation and maintainability), each justified

by a constraint of the deployment context.

II.3.1 Security

All traffic between the client and the platform is carried over Transport-Layer Security (TLS) in its current widely-deployed version; data at rest in the event store and the relational projections is encrypted with a key under the operator's control. Every command and query is authenticated; authorisation is enforced before the request reaches the domain layer, on the basis of the caller's role. Passwords are stored as salted hashes computed with a modern key-derivation function. A second factor of authentication, a one-time code delivered by email, is mandatory for Admin accounts and offered to every other role.

II.3.2 Compliance and data residency

The platform must satisfy 3 regulatory baselines simultaneously. GoBD requires that contract records be retained immutably and recoverable in their state as of any past date, for a period of up to 10 years. The GDPR requires that personal data be processed under documented purposes, subject to access and erasure requests, and stored within the European Union. ISO 27001 imposes a documented information-security management system, with access controls, change tracking and incident response procedures. The production environment is hosted in the European Union by default, under a contract with an EU-resident provider; no customer data is processed or stored outside the EU without an explicit opt-in.

II.3.3 Availability and reliability

The minimum viable product (MVP) carries an availability target rather than a contractually committed Service-Level Objective (SLO): 99.5% monthly during DACH business hours and 99.0% outside them, to be validated against operational data once production traffic is observed. Recovery-Point Objective (RPO) is set at 24 hours, sustained by nightly off-site backups; Recovery-Time Objective (RTO) at 4 hours. These figures are MVP targets and will be tightened or relaxed based on customer commitments in later phases. The platform degrades gracefully when external document stores are unreachable: contract metadata and workflow remain available, and document previews show a transient-error state rather than failing the surrounding page.

II.3.4 Performance

The performance budget for the MVP is expressed as engineering targets to be measured and tuned against, not as contractual guarantees. On a contemporary EU broadband connection, median page-load latency aims for under 1 second on the dashboard and the contract list; search and filter operations over a portfolio of 10,000 contracts aim for under 500 milliseconds; and document-link resolution (the round-trip to the external store, plus the metadata fetch) aims for under 2 seconds in the median case. Failure modes degrade to the graceful state described above. Table II.2 states these budgets as the metrics against which the performance tests measure the platform; each row can be exercised by an automated test against a seeded environment. The availability and recovery objectives of the previous subsection are not testable this way: they are validated operationally, through the uptime probe and the periodic restore drill.

Table II.2 – Performance test metrics: the measured operation, the metric observed, the test condition, and the MVP target.

Operation under test	Metric	Condition	Target
Dashboard page load	Median latency	EU broadband client	< 1 s
Contract-list page load	Median latency	EU broadband client	< 1 s
Search / filter over the registry	Latency	10,000-contract portfolio	< 500 ms
Document-link resolution	Median latency	external store round-trip	< 2 s

II.3.5 Auditability

Every state-changing action in the platform is recorded as an immutable, time-stamped, actor-attributed event. There is no primitive for in-place update or destructive delete at the storage layer: a correction is itself an event, layered on top of the prior history. The audit log is exportable in a tabular format independent of the application’s runtime and is reconstructible at any point in time within the GoBD retention window. This requirement aligns directly with the principle of compliance by architecture and drives the architectural choices presented in Chapter III.

II.3.6 Internationalisation

English is the primary language of the user interface, with German maintained alongside it; other languages can be added later, with translation slots reserved in the user-interface layer from the outset. Dates, numbers and currencies are formatted according to the user’s declared locale rather than the server’s, and the platform stores time-stamps in Coordinated Universal Time (UTC) and renders them in the user’s time zone. Email notifications inherit the recipient’s locale.

II.3.7 Maintainability

The platform was built by a single developer during a brief internship, so every architectural choice had to minimise the long-term maintenance burden. The platform relies on a narrow, well-understood technology stack rather than a fragmented mix of specialised tools. To keep the project manageable for future developers, it incorporates conventional commit messages, a predictable release-train cadence (Section I.5), and 3 distinct layers of automated testing (unit, integration, and end-to-end). Finally, documentation is stored directly alongside the code and compiled straight into the running application’s help system, preventing the documentation from drifting out of date.

II.4 Use cases

The requirements above are exercised through the use cases the actors initiate. This section expands them from a single high-level overview down to the individual cases of each functional area, and closes with the most critical scenarios described step by step. Where a diagram contains actors outside the scope of this project’s contribution (the platform-level Super Admin and the default *User* parent supplied by the framework), the actors carry a colour code for identification: red marks the out-of-scope actors, yellow the actors within scope.

II.4.1 Use-case diagram

Figure II.1 gives a high-level view of the platform: each use case inside the system boundary names one functional area, and every actor is linked to the areas it drives. The 4 human roles specialise a common *User* and inherit the shared authentication case from it, and the platform-level *Super Admin* administers the tenants themselves. 2 secondary actors complete the picture: the *System clock* drives the scheduled deadline and renewal reminders, and the *LLM* (the intelligence service) supports the intelligence-assisted capture of contract data. The individual use cases inside each area, with their relationships and constraints, are expanded one subsystem per figure in Figures II.3–II.6.

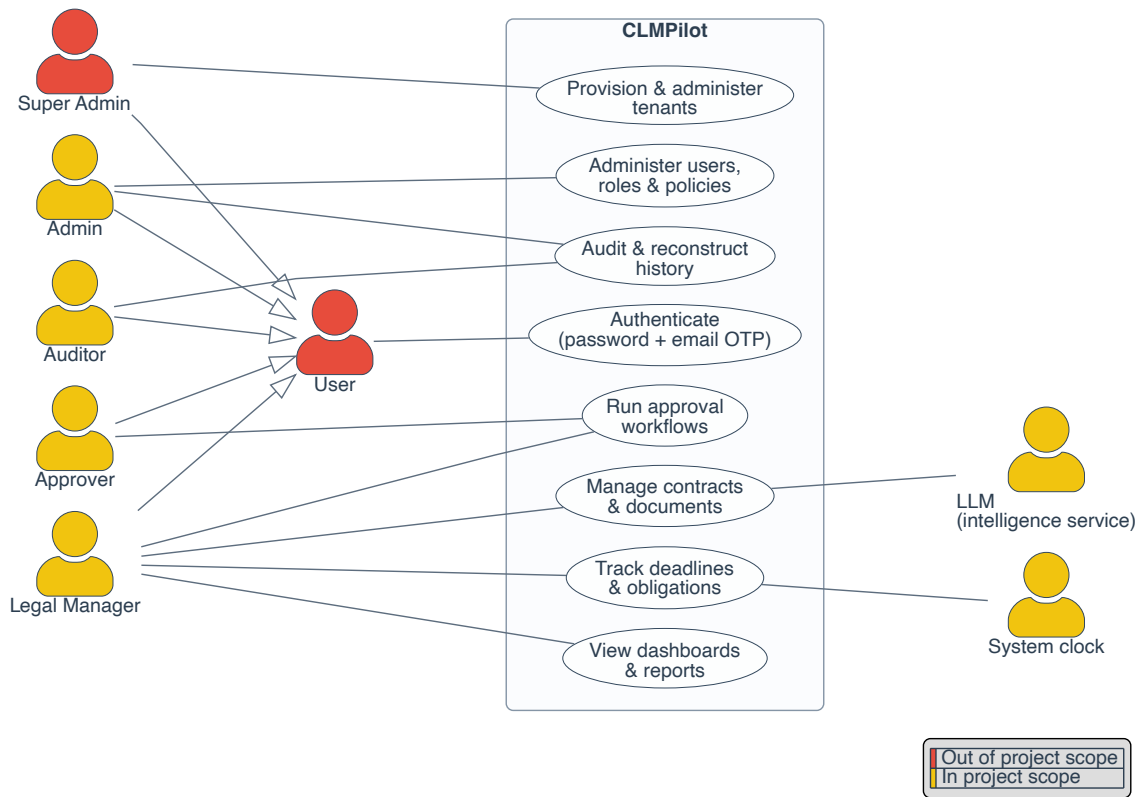


Figure II.1 – High-level use-case diagram for CLMPilot

II.4.2 Authentication use cases

Authentication is deliberately identical for every role: each human actor signs in through the same use case, which *includes* the verification of a second factor (a one-time code by email), mandatory for administrators and offered to every role; password reset and invitation acceptance follow the same shared rails. Figure II.2 makes that symmetry explicit: a single *Client User* actor stands for the 4 tenant roles (Admin, Legal Manager, Approver and Auditor), which all share the same authentication cases with the Super Admin.

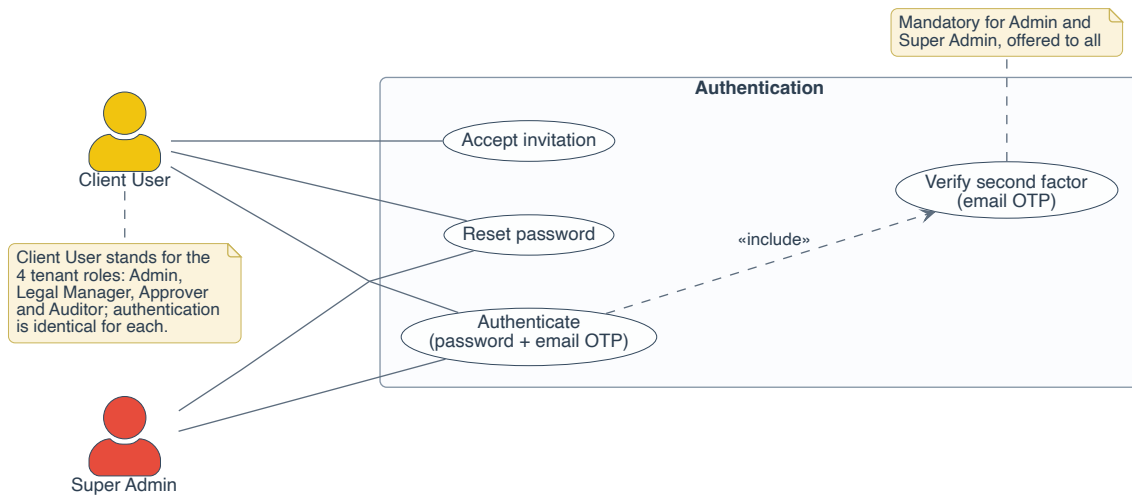


Figure II.2 – Detailed use cases: authentication

II.4.3 Use cases by functional area

Each functional area is then expanded into its individual use cases. Identity and access (Figure II.3) covers the administration of identities: the Admin provisions users, invites them, assigns roles, revokes access, and configures organisation-wide policies and custom fields; the authentication cases shared by every role were expanded in Figure II.2.

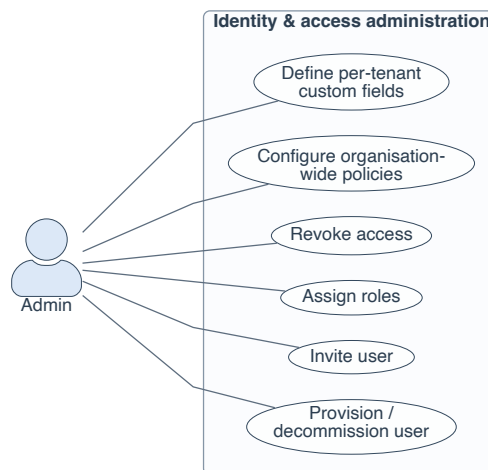


Figure II.3 – Detailed use cases: identity and access administration

Contracts and documents (Figure II.4) covers the registry and its document links: archiving never hard-deletes a record, and intelligence-assisted capture, supported by the LLM, extends the attachment of a document link.

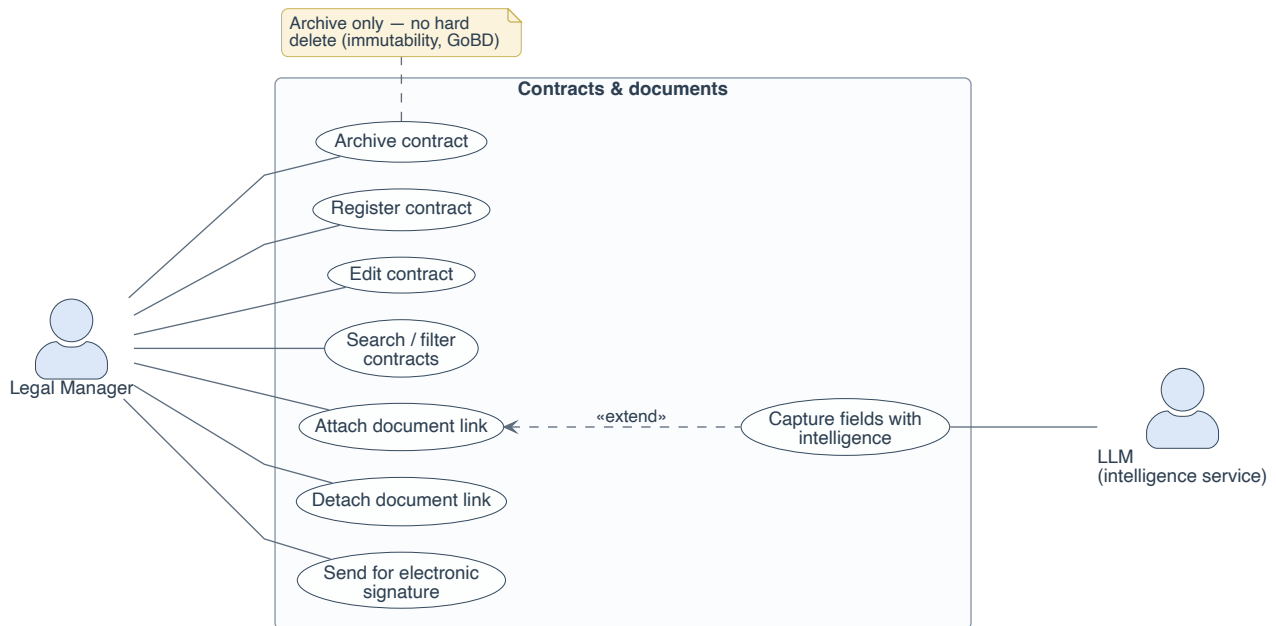


Figure II.4 – Detailed use cases: contracts and documents

Approvals and deadlines (Figure II.5) combine the workflow (where initiating an approval includes defining the workflow template, and a reject or request-changes outcome returns the contract to its initiator) with the time-driven reminders the system clock issues before each notice window.

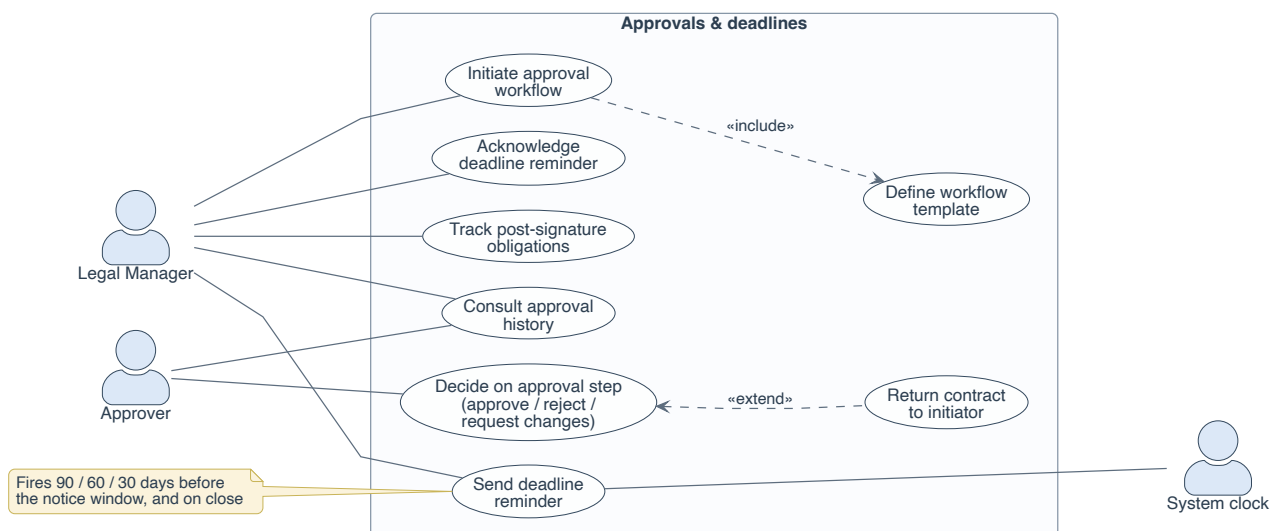


Figure II.5 – Detailed use cases: approvals and deadlines

Dashboard, reporting and audit (Figure II.6) groups the portfolio views and reports, where

scheduled reporting extends report generation, with the read-only audit capabilities through which any past contract state can be reconstructed within the 10-year retention window.

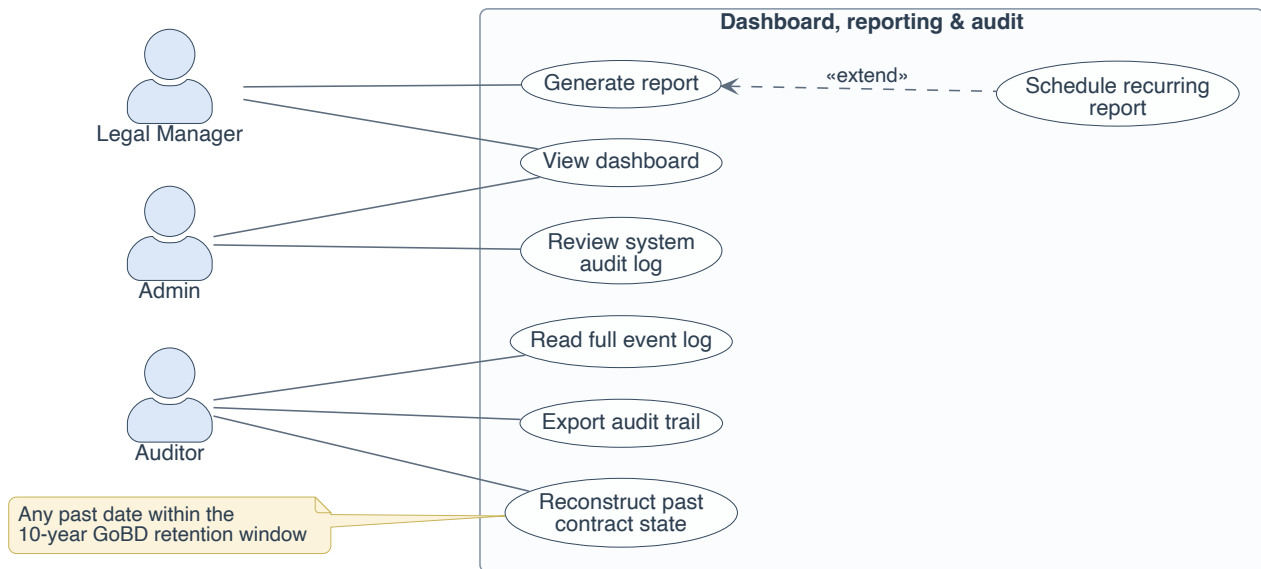


Figure II.6 – Detailed use cases: dashboard, reporting and audit

Across these areas, the enterprise capabilities of Section II.2.8 (intelligence-assisted capture, electronic signature, post-signature obligation tracking, scheduled reporting and per-tenant custom fields) extend the core registry rather than replacing it.

II.4.4 Detailed use-case descriptions

The most critical use cases to the platform are: the creation of a contract record, the approval workflow, and the reconstruction of past contract state for an audit. Tables II.3–II.5 describe each of them in the structure of a standard use-case description (goal, actors, preconditions, main scenario, alternative and exception flows, and postconditions). Table II.3 opens with the registration of a contract, the first step before any other capability.

Table II.3 – Use-case description: register a contract.

Goal	Register a new contract as a structured record in the organisation’s contract registry.
Actor(s)	Legal Manager (primary).
Preconditions	The Legal Manager is authenticated in their tenant and holds a role authorised to create contract records.

Main scenario	1. The Legal Manager opens the contract registry and starts a new record. 2. The Legal Manager supplies the counterparty, the contract category, the start and end dates, the notice window and the contract value. 3. The Legal Manager attaches a document link to the file that embodies the contract in the organisation's own storage («extend»: intelligence-assisted capture proposes counterparty, dates, value and detected clauses for confirmation). 4. The Legal Manager confirms the record; the platform validates it. 5. The platform creates the contract in <i>draft</i> status, records the creation in the audit log, and derives the contract's deadline reminders from its dates.
Alternative / exception flows	3a. No document is attached: the record is created without a link, and one can be attached later. 3b. The intelligence proposal is absent or wrong: the Legal Manager fills or corrects the fields manually. 4a. Validation fails (missing mandatory attribute, inconsistent dates): the platform rejects the submission and returns the Legal Manager to the form.
Postconditions	The contract exists in the registry in <i>draft</i> status; its creation is immutably recorded in the audit log; its deadline reminders are scheduled against its notice window and expiry.

Table II.4 then follows a contract through the approval chain it must complete before it becomes active.

Table II.4 – Use-case description: approve a contract.

Goal	Take a contract through an ordered chain of recorded approval decisions before it becomes active.
Actor(s)	Legal Manager (initiator); Approver (decides on the steps addressed to them).
Preconditions	The contract record exists and is not yet active; a workflow template describing the ordered approval steps is available; the Legal Manager is authorised to initiate approvals.
Main scenario	1. The Legal Manager initiates an approval request on the contract («include»: defining the workflow template that fixes the ordered steps and their Approvers). 2. The platform opens the first step and notifies its Approver(s). 3. An Approver reviews the request and records the decision <i>approve</i>, with an optional comment. 4. The platform records the decision (author, timestamp and comment) immutably, and advances the workflow to the next step. 5. Steps 2–4 repeat until every step is approved. 6. The workflow completes and the contract becomes <i>active</i>.
Alternative / exception flows	3a. The Approver records <i>reject</i>: the workflow is interrupted and the contract returns to its initiator («extend»: return contract to initiator), with the decision and its comment recorded. 3b. The Approver records <i>request changes</i>: as 3a, with the requested changes recorded for the initiator.

Postconditions	Every decision is immutably recorded with its author, timestamp and comment, and is not editable after the fact; on full approval the contract is active; on interruption the contract is back with its initiator and the workflow's record is preserved.
-----------------------	---

Finally, Table II.5 describes the audit consultation that compliance must support at any time.

Table II.5 – Use-case description: reconstruct past contract state for an audit.

Goal	Reconstruct the state of a contract as of any past date, and export the audit trail in a format independent of the application.
Actor(s)	Auditor (primary).
Preconditions	The Auditor is authenticated with read-only audit access; the requested date lies within the 10-year GoBD retention window.
Main scenario	1. The Auditor selects a contract and a past date of interest. 2. The platform reconstructs the contract's state as of that date from its immutable history, even where the current business state has since changed. 3. The platform presents the reconstructed state together with the actions recorded up to that date: one row per action, with the actor, the timestamp, the contract identifier and the action type. 4. The Auditor exports the audit trail as a tabular file usable in spreadsheet software.
Alternative / exception flows	<i>1a.</i> The Auditor consults the full event log across the portfolio instead of a single contract: the same read-only view and export apply to every recorded action in scope.
Postconditions	The Auditor holds a reproducible view of the contract as of the requested date and, where exported, a tabular audit file independent of the application's runtime; no record was altered by the consultation.

Conclusion

This chapter has specified what the platform must do and how well it must do it. We catalogued the actors: 4 per-tenant human roles specialising a common user, the platform-level Super Admin outside this project's scope, and the system clock and the LLM as non-human actors. From the problem statement we derived the functional requirements, from the core contract registry to the tier of enterprise capabilities, and the 7 axes of non-functional requirements that constrain how they must be delivered. We then expanded the use cases the actors initiate from a single high-level overview down to the individual cases of each functional area, describing the contract-creation, approval and audit-reconstruction scenarios step by step.

With the requirements specified, Chapter III presents the structural response: the architecture through which these requirements are satisfied by construction.

Chapter III

Architecture and Design

Introduction

This chapter details the architecture of CLMPilot: a structure designed to satisfy the platform’s requirements by construction. Instead of assembling the design feature by feature, we let a few core forces drive the system’s foundational choices.

3 primary forces drive the design. **Integration-first**: contract documents remain entirely within the customer’s own storage, meaning the platform handles only their lifecycle management. **Compliance by architecture**: the storage model natively provides the immutable, reconstructable audit trails demanded by GoBD, GDPR, and ISO 27001. **Multi-tenant isolation**: the architecture ensures that a single deployment can securely serve multiple customer organisations, keeping each tenant’s data strictly isolated.

III.1 Architectural drivers

To solve the problem set out in Chapter II and shape the solution, we did start from the requirements where any acceptable design must be satisfied, and then promoted to *drivers* that arbitrate every architectural decision in this chapter. The non-functional requirements thus become 5 drivers. Each is a constraint any acceptable design must meet, and each reappears in the later chapters as the reason for a delivery-time decision.

Auditability and immutability. GoBD requires contract records to be kept immutably and recoverable as of any past date for up to 10 years; ISO 27001 adds change tracking (Section II.3.5); this is the strongest of the 5 drivers, and it rules out any design in which history can be mutated, or in which the audit trail is reconstructed after the fact; what happened is the primary record, not a by-product of the current state.

Integration-first. A contract’s document stays in cloud storage or any URL; the platform stores a reference, never the contents (Section I.2.3). So the architecture needs an explicit integration boundary with interchangeable providers, and it must keep working when a store is unreachable. This driver is fixed: it is the product’s differentiator.

EU data residency. The GDPR requires personal data to stay in the European Union (Section II.3.2). The stack hosts in the EU by default, and every external dependency (email,

III.2 Why CQRS and Event Sourcing for CLM?

the document stores, the intelligence service) counts as a data-processing boundary.

Multi-tenant isolation. One deployment, many tenants (Section II.1). Every command and query carries a tenant context, and cross-tenant access is blocked by construction. Isolation that depends on every query filtering by tenant will eventually be broken by one that does not.

Maintainability under solo development. One engineer builds and runs the platform (Section I.5). This driver selects against complexity: a narrow stack, conventional patterns, automation over manual work. Every added component adds to the load on a single engineer.

Availability and performance (Section II.3.4) matter too, but at MVP scale they constrain operations more than structure. Chapter V takes them up where they can be measured.

III.2 Why CQRS and Event Sourcing for CLM?

The drivers of Section III.1 now have to select the platform’s foundational pattern: how state is stored, changed and queried. Because that choice is the hardest one to revisit later, we make it explicitly rather than by habit: we fix the selection criteria, present the candidate architectures, and justify the choice (Section III.2.1); the retained pattern is then presented in detail (Section III.2.2).

III.2.1 Architectural choice

4 criteria follow from the drivers. *Compliance-grade history*, decisive: an immutable, attributable record from which any past state can be rebuilt. *Domain fit*: reads dominate, writes are rare and pass multi-step validation. *Tenant isolation by construction*: every record and every access carries its tenant.

Table III.1 sets the established architectural style against the one retained, on each of the 3 dimensions of the choice.

Table III.1 – Established architectural style opposed on each dimension of the foundational choice.

Dimension	Established style	Style retained
State persistence	Current-state persistence, the default of the classic 3-tier application [9]; history bolted on as a shadow-table audit log, a second write that can drift from the first.	Event Sourcing: the append-only event store is the record [1, 3], current state a derivation, any past state a replay.
Domain modelling	Layered (N-tier) architecture with Active Record / ORM mapping [9]: one entity set serves queries and mutations alike.	CQRS: commands enforcing invariants and purpose-built read projections are separate pipelines [1, 2].
Communication	Request-response (REST/RPC) over a shared ACID database: synchronous, immediately consistent.	Event-driven: projections consume events asynchronously; eventual consistency on the read side.

III.2 Why CQRS and Event Sourcing for CLM?

Two packaged architectures cut across the 3 dimensions: a document-oriented store, current-state with a flexible schema, and a workflow (BPM) engine, which models the approval process rather than the contract.

The first criterion settles it. Current-state persistence cannot rebuild a past state, and a shadow-table trail can be edited or diverge from the data it describes (Section II.3.5). A document store inherits that and has no transaction for a multi-step approval; a BPM engine adds a heavy runtime and vendor lock-in, and still leaves the trail unsolved. Event Sourcing meets the criterion by construction, CQRS fits the read-heavy workload, and both carry a tenant context on every event and every query. The price is eventual consistency, long-lived event schemas and a learning curve, accepted because state can be re-derived later, a missing history cannot (Section III.2.2).

III.2.2 CQRS and Event Sourcing

A contract's life is a sequence of events: registered, document linked, approval granted or refused, deadline passed, terminated. These are facts that accumulate, not states that overwrite one another. *Event Sourcing* stores that sequence and derives the current state by replaying it, instead of mutating state in place [1, 3]. *CQRS* is its companion [1, 2]: it splits the write model (commands, which validate rules and emit events) from the read model (queries, which read purpose-built projections). The two are independent but reinforce each other [10, 11].

This pairing satisfies the auditability driver. Under Event Sourcing the immutable, time-stamped, actor-attributed record that GoBD and ISO 27001 demand (Section II.3.5) *is* the event log: there is no separate audit table to maintain or to fall out of sync. *Who approved this contract, when, and against which version* is answered by replaying its events.

CQRS also fits the workload. Reads dominate and fetch whole aggregates: the dashboard counts expiring contracts and sums value by quarter. Writes are rarer and guarded: an approval or an edit must pass validation. Splitting them lets each read model be its own projection [9], so a heavy query never blocks a write, and it expresses the domain in the terms of Domain-Driven Design [12, 13]. Figure III.1 shows both paths.

III.2 Why CQRS and Event Sourcing for CLM?

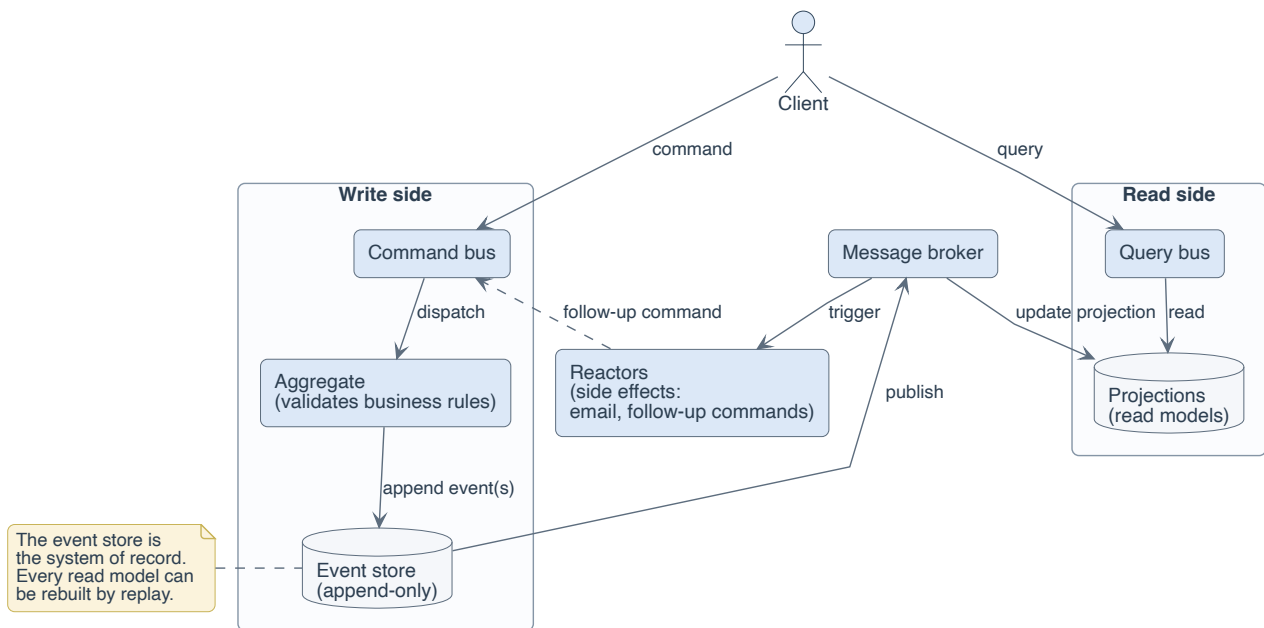


Figure III.1 – The CQRS and Event Sourcing flow

Figure III.2 shows the same flow in time. The command path is short and synchronous: validate, append an event, answer the caller. The reactions come after, once the caller already holds its answer: a projection updates the read models, and a reactor turns the new event into a follow-up command. That is the reactor pattern, and the point where eventual consistency becomes visible.

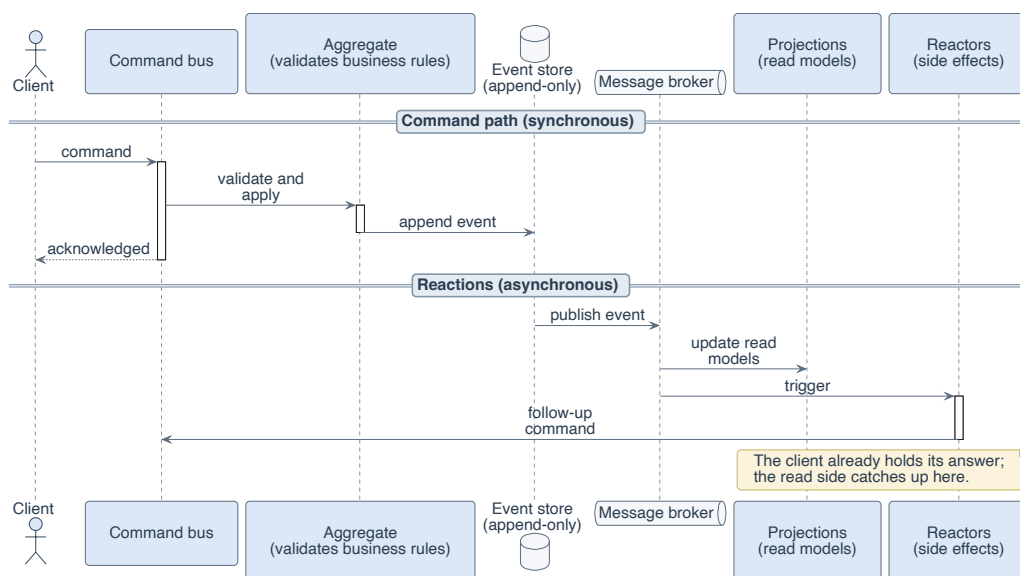


Figure III.2 – The command path and its reactions, in time

The costs anticipated in Section III.2.1 are now concrete. Read models are eventually consistent: a new contract may show in its detail view a moment before it shows in a dashboard count, which for CLM is acceptable. Events are a long-lived schema, so they need a versioning convention. And the pattern has a learning curve, mitigated here because an existing framework can supply the mechanics; the choice of that framework is the subject of the next section.

III.3 The comby framework

The framework that carries the patterns of Section III.2 did not appear for this project: it comes out of Gradient Zero's own history. The company builds compliance-first products for regulated sectors (Section I.1.2), and each new product kept running into the same problems: an audit trail to guarantee, approval workflows to model, temporal queries to answer: infrastructure that would otherwise be reimplemented, or imported from an open-source project, once per product. Its response was to consolidate that recurring infrastructure into an in-house Go framework, maintained as a strategic asset and reused across the portfolio.

comby is that framework. It supplies the CQRS/ES building blocks (command and query buses, an append-only event store, a projection runner, a reactor scaffold, and a message broker) and a set of *defaults*: tenants, accounts and identities, group-based access control, invitations, asset storage, webhooks, and an auto-generated REST API with an OpenAPI specification. CLMPilot runs on comby v3.

III.3.1 Role in the architecture

CLMPilot is a thin layer of domain meaning over a thick layer of framework mechanics (Figure III.3). comby owns the *how*: dispatching and authorising a command, persisting and publishing an event, rebuilding a projection, isolating a tenant. CLMPilot owns the *what*: what a contract is, when an approval advances, how a deadline raises reminders. That split is the maintainability driver in practice.

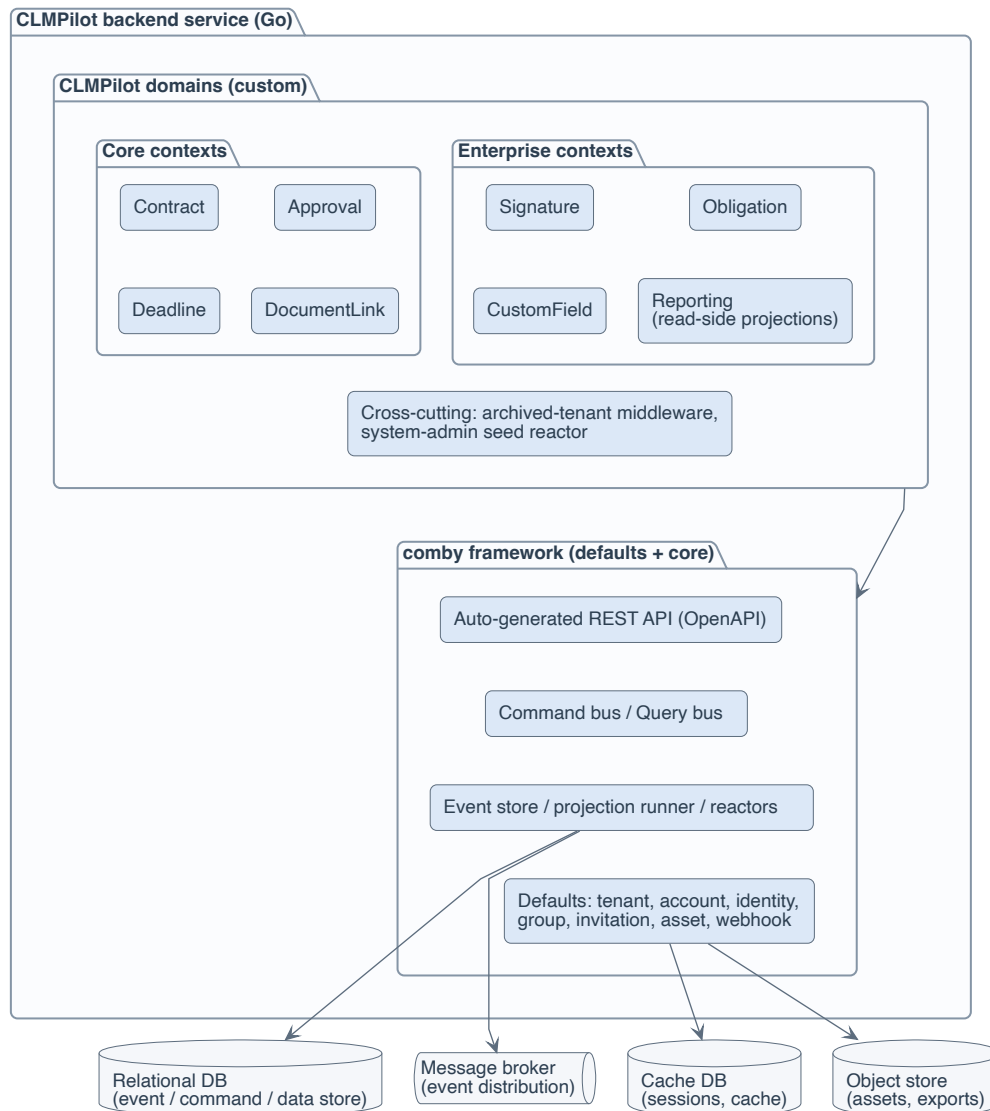


Figure III.3 – Layering of CLMPilot on comby.

What CLMPilot inherits is large: tenancy, accounts, identities, groups, invitations, asset storage, and the whole authentication layer. The REST API and its OpenAPI document are generated from the registered commands and queries, so the frontend uses a generated, type-safe client. What CLMPilot adds is small: the 4 core domains of Section III.4, the handful of enterprise contexts that extend them on the same pattern (Section III.4.4), and 2 cross-cutting pieces: a middleware that blocks archived tenants, and a boot-time reactor that seeds the system administrators. Figure III.4 shows the same split as packages: the domains register on comby, the satellites depend on Contract, and the integration and internal packages keep the external services and cross-cutting concerns apart.

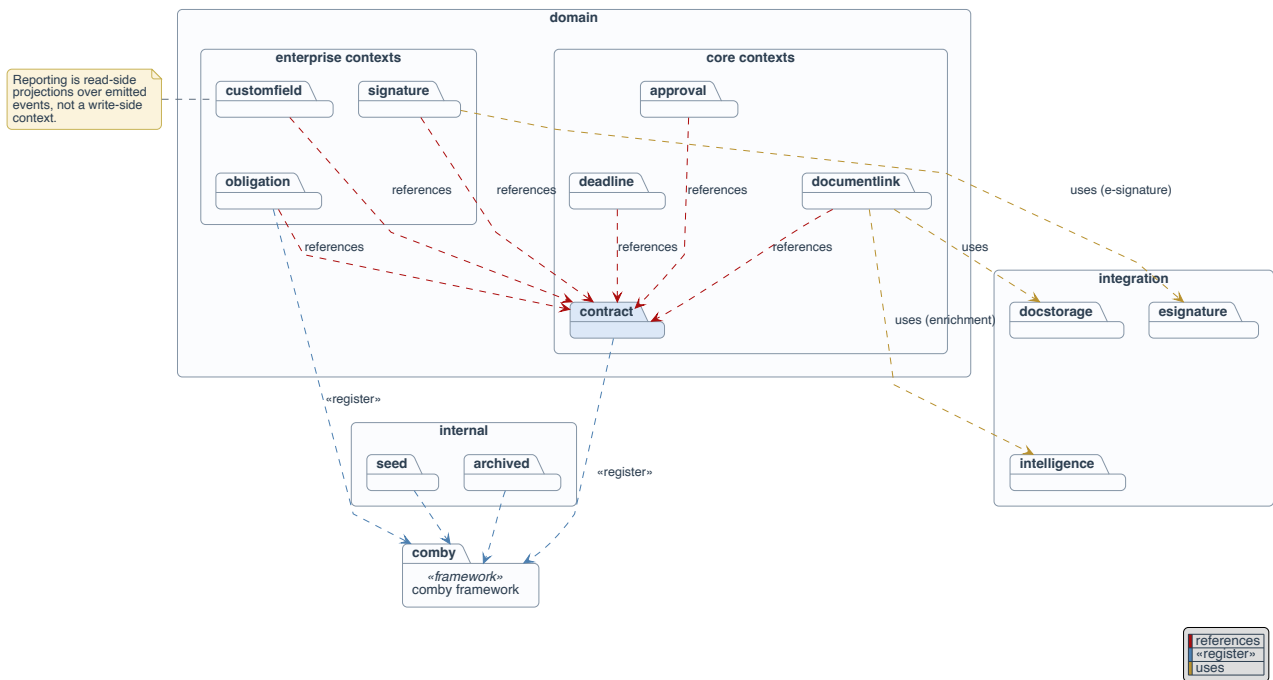


Figure III.4 – Packages structure of the backend

III.3.2 Justification of choice

Choosing comby is a build-versus-buy decision, settled by fit rather than feature count. comby is a strategic asset of Gradient Zero, not a convenience library (Section I.1): CLMPilot exists partly to exercise it, so its authors are on hand to support it. Table III.2 sets it against the alternatives that were considered.

Table III.2 – CQRS and Event Sourcing platforms considered for CLMPilot, assessed against the project’s decisive criteria.

Option	Ecosystem	CQRS/ES core & built-in tenancy, identity, RBAC	Support and strategic fit
comby v3	Go	Command/query buses, append-only event store, projection runner and reactors, <i>plus</i> tenant, account, identity, group and invitation defaults out of the box.	Authored in-house (Section I.1.2); the framework team supports the product directly; reused across the company’s portfolio.

III.4 Domain model overview

Option	Ecosystem	CQRS/ES core & built-in tenancy, identity, RBAC	Support and strategic fit
EventStoreDB + Go [14]	Go	A mature event store only; the command and query buses, projection wiring, multi-tenancy, identity and RBAC must all be hand-built.	External product; the entire application framework remains the project's burden.
Axon Framework [15]	Java/JVM	Mature CQRS and Event Sourcing, but no multi-tenant identity layer; would require a JVM service.	Open-source; adopting it would split the stack across 2 languages.
EventFlow [16]	.NET	CQRS/ES primitives for the .NET ecosystem; tenancy and identity not provided.	Open-source, but the wrong ecosystem for a Go house.
Bespoke (in Go)	Go	Full control over every primitive, at the cost of re-implementing all of the above.	No external dependency, but unmaintainable by one engineer alongside the product itself.

3 points decide it: comby is Go-native, so the backend stays one language. Its CQRS/ES primitives are proven, so they need not be rebuilt. And its defaults supply the biggest saving: the tenancy, identity, RBAC and invitation logic that every other option would leave to the project. EventStoreDB and a bespoke build leave the whole framework to build; Axon and EventFlow are the wrong ecosystem.

III.4 Domain model overview

The requirements of Chapter II decompose into 4 core *bounded contexts*, each with its own aggregate, events and rules. We give them here at the conceptual level (responsibilities and relationships) and their realisation in Chapter IV. The enterprise capabilities of Section II.2.8 add a handful of further contexts that follow the same pattern; they are introduced at the end of this section so the core model stays readable.

III.4.1 Bounded contexts

Figure III.5 shows the 4 bounded contexts and how they relate. Contract is central; Approval, Deadline and DocumentLink each reference it. All 4 sit inside a tenant boundary: every instance belongs to one tenant, and no relationship crosses it. Identity and the audit log are absent here: they are platform concerns owned by the framework (Section III.3.1), not business contexts.

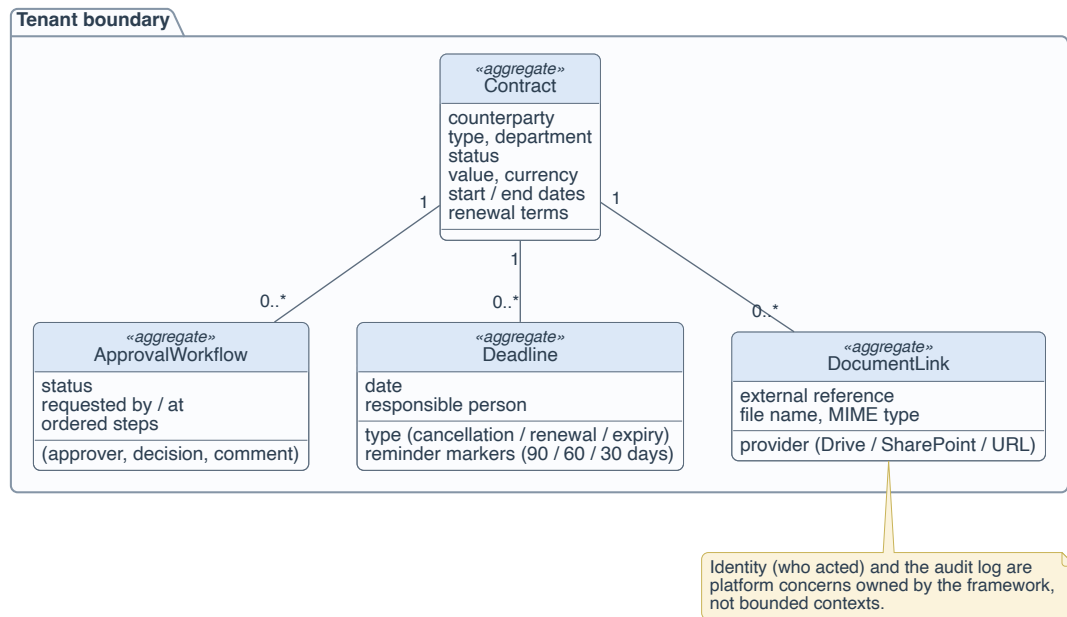


Figure III.5 – The 4 bounded contexts, within the tenant boundary

III.4.2 Aggregate responsibilities

Contract is the record of a contract’s metadata and status: counterparty, type, value, dates, renewal terms. It does not hold the document; that is a **DocumentLink**. Every other context points at it, and it anchors the audit trail.

Approval is the multi-step review a contract passes before it goes active. It is its own aggregate, not a status field on **Contract**. That gives the approval chain its own lifecycle and its own immutable record of who decided what, and when.

Deadline tracks cancellation windows, renewal and expiry dates, and the reminders before them. It is a first-class aggregate, so one contract can carry several independent deadlines. That is how the platform answers the missed-deadline problem: each deadline is an object the contract owns, not just a field on it.

DocumentLink is the integration boundary turned into a domain object. It records that a contract is embodied by a file in an external store (Drive, SharePoint, or any URL) without ingesting the file. One aggregate serves all providers, through provider-specific adapters.

III.4.3 Behavioural views: lifecycle and workflow

2 contexts are clearest through their behaviour. Figure III.6 is the **Contract** state machine. A contract starts as a *draft*, goes *active* once approved, may turn *expiring* near its end date and

then *expire*; an active contract can instead be *terminated* early. Both paths end in *archived*. Each transition is a command or a time-driven event, so the diagram is a compact summary of a contract's audit trail.

Soft delete (tombstone) and restore are available from any state and are omitted here for clarity.

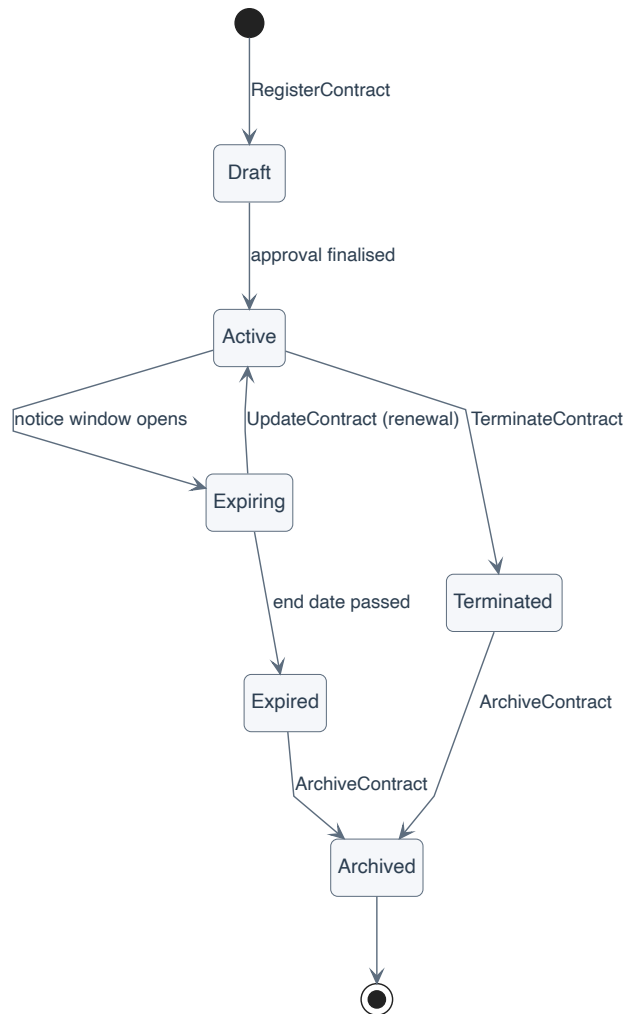


Figure III.6 – State machine of the Contract aggregate

Figure III.7 shows the approval workflow as activity. The Legal Manager starts it against a template of ordered steps. Each approver either advances it or interrupts it by rejecting or requesting changes; only when every step is approved does the contract go active. The diagram makes the key rule explicit: a single rejection returns the whole contract to its initiator.

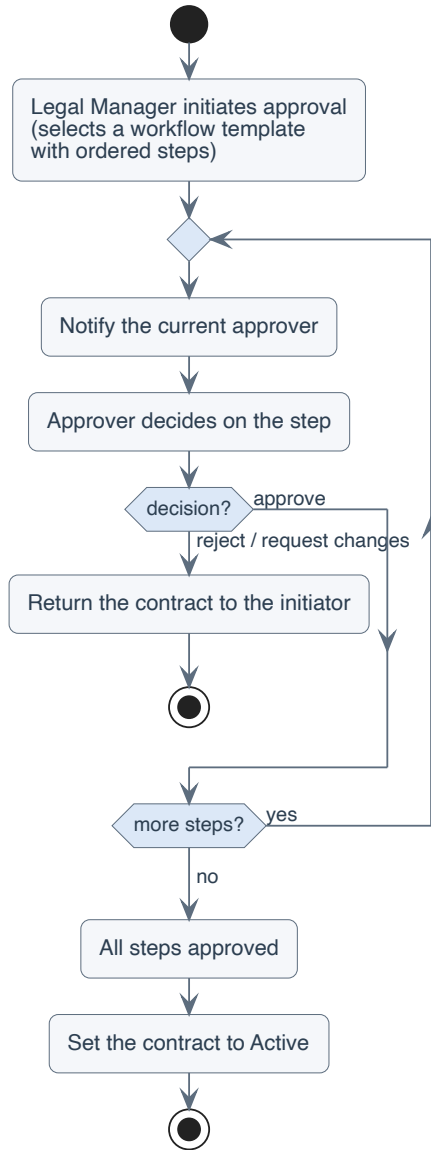


Figure III.7 – Activity diagram of the approval workflow

III.4.4 Extension contexts

The enterprise capabilities of Section II.2.8 did not change the core model; they added new contexts that follow the same aggregate-and-events discipline, referencing `Contract` exactly as the core satellites do.

Signature manages the electronic-signature lifecycle of a contract, tracking the envelope sent to external counterparties and individual signer outcomes. Modelled similarly to *Approval*, this is built as a distinct multi-step aggregate rather than a simple flag on `Contract`. **Obligation**

handles post-signature commitments and their recurrence, to track what the organisation is required to do once a contract goes active. **CustomField** enables per-tenant schema extensions, allowing organisations to record their own custom attributes against contracts without altering the core aggregate. **Reporting and analytics**, finally, are handled by a family of read projections driven by the events that other contexts already emit.

These capabilities arrived as new contexts and projections, not as edits to the 4 core aggregates: the decomposition let the enterprise features extend the model at its seams, without disturbing its core.

III.5 System topology

The structure has 2 views: a *component* view (which services exist, and through which interfaces) and a *deployment* view (where the code runs and the data is stored). Both describe the design; the operational detail (the host, the reverse proxy, the backups, the release process) is Chapter IV.

III.5.1 Component view

The services connect through a small set of named interfaces (Figure III.8). The backend provides one interface, the generated REST/OpenAPI contract, which the frontend consumes through its generated client. The backend requires 3: `IntelligenceClient` from the intelligence service, `Email` from the mail provider, and `DocumentStore` behind which the cloud storage adapters sit.

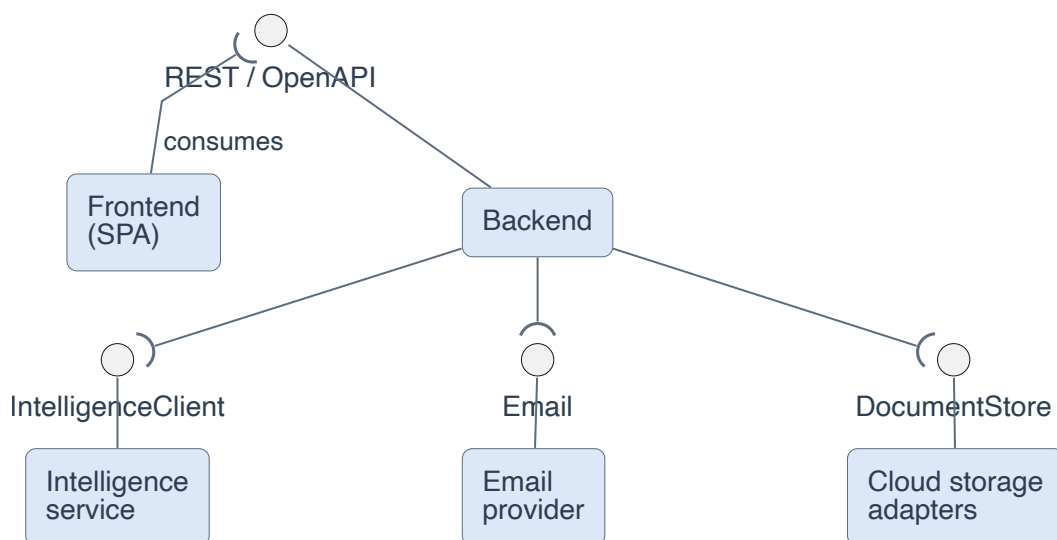


Figure III.8 – CLMPilot component view

III.5.2 Deployment topology

The deployment runs as a single EU-hosted, containerised stack (Figure III.9) of 3 runtime services: the **backend**, the **frontend** (a single-page application running in the user's browser) and the **intelligence** service (backed by a large language model). We name the parts by their role; the concrete products behind them are a realisation decision (Chapter IV). The services are backed by 4 stores. A **relational database** is the system of record (the event store and the command store) and the source every read model is rebuilt from. A **cache database** holds sessions and accelerates read models. A **message broker** carries events to projections and reactors and is designed to distribute them across backend instances when the platform scales horizontally. An **object store** keeps exports, signed documents and other assets. One reverse proxy terminates TLS in front of them.

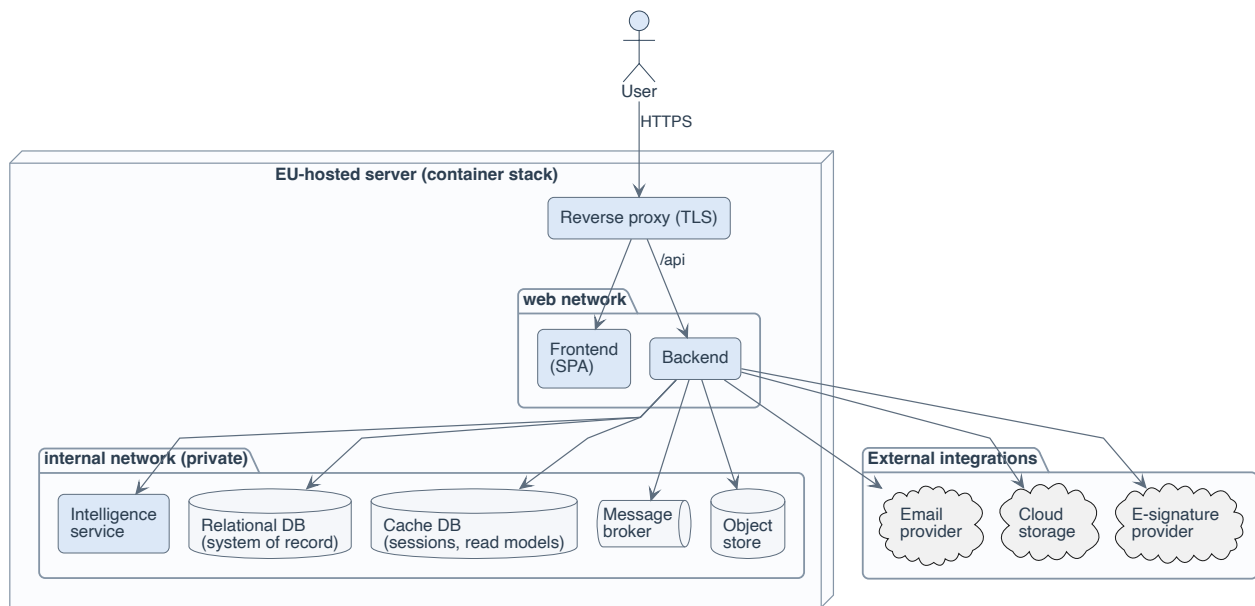


Figure III.9 – CLMPilot deployment topology

Network isolation is built into the topology: only the reverse proxy is reachable from outside, while the intelligence service and the stores are reachable only from the backend, so the most sensitive parts are not just access-controlled but unaddressable. Outward, the system depends on an email provider, on the users' own cloud storage for documents, and on an e-signature provider, each behind a named interface so the concrete provider is interchangeable; ERP integration is anticipated in the design but not yet built.

III.6 Security and compliance model

Security is a property of the architecture, not a subsystem beside it. The comby framework supplies the mechanisms (identity, access control and tenant isolation) and CLMPilot inherits it (Section III.3.1). We describe the identity and access model it provides, the layered pipeline that enforces that model on every request, the defence in depth that keeps tenants apart, and finally compliance, which largely restates structure already in place.

III.6.1 Identity and access model

The model has 4 levels from the comby defaults, shown in Figure III.10. A **tenant** is a customer organisation. An **account** is a person's login, which may hold an **identity** in more than one tenant: the same person can act in several organisations without sharing a password. A **group** is a role within a tenant (Legal, Finance, Admin); a **permission** is the right to run a given command or query. Access control is role-based and always scoped to a tenant.

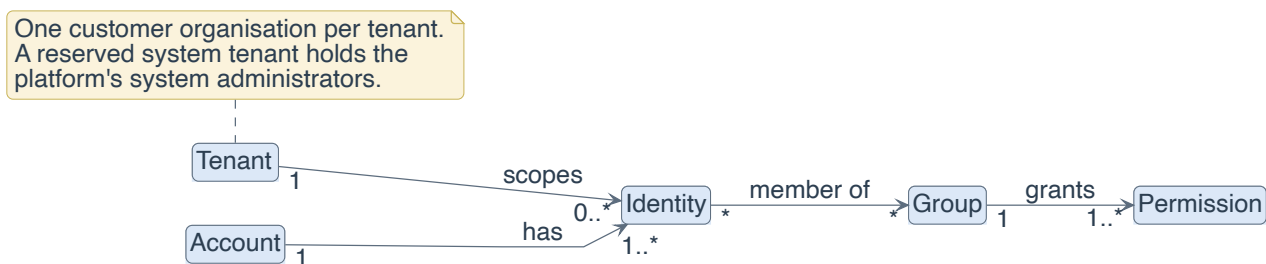


Figure III.10 – The identity and access model

Above the per-tenant model sits the platform **system administrator**: a small, seeded set of operator accounts in a reserved system tenant, able to provision, inspect and archive tenants through a cross-tenant interface.

III.6.2 Authentication and the request security pipeline

Authentication establishes who is calling, and does so once, at the edge. A user signs in with an email and a password (held only as a salted hash) and a second factor, a one-time code sent by email. What the browser then carries is a cookie backed by an opaque, server-side session, not a self-contained bearer token: the session holds no claims a client could read or replay, and it can be revoked at will. comby's account and session handling covers this.

From that point, authorisation is a stage every request passes through: the framework enforces it, so the application never has to invoke it explicitly. The framework resolves the

III.6 Security and compliance model

caller's identity and tenant once, from the session, then checks each command or query against the caller's permissions *before* it reaches a domain handler. Figure III.11 shows the 2 halves: identity is established at the edge, and authorisation runs at dispatch, within the caller's tenant. A request beyond the caller's authority, or one aimed at another tenant, is refused at the boundary; and because the tenant context is taken from the session rather than from the request, the ordinary API has no way to address another tenant's data at all.

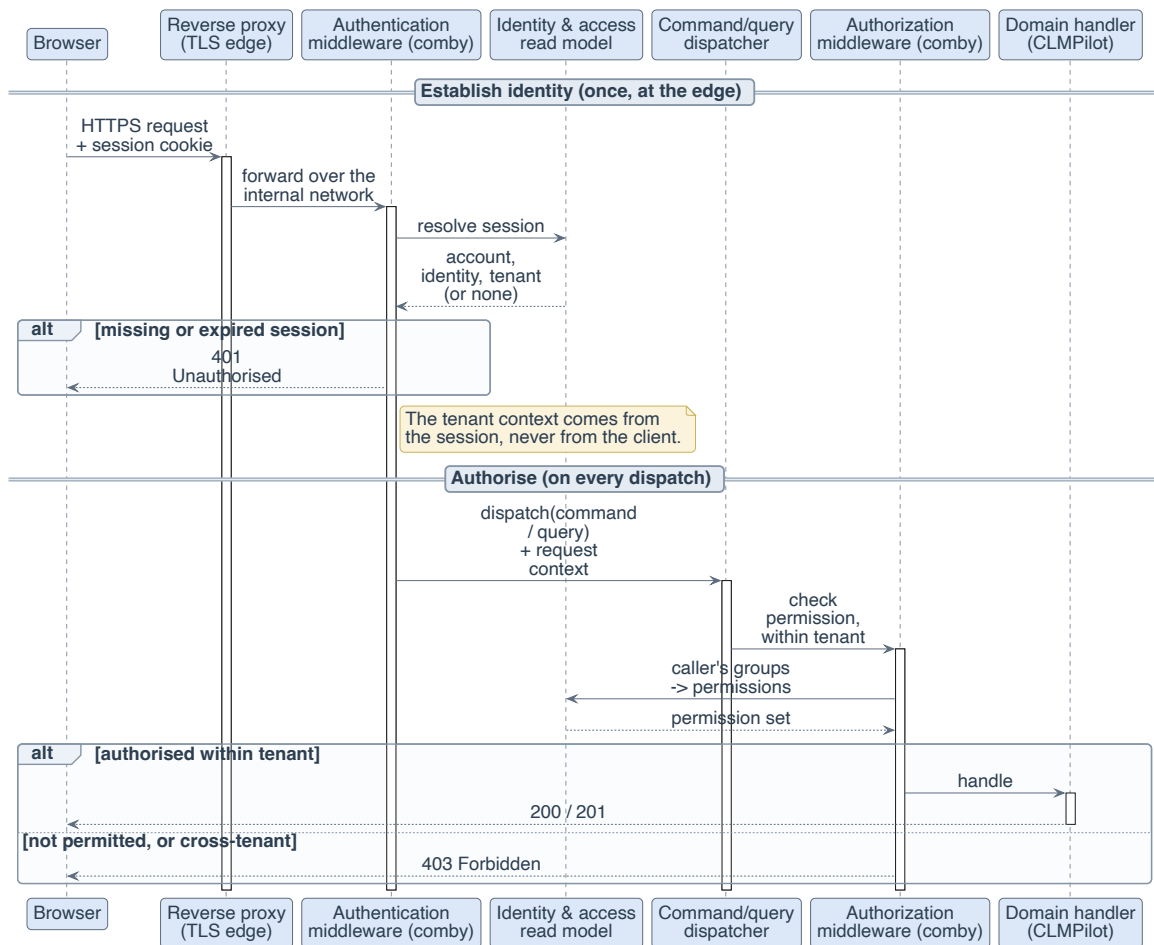


Figure III.11 – The request security pipeline

The authorisation check itself is a short, ordered cascade in which the first rule that resolves decides the outcome (Figure III.12). Internal operations (a seed at boot, a reactor acting on an event) run without a caller and are allowed; a command may carry a custom rule for a case that role membership alone cannot express; a platform administrator is allowed across tenants; any request whose target tenant differs from the caller's is denied outright; and otherwise access is granted only if one of the caller's tenant groups carries the matching permission. The framework offers a finer, workspace-scoped grant beneath the tenant, which CLMPilot does not yet exercise.

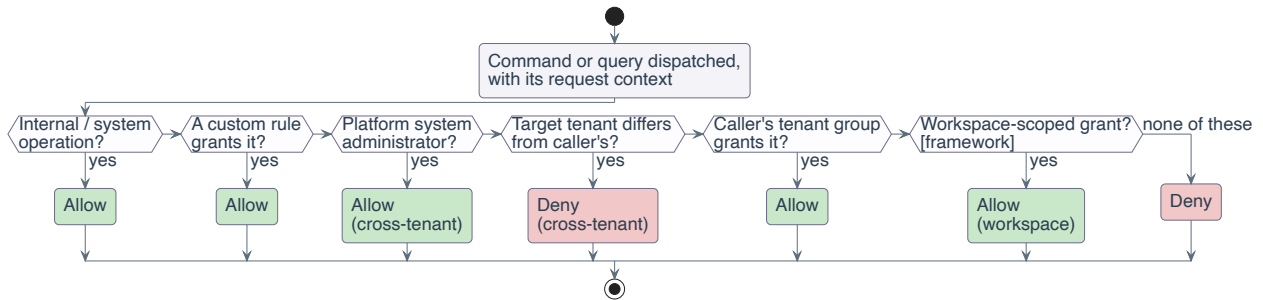


Figure III.12 – Authorisation process of commands and queries

III.6.3 Defence in depth and tenant isolation

No single control is trusted to be enough: the same request crosses several independent layers, each of which would hold if an outer one were bypassed (Figure III.13). The network perimeter exposes only the reverse proxy and keeps the stores on an internal-only network, unaddressable from outside (Section III.5.2). Authentication and per-tenant authorisation are the 2 layers just described. Beneath those 2 layers is the tenant isolation that all of the compliance guarantees depend on, and at the core the immutable event log, which serves as both the system of record and the audit trail.

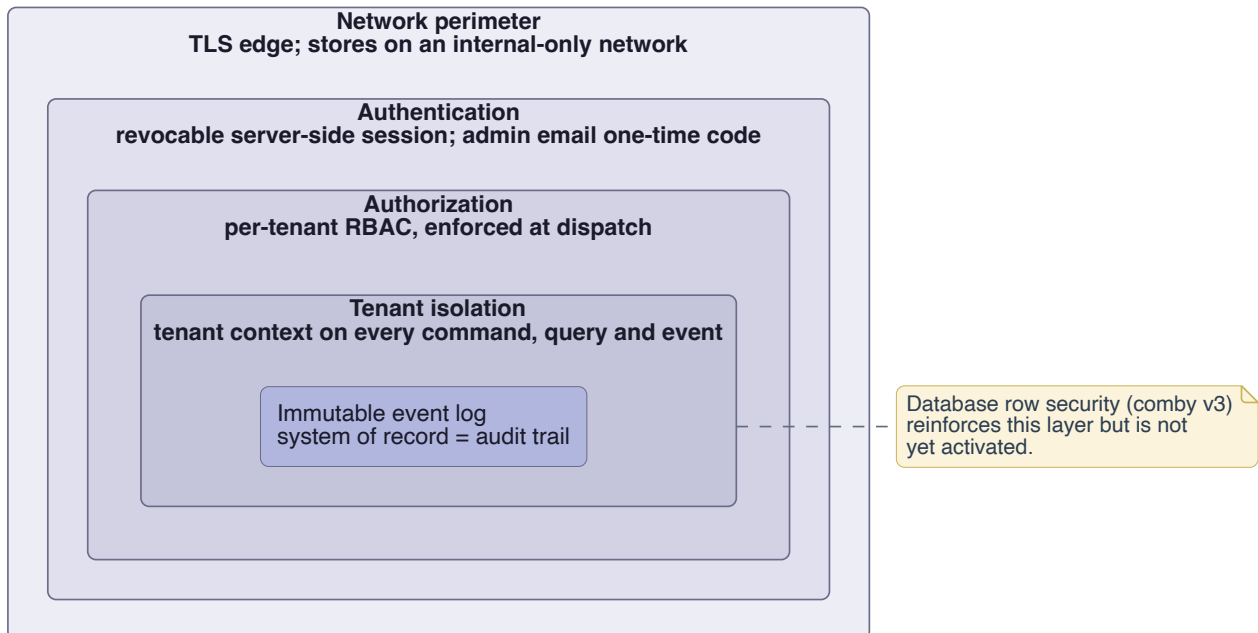


Figure III.13 – Defence in depth

Tenant isolation today is enforced in the application: every command, query and event carries a tenant context, and a single logical event store is partitioned by that context rather than split into physical per-tenant streams (Section III.6.1). That isolation is therefore only as strong as the authorisation layer above it, which refuses cross-tenant requests. comby v3 adds a deeper reinforcement, which CLMPilot does not yet activate: row-level security in the database itself, so that the data layer would refuse a cross-tenant read even if the application layer were wrong.

III.6.4 Compliance positioning

The compliance controls follow directly from the architecture. GoBD’s immutable-retention requirement is met by the event store (Section III.2): the audit trail *is* the log. The GDPR’s residency requirement is met by EU hosting; its data-minimisation principle, by integration-first design, since the platform never holds the documents at all. ISO 27001’s access-control and change-tracking expectations are met by per-tenant RBAC and, again, the log.

One tension remains: an append-only store versus the right to deletion. The platform reconciles them by *tombstoning*: a deletion is itself an event that strips the personal data but keeps the structure and ordering of the history. The record that something happened is preserved, while the personal data it contained is removed. That is only possible because corrections here are events, not in-place edits.

III.7 Internationalisation strategy

Internationalisation spans both the server and the client, so it is built into the architecture rather than handled as a translation file added later. English is the platform’s primary language, with German maintained alongside it from the start. The capability is present from the outset; only the translations themselves are deferred, which keeps activating a new language later cheap.

Human-readable text appears in 2 places, one per process. The single-page application carries message catalogues, one per locale, for every label and message; the backend renders its own outbound text, email above all, in the recipient’s locale. The two are connected by the locale itself: a user’s chosen locale is threaded through to the point where an email is rendered, so the platform replies in the same language the user reads. Locale resolution prefers an explicit per-user setting and falls back to English. The chosen locale also governs date, number and currency formats, which differ across the German-speaking markets; time-stamps are stored in UTC and rendered in the user’s zone.

Conclusion

This chapter set out the architecture the rest of the report delivers. 5 drivers (auditability, integration-first design, EU data residency, multi-tenant isolation and maintainability) led to CQRS and Event Sourcing, because together they make the compliance-grade audit trail a property of the storage itself. The comby framework supplies those patterns and the multi-tenant components, which is what lets us cover the platform's requirements with only a small amount of custom code. The requirements map onto a 4-context core domain model (Contract, Approval, Deadline and DocumentLink, extended at its seams by the enterprise contexts) on a single-backend topology hosted in the EU, with tenant isolation and the audit trail as structural guarantees.

Chapter [IV](#) turns from this design to how it was built, consolidating the engineering work delivered across the project: the foundations, the contract core, and the enterprise capabilities.

Chapter IV

Technical realisation of the platform

Introduction

This chapter presents how the CLMPilot platform was built. The result is a multi-tenant pilot, live in production at <https://clmpilot.com>, that covers the full contract lifecycle and extends it with an enterprise tier of capabilities: intelligence-assisted extraction, electronic signature, per-tenant custom fields, obligation tracking and advanced reporting. Each section follows one engineering theme across the whole stack and uses the realised domains to illustrate it: the foundations and the delivery process that carried every later phase, the backend dispatch model, the contract domain and its integration boundary, the workflow and temporal automation, the intelligence and signature subsystems, the architecture of the web application, the identity model that governs access, and the production deployment that keeps the result running.

IV.1 Engineering foundations and the delivery process

Phase 0 produced little product code, but it fixed the decisions every later phase relied on. The first of them was to split the codebase into 4 repositories, each owning one concern and one release cadence; Figure IV.1 shows what each repository owns in the realised platform. The backend repository carries the Go service and its domain contexts. The frontend repository carries the single-page application, and the intelligence repository the Python extraction service. The fourth, the deployment repository, holds the operational stack itself: the Compose files, the reverse-proxy and service-unit configuration, the release and backup scripts, and the runbooks that explain how to operate them.

IV.1 Engineering foundations and the delivery process

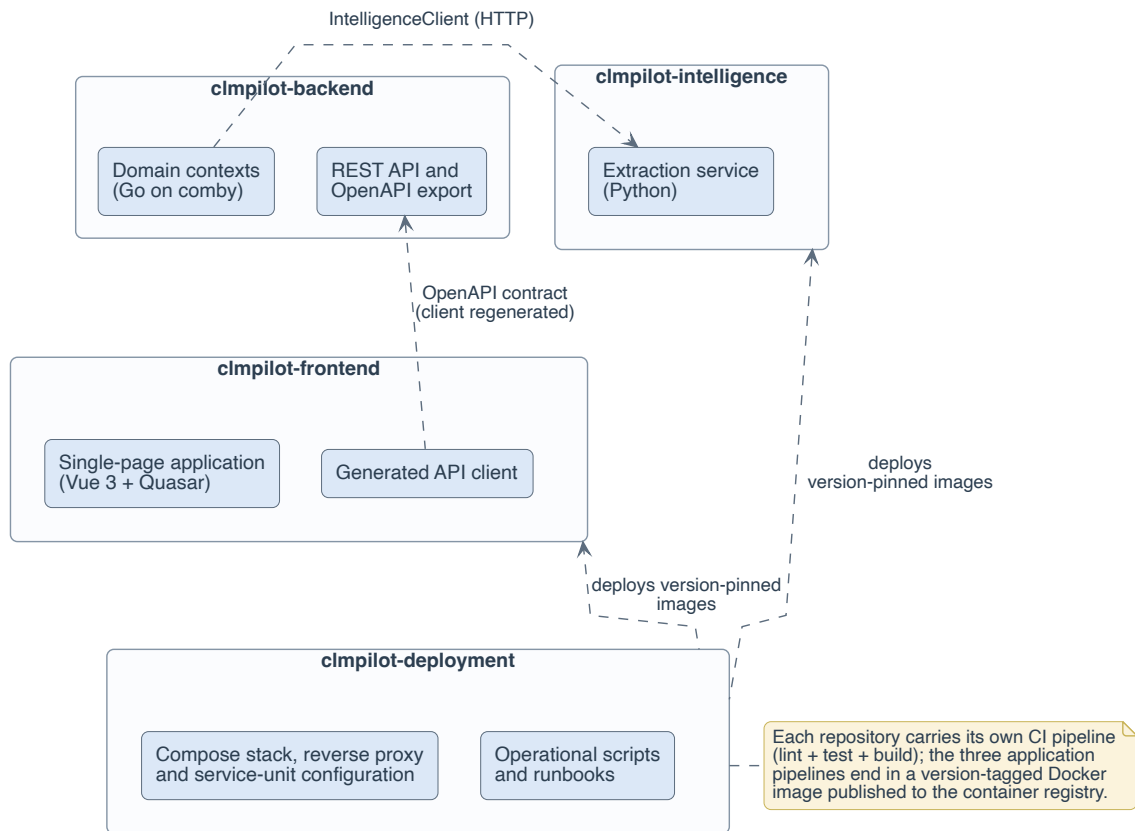


Figure IV.1 – The 4 repositories by concern

The split is by concern, not by microservice: there is one backend, not many. Each repository keeps its own toolchain and its own release pace, so the frontend can ship without rebuilding the backend, and a broken intelligence build never blocks a release. Each also carries a small CI pipeline that runs in minutes. The price is that a cross-cutting change, a new API field for instance, touches 2 repositories at once. We accepted that cost because the OpenAPI contract makes the seam explicit: the client is regenerated from the specification, and a build step keeps the 2 sides in sync.

Every repository follows the same branching rules. Day-to-day work happens on short-lived feature branches merged into `develop` through a reviewed pull request, and `main` receives merges only from `develop`, one per release. This keeps `main` deployable at all times, and it gives each release a diff of its own that can be reviewed. Commit messages follow the conventional format and name the roadmap item behind the change (Section 1.5).

Continuous integration guards branches in all repositories, and each pipeline is specific to what it protects. The backend, frontend and intelligence pipelines run their own linter, tests and builds; the deployment pipeline validates the Compose files and shell scripts. The 3

application pipelines end by building the repository's Docker image and publishing it to the container registry the production host pulls from, tagged with the commit, the branch and the release version. The pipeline and stack definitions are configuration; they are reproduced in Appendix A.3.

2 further foundations complete the delivery process. The local development environment reproduces the production backing services in a Compose file of its own, brought up by an idempotent setup script, so code is developed against the same services it will run on. And scope was tracked in one place throughout: a versioned roadmap files, with a progress tracker. Under this process the 4 repositories accumulated more than 40 tagged releases over the internship, each traceable from its version number back through the release pull request to the roadmap items it delivered.

IV.2 Backend service architecture on comby

The backend is a single Go process. At start-up it constructs the comby facade and connects the backing services (the PostgreSQL event and command stores, the Redis cache, the MinIO object store and the NATS broker). It then registers the framework's default domains (tenant, account, identity, group, invitation, asset and webhook), registers CLMPilot's own contexts after them, and finally restores its state by replaying the event log into the in-memory projections. Only then does it serve requests. The comby layering of Section III.3.1 is visible in the code layout: the domain packages hold aggregates, commands, events, read models and reactors, while dispatch, persistence, tenancy and authentication all belong to the framework.

The boot sequence is ordered, and the order carries rules the realisation had to respect. A command can be dispatched only once its domain's handlers are registered; a boot-time action placed after the facade exists but before registration completes simply fails, because nothing routes the command yet. Administrator seeding (Section IV.8) runs inside domain registration, right after the framework defaults are in place.

HTTP enters through a thin REST layer. comby generates handlers and an OpenAPI document for every registered default, and CLMPilot's own routes follow the same discipline: a handler validates the request's syntax, translates it into a command or a query, dispatches it, and translates the result back into a response. No handler contains a business rule. Every rule lives in an aggregate or a command handler, where the event log records its outcome. The exported specification counted +133 endpoints when the backend was first containerised and grew with each later context; the frontend consumes it through a generated, type-safe client.

Figure IV.2 shows the 2 request paths side by side. A command passes the authorisation stage, reaches the aggregate, is validated against the current state, and appends an event;

IV.2 Backend service architecture on comby

the response returns once the event is durable. A query passes the same authorisation stage, then reads a projection and writes nothing. Most views are served by the framework's generic per-aggregate projection, which covers retrieval and listing without custom code. Where a view has its own access pattern, we built a dedicated read model: expiring contracts sorted by end date, portfolio volume rolled up by quarter, department or type at query time.

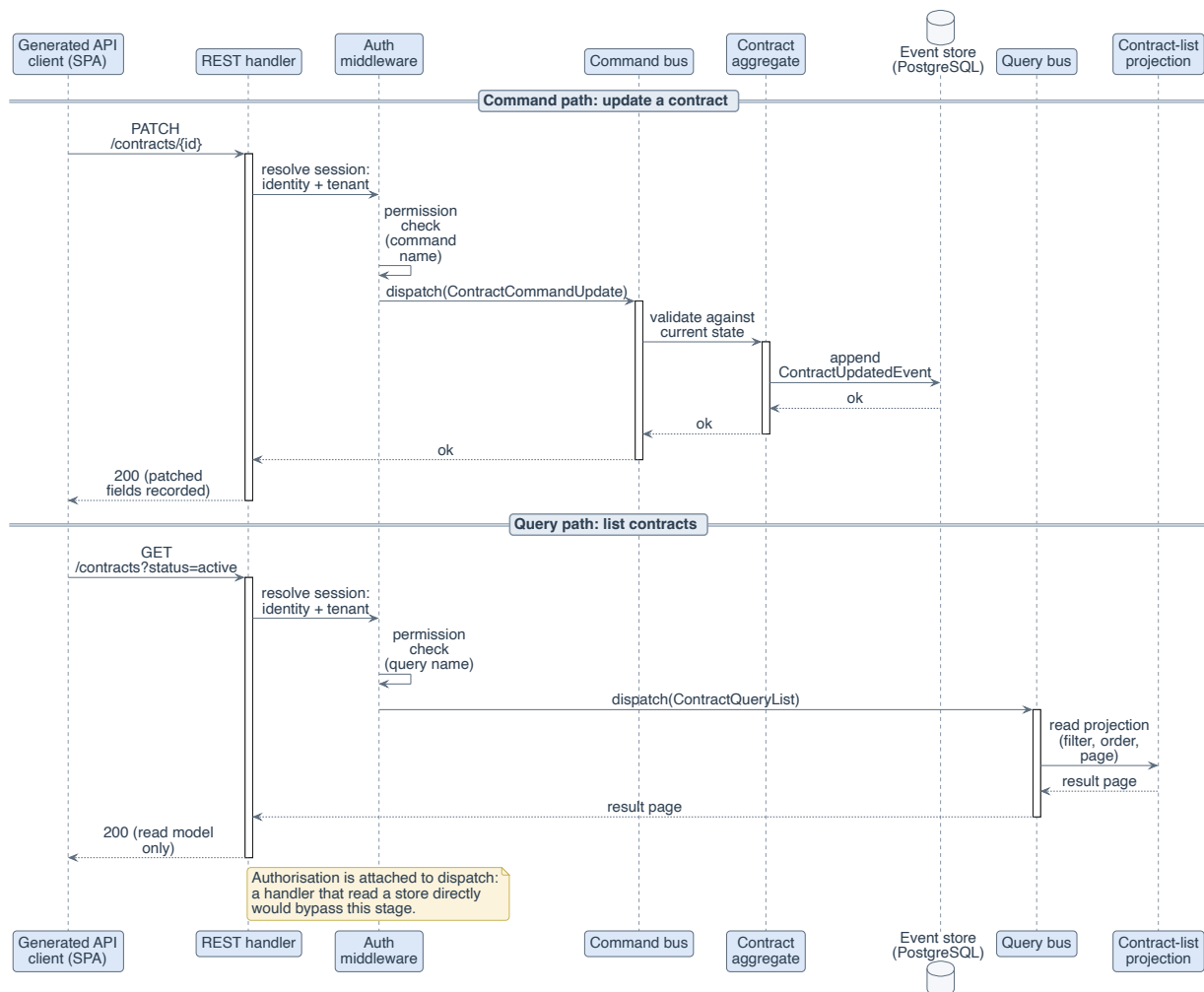


Figure IV.2 – The realised command path and query path, side by side

Authorisation is attached only to dispatch. Every command and every query is checked against the caller's permissions when the bus routes it, so an endpoint that read a store directly, without dispatching, would silently bypass the check. The convention is therefore to route reads through dispatch even where a handler could reach a store directly. The framework's command, query and event catalogue, including the system-dispatch convention, is summarised in Appendix A.2.

IV.3 The contract domain and its lifecycle

The heart of the platform is the Contract aggregate: it realises the contract registry that the functional requirements of Section II.2 place at the core of the product. The aggregate carries the business metadata (counterparty, category and department, value and currency, start and end dates, renewal terms) and a status that moves only along the transitions of the state machine designed in Section III.4.3: a draft becomes active, an active contract may pass through expiring to expired, and an active or expiring contract can be terminated early. The aggregate exposes 7 intentions (register, update, terminate, archive, soft-delete, restore, and request an audit export), and each guards its own preconditions. An update is expressed as a set of patched fields, so the emitted event records exactly what changed. Deletion is a flag rather than a removal, since the immutability requirement of Section II.3.5 forbids destroying history. And a registration is only accepted after the command handler has checked that the contract number is unique within the tenant.

Every state change is an event appended to the contract's stream, so the audit trail demanded by the compliance drivers exists by construction. The platform exposes it in 3 forms. The application renders the raw event list of a contract in its audit-log view, with the actor, timestamp and payload of each entry. A reactor produces an exportable report asynchronously, by replaying the contract's events into a tabular file kept in the object store. And a download endpoint serves that file after first dispatching the contract-retrieve query, so the export inherits exactly the access control of the record it describes.

Phase 2 extended the aggregate with per-tenant custom fields without touching its core. A separate CustomFieldDefinition aggregate lets each tenant's administrator define typed fields (text, number, date, selection and boolean). A definition's key and type are immutable after creation, because changing either would orphan or mistype the values already recorded on contracts. The contract stores its custom values as typed values with a canonical string encoding, chosen so that replaying old events stays deterministic; the events also kept their earlier plain-string form alongside the typed one, so history recorded before the change replays losslessly. Validation crosses the 2 contexts. The Contract command handlers accept an injected validator that checks each value against the tenant's definitions: required fields are present, values parse for their type, selections stay among the allowed options. Values whose definitions have since been deleted are ignored, so re-saving an old record never fails on removed configuration.

DocumentLink realises the integration boundary as a domain object, as designed in Section III.4.2. One aggregate serves all providers. Linking records a typed reference (provider, external identifier, file name and MIME type) and overlays metadata fetched from the provider at link time; unlinking and metadata refresh complete its 3 intentions. The provider-specific

IV.3 The contract domain and its lifecycle

work stays in adapters behind a single `DocumentStore` interface: one for Google Drive, one for SharePoint through the Microsoft Graph API, and a pass-through for arbitrary URLs. The adapters authenticate with per-tenant OAuth tokens held encrypted in the object store, only tenant administrators may run the connect flow, and Appendix A.4 walks through the operator-facing setup for each provider. The integration-first rule (a reference, never the contents) is enforced at this seam: no type in the domain model can hold document bytes, so the claim that the platform does not ingest customer documents can be checked in the model itself. Figure IV.3 draws the realised model, with the 2 aggregates, the definitions the contract validates against, the adapter family behind the storage interface, and the events each aggregate emits.

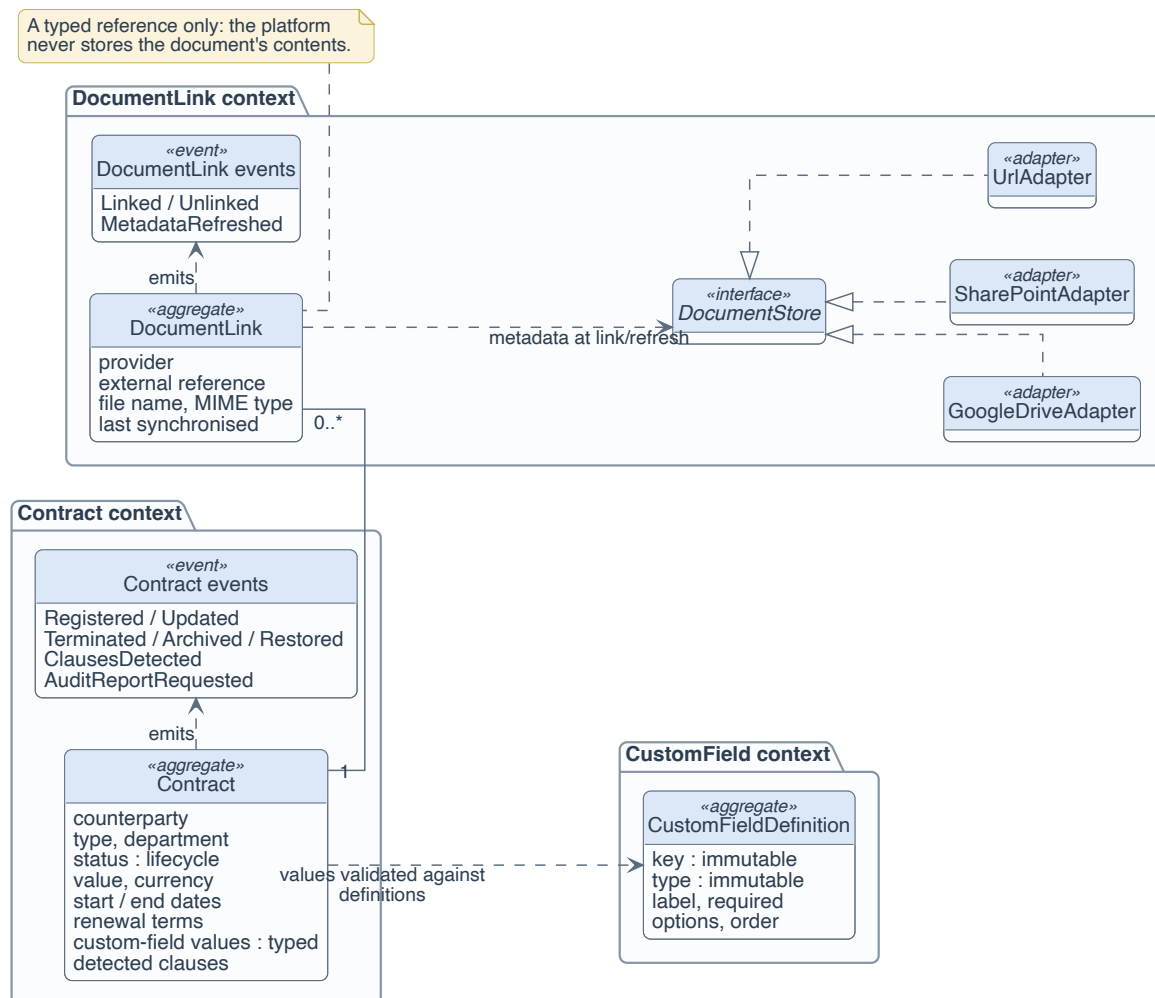


Figure IV.3 – The Contract and DocumentLink model with its events

IV.4 Workflow and temporal automation

3 contexts give the platform its behaviour over time. **Approval** is the ordered review a contract passes before activation; **Deadline** covers the temporal markers the platform watches on the contract's behalf; **Obligation** covers the recurring commitments a signed contract creates. All 3 are built from the same 2 mechanisms: reactors, which turn one domain's event into another domain's command, and an in-process scheduler, which turns the passage of time into commands on a fixed calendar and so realises the system-clock actor of Section II.1.

The approval workflow is an aggregate of ordered steps, each addressed to a designated approver: steps complete strictly in order, only that approver may act, and a single rejection rejects the whole workflow. on the last step a reactor dispatches the finalisation and that flips the contract to active. A second reactor owns the notification mail. Keeping state transitions and side effects apart means a mail failure cannot touch workflow state.

Deadlines are mostly automated, Figure IV.4 follows the realised chain. A cross-domain reactor listens for contract registration and derives up to 3 deadlines from the record: expiry at the end date, the cancellation window at the end date minus the notice period when that window still lies ahead, and renewal when the contract auto-renews.

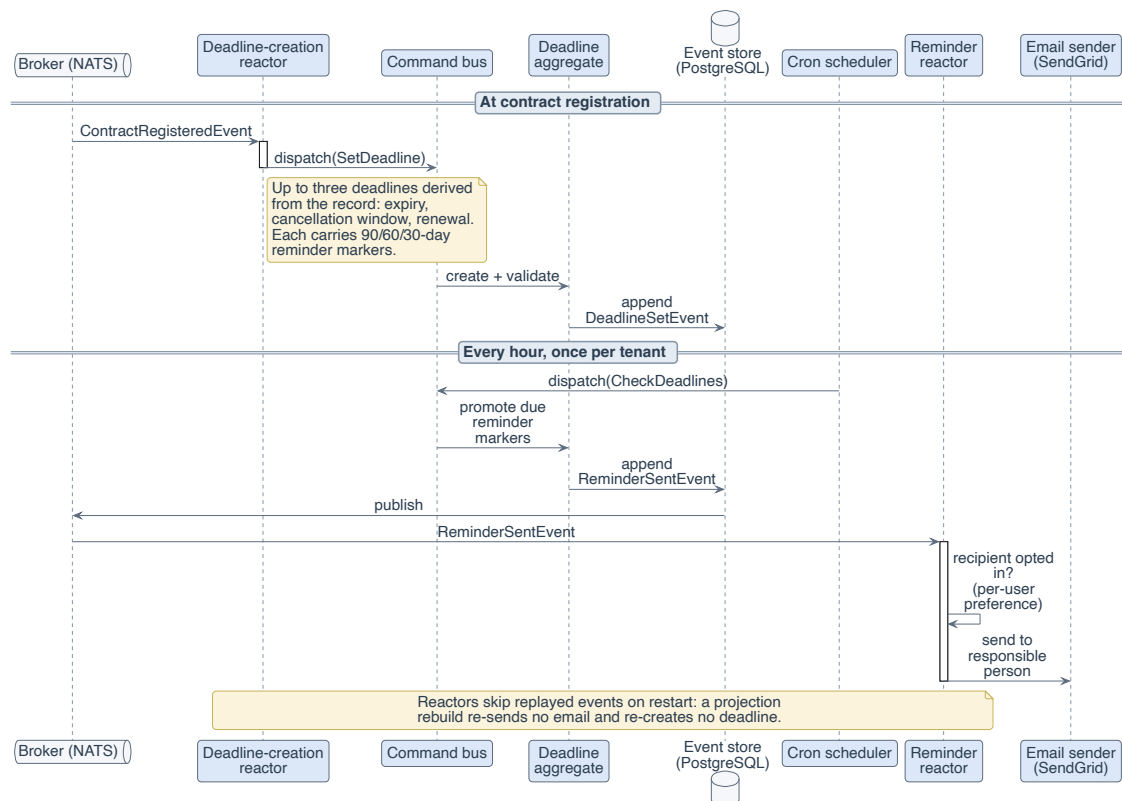


Figure IV.4 – Temporal automation in the Deadline context

Time enters through the scheduler, an in-process cron runner on UTC-anchored jobs. An hourly job promotes every due reminder marker, a nightly one flips overdue deadlines to missed, and both dispatch once per tenant, so a failure in one tenant's batch cannot halt the others; every job logs its tenant count and duration. Reminder mail is opt-in: the reactors consult a per-identity preference, and its absence means no mail.

Obligations, added in Phase 2, reuse the deadline mechanics for a different thing: a deadline is a date the platform watches, an obligation a task a person owns and completes. Users create them; nothing derives them from contract fields. Their markers sit at 14, 7 and 1 days, the shorter horizon of operational tasks, and their lifecycle ends in completed or missed. Recurrence is a third reactor: completing a recurring obligation emits an event from which it spawns the next occurrence, dated by calendar arithmetic anchored to the original one, so a late completion never schedules an occurrence already overdue.

Idempotency is designed in case by case. Every reactor skips replayed events on restore, so a restart, which replays the whole log to rebuild the projections, re-sends no mail and re-creates no record.

IV.5 Document intelligence

The intelligence service is a stateless Python service behind the `IntelligenceClient` interface named in the architecture's component view (Chapter III). It offers 3 operations (metadata extraction, contract-type classification and clause detection), each implemented as a prompt sent to a managed large-language-model API [17] behind a small internal gateway. The service parses DOCX, plain-text, Markdown and text-layer PDF documents itself, and passes a PDF to the model as a document input only when it finds no text layer. A language heuristic separates German from English, so German legal-entity forms are preserved verbatim in extracted names.

Clause detection carries the subsystem's strictest design rule: every detected clause must quote its source verbatim, and a clause whose quoted snippet is not a substring of the document text is discarded. This makes a hallucinated quotation structurally impossible. The same guard also fixes the subsystem's limit. A scanned PDF has no text layer to check quotations against, so clause detection on scans stays out of scope until an optical-character-recognition step is added; extraction, which does not quote, still works on scans through the model's vision input.

The subsystem's quality is measured continuously. An evaluation harness of 20 hand-labelled fixtures (plain-text and DOCX documents, 13 English and 7 German, covering every contract type) runs in the integration pipeline and scores each extracted field against its label. The baseline stands at 158 of 160 fields correct (98.8%). The harness paid for itself during

IV.5 Document intelligence

development, when a prompt revision raised counterparty-name accuracy from 80 % to 90 % and the regression risk was visible before release.

On the backend, an enrichment reactor connects this pipeline to the contract. Figure IV.5 follows the scenario. Once a document is linked, the reactor downloads its bytes through the same storage adapters as the link itself (capped at 50 MiB), calls extraction and clause detection, and applies the results to the contract. Only fields the user has left empty are filled; a human-entered value is never overwritten. The call runs detached from the linking request, under a 5-minute ceiling and a panic guard, which brought the latency of the link operation from over 6 seconds to under 1. The suggestions arrive on the record moments later, and the user reviews them field by field (accept, edit or undo) in the interface of Section IV.7. Re-linking the same document is treated as a synchronisation and refreshes every extractable field.

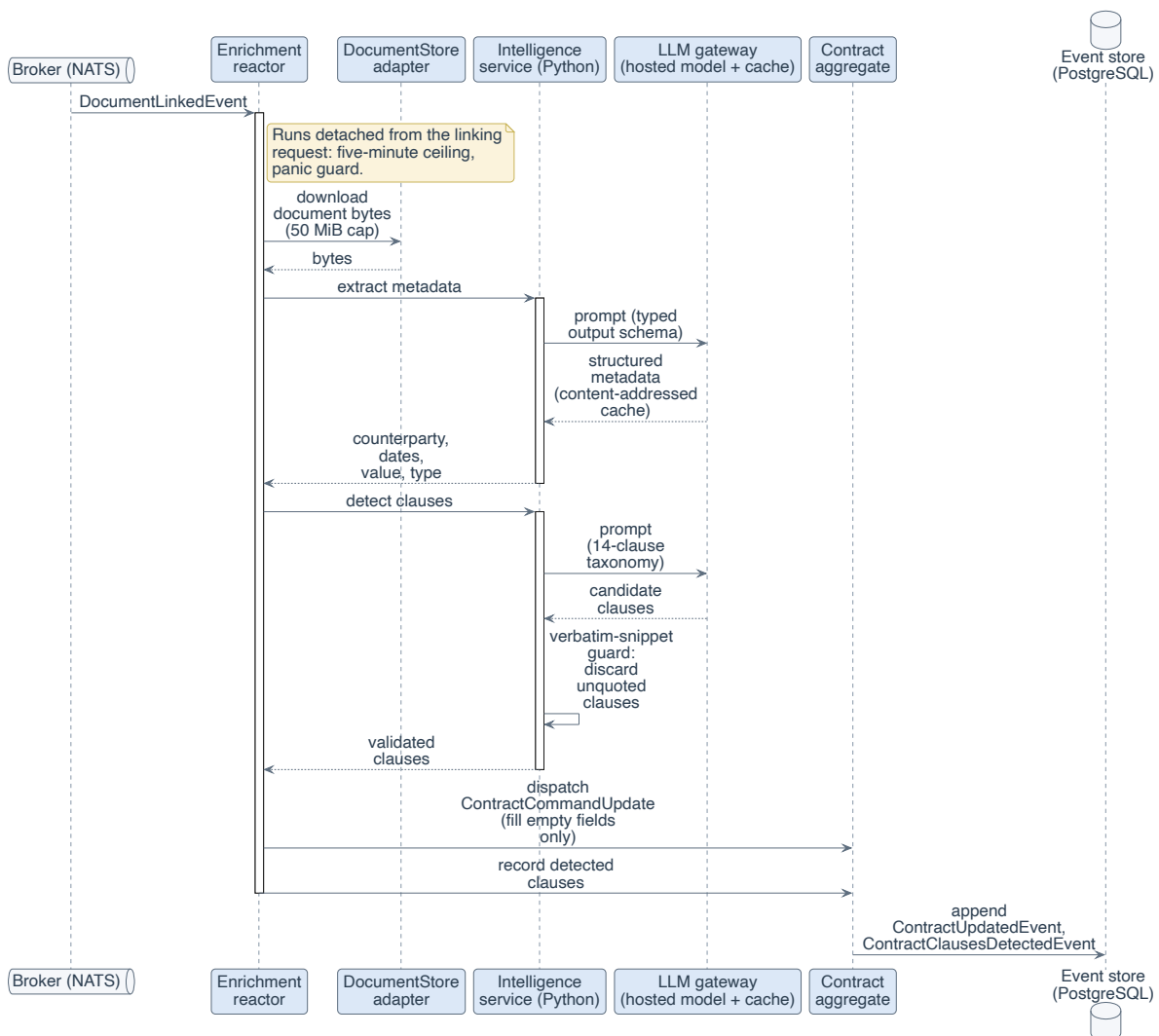


Figure IV.5 – The enrichment pipeline

IV.6 Electronic signature

Electronic signature sits behind a provider interface, with the same discipline as document storage: the domain model never names a vendor. The first adapter targets DocuSeal [18], selected on 4 grounds. It can be self-hosted under an open-source licence, which keeps signed documents on the platform's own infrastructure. The simple electronic signature it produces has legal effect for ordinary business contracts under Article 25 of the eIDAS regulation [19]. It offers the REST-and-webhook integration the reactor pattern needs.

The signature workflow is its own aggregate, modelled on Approval. A request holds ordered signers, and events record dispatch, each signer's completion or refusal, overall completion and voiding. Sending validates that the contract is in a signable state, pulls the primary linked document's bytes through the storage adapters, and hands them to the provider. The tenant, contract and request identifiers are stamped into the provider's submission metadata, so whatever comes back can be routed.

Figure IV.6 shows the return path. Once the envelope is sent, the provider collects the signatures on its own surface and reports back when it has something to say. The public endpoint that receives them, the command it dispatches inward, and the reactor that acts when signing completes; wait on those deliveries

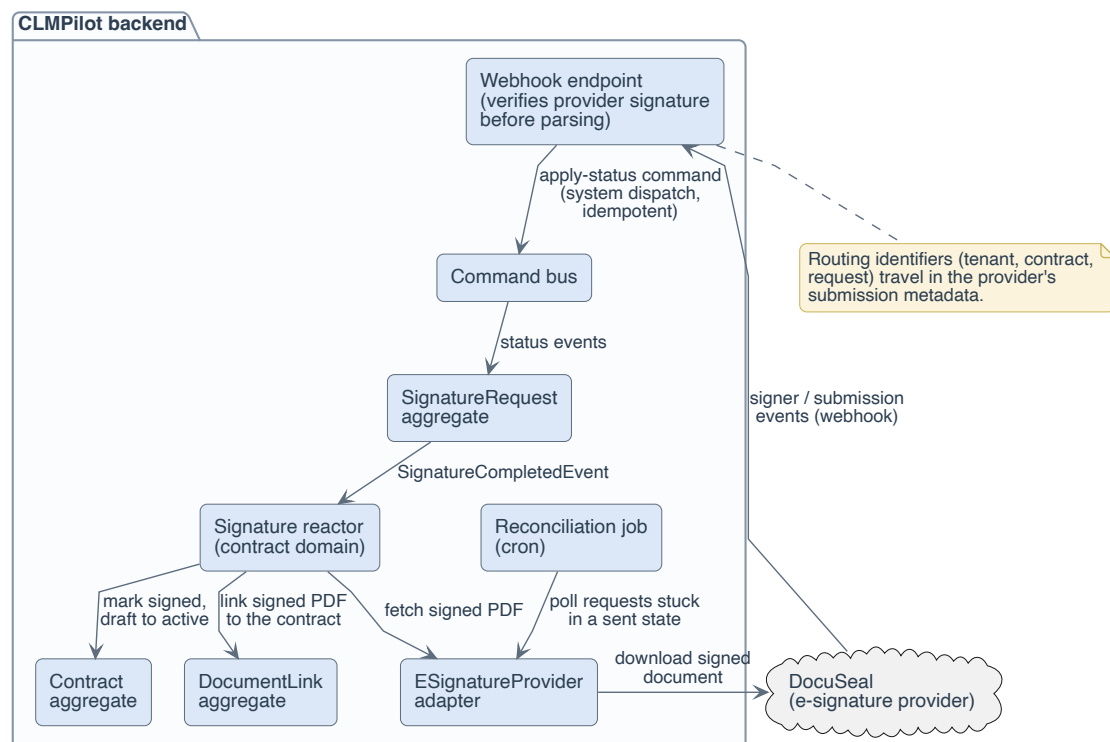


Figure IV.6 – Component view of the e-signature webhook flow

IV.7 The web application and its internationalisation

Progress arrives as webhooks on that endpoint, which verifies the provider's signature before parsing anything, reads the routing identifiers back out of the submission metadata, and translates the provider's event into a status-application command dispatched with system authority. Applying status is idempotent: a duplicate or out-of-order delivery is compared against the aggregate's current state and ignored when stale. Completion then triggers the contract-domain reactor, which marks the contract's signature status, flips a draft contract to active, downloads the signed PDF from the provider and links it back to the contract as an ordinary `DocumentLink`, so the executed document enters the same audit trail as everything else. Webhooks can be lost, so the scheduled reconciliation job of Section IV.4 polls the provider for requests stuck in a sent state past a threshold. We validated the whole path end to end against DocuSeal's hosted sandbox; in production the integration stays inactive until an operator configures a provider key.

The outbound email these subsystems and the workflow reactors rely on is delivered through SendGrid's HTTP interface [20], sending from an already-authenticated company domain. That choice avoided any DNS work on the product domain while keeping deliverability verifiable. Messages are rendered from bilingual templates (7 at the MVP, growing with each later context) as multipart text-and-HTML, in the recipient's locale (Section IV.7).

IV.7 The web application and its internationalisation

The web application is a single-page application built with Vue 3 and the Quasar component framework [21, 22], written in strict TypeScript. Figure IV.7 shows its structure, which separates 4 kinds of module: pages and components render; composables package reusable view logic such as contract lookups, permission checks and deadline bucketing; Pinia stores own the client-side state and every server call; and a small utility layer holds pure functions. Testability is the reason for the split. The project runs no component-mount test harness, so everything worth testing is kept out of the components, in stores, composables and utilities that the unit suite (198 tests at the close of Phase 2) exercises directly.

IV.7 The web application and its internationalisation

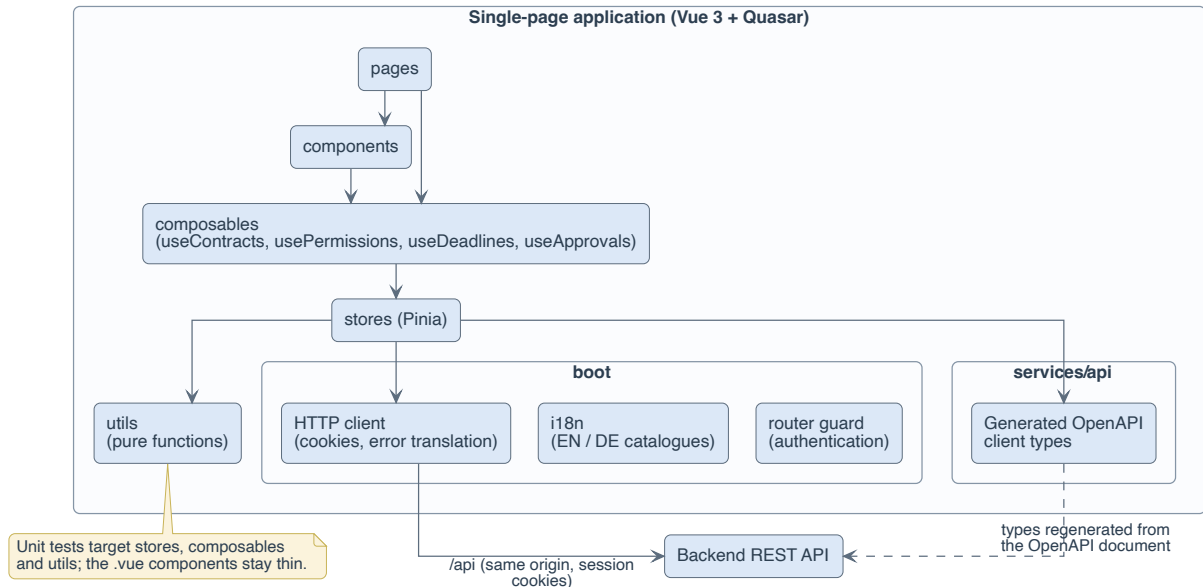


Figure IV.7 – Package structure of the single-page application

The backend is reached through exactly one integration point: a typed client generated from the backend’s OpenAPI document [23]. Regenerating the client is a build step, so an API change becomes a compile error in the frontend, caught before it can become a runtime failure; this is the explicit, regenerated contract that Section IV.1 accepted as the cost of the repository split. The HTTP layer adds the cookie-based session, one place where backend errors are translated into localised messages, and the proactive session refresh described in Section IV.8.

Internationalisation follows the strategy of Section III.7, realised end to end. The interface carries English and German message catalogues, split by feature namespace. The locale is resolved from the user’s stored preference, persisted on the identity so it follows the user across devices, with a browser-language fallback; dates, numbers and currencies format by locale, including day-first date ordering for the English interface, whose users are European. The locale crosses the stack as well. It travels with an invitation, so an invited user’s first email arrives in the language chosen by the person who invited them, and every notification renders in its recipient’s locale. The user documentation ships inside the application itself: a help area bundles the documentation source at build time, in both languages, so the documentation is versioned with the code it describes. The packaging caveat this uncovered is recorded with the configuration material in Appendix A.3.

2 Phase-2 capabilities demanded the most from the presentation layer. Custom fields make the contract form partly data-driven: the form renders one input per tenant definition, choosing the widget by the field’s type and mirroring the backend validation rules, while administrators

manage and re-order the definitions from the settings area. The report builder turns the reporting projections of the backend into a self-service tool. Those projections include portfolio volume, per-department and per-counterparty rollups, and a risk score computed as an additive sum of named factors (contract value, expiry proximity and missing documents), kept transparent so a reviewer can see why a contract scored as it did. In the builder, the user picks columns, groupings and a chart type, saves the configuration under their own identity, and can schedule it as a recurring export delivered by email through the scheduler of Section IV.4.

Every screen of the application is assembled the same way: stores own the server calls and the client-side state, composables carry the reusable view logic, and components render. No page departs from that division, which is what keeps the unit suite meaningful in the absence of a component-mount harness.

IV.8 Identity, access control and multi-tenancy in practice

The identity model in production is the comby default model of Section III.6.1, used without modification: tenants for customer organisations, accounts for people, identities binding an account into a tenant, groups as roles, and permissions as the right to dispatch a given command or query. What CLMPilot adds is the vocabulary on top of it: a typed catalogue of permission constants covering every domain, and 5 role groups (administrator, legal, finance, manager and viewer) whose permission bundles are seeded into every new tenant by a reactor listening for tenant creation, so a freshly provisioned organisation starts with a working role set. Tests guard the catalogue's invariants and hold the viewer and finance roles to read-only bundles, with no command permission at all.

Figure IV.8 follows a sign-in across the stack. Authentication is 2-step: the credentials are checked first, then a one-time code sent by email must be confirmed, after which the backend issues the opaque session cookie and the companion identity cookie that scopes every later request to a tenant (Section III.6.2). The client mirrors the session's lifetime and refreshes it shortly before expiry, so a working user is never interrupted. Self-serve registration through an invitation follows the same rails, with one refinement found in the framework's internals: confirming a registration already returns a pre-issued login token, which the client feeds straight into the login confirmation, so a new user completes registration with exactly one code and lands authenticated. A password-reset flow completes the account lifecycle. The invitation page also inserts an explicit human click before any state-changing step, because corporate mail scanners follow links and would otherwise consume single-use invitation tokens.

IV.8 Identity, access control and multi-tenancy in practice

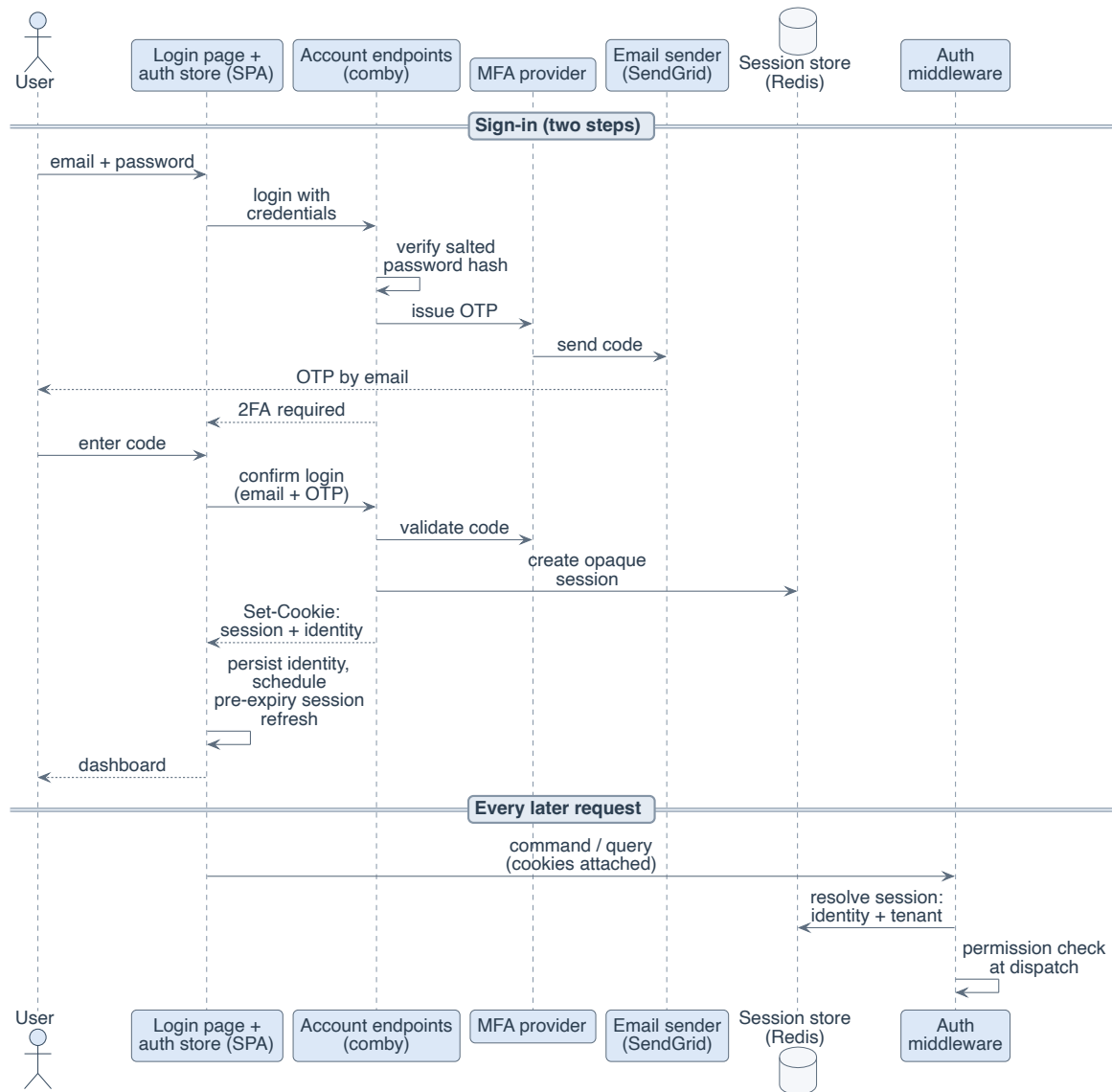


Figure IV.8 – The authentication journey across the stack

Role-based access control is enforced at dispatch (Section III.6.2), and the realisation taught us 2 rules about it. First, the permission string checked at dispatch is derived from the command or query type itself, so the catalogue’s constants must match those derived names exactly. After a mismatched constant silently produced a permission that no role could ever hold, a regression test now derives every name by reflection and compares. Second, integration tests that dispatch with system authority never exercise the authorisation stage. That blind spot let 3 role-configuration defects stay invisible until the first ordinary tenant hit the standard path, because the platform operators used until then bypass the role check entirely. The countermeasure is an end-to-end suite that authenticates as a real user against the running

IV.9 Production deployment and release operations

API, so every permission a role needs is proven from the outside. The frontend consumes the same vocabulary through a permission composable that hides actions a role cannot perform; the backend remains authoritative.

Multi-tenancy in operation needs administration as much as isolation. A reserved system tenant holds the platform’s operators, seeded at boot from configuration, so an administrator can be added or recovered without touching the database. The operators work through a cross-tenant administration area in the application itself, which provisions a tenant, waits for its role groups to be seeded, and issues the first administrator invitation. Decommissioning is archival. Archiving marks the tenant, and a middleware on both dispatch paths refuses every request scoped to an archived tenant unless the caller is a platform operator; the client translates the refusal into an explicit end-of-service screen. The archived state is therefore enforced at the same chokepoint as every other rule, and the user interface only reflects it.

IV.9 Production deployment and release operations

Production realises the topology of Section III.5.2 on a single EU-hosted virtual machine at Hetzner, a 4 GB instance sized for the pilot. Figure IV.9 shows the operational view. The 3 service images and the 4 backing stores run as one Docker Compose stack [24] under a systemd unit, so the stack survives a reboot. Every container binds to loopback, and the only process facing the network is an nginx reverse proxy on the host, which terminates TLS with certificates issued and renewed automatically through Let’s Encrypt. Memory limits are set per container, and PostgreSQL is tuned to the small host, which keeps the single-machine footprint predictable under load.

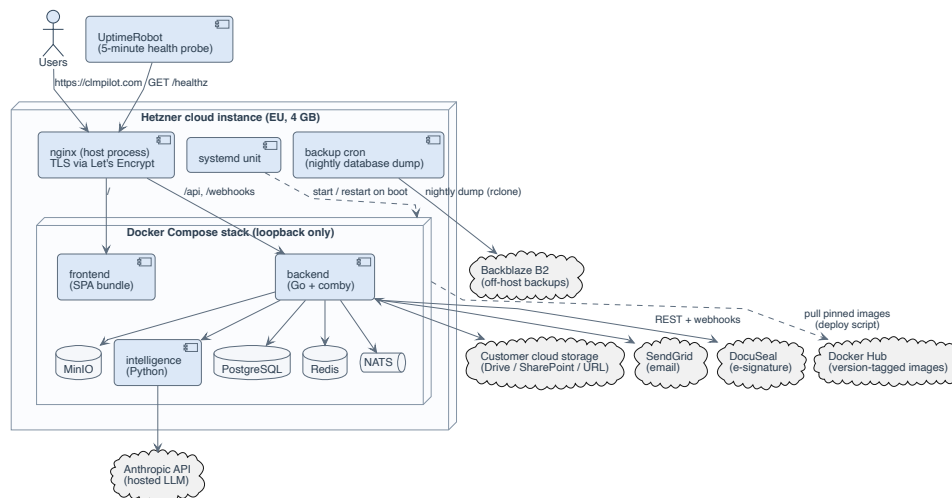


Figure IV.9 – The production topology

IV.9 Production deployment and release operations

The platform serves at the apex domain, `clmpilot.com`, with the API under the same origin. That choice removes a class of complexity: same-origin requests need no cross-origin configuration, cookies operate under the strictest same-site policy, and one certificate covers the whole product. An earlier subdomain plan was dropped, and its stale references were cleaned out of the stack once the apex choice was final.

Backups follow the recovery objectives fixed with the non-functional requirements (Chapter II). A nightly database dump leaves the host for object storage at a second provider, so the loss of the machine cannot take the backups with it. We validated the procedure with a full drill against production: back up, drop the databases, restore, and let the backend rebuild every projection by replaying the restored event log. The drill confirmed the recovery-point objective (under 24 hours, one nightly backup) and beat the 4-hour recovery-time objective by a wide margin; restore and replay completed in under a minute at pilot volume. The replay step doubles as an integrity check of the event-sourced design, since the read models are rebuilt from the restored log alone: a successful drill proves the log is sufficient to reconstruct the system.

Observability at pilot scale deliberately adds no external tooling. The services emit structured logs, tagged by component, and these are the primary diagnostic tool. The backend exposes metrics in Prometheus format, but no aggregation stack runs on the host yet, a decision consistent with the maintainability driver of Section III.1. External monitoring covers what users experience: an independent probe checks the public health endpoint every 5 minutes and alerts by email, and the TLS configuration holds the highest public rating. One detail forced a choice here: the backend’s minimal container image ships no shell, so container-level health checks gave way to probes from outside the container. What the host does not aggregate, the backend still measures in-process; that evidence is presented in Section V.9.

A release is a promotion, not a build performed on the host. Work merges into `develop`; a release pull request promotes `develop` to `main`; a semantic-version tag triggers the pipeline that builds and publishes the image; and a deployment script on the host pins the new versions in the environment, pulls the images, restarts the stack and polls the health endpoint before reporting success. Rollback is the same procedure with the previous version numbers. The script carries one caveat we hit in practice: it pins all 3 service versions in lockstep, which keeps a full release reproducible but makes a single-service hotfix awkward. Partial releases therefore bypass the script and edit the pinned versions directly, and the services were re-unified onto one version at the close of Phase 2 so that the lockstep procedure and its rollback work again. The script and the bypass are reproduced in Appendix A.3.

With production traffic observable, the availability and performance targets of Section II.3.4 moved from assumptions to measurements. Deploys complete in well under the recovery-time budget, the uptime probe provides the availability record against which the MVP target is

checked, the runtime measurements of Section [V.9](#) supply the server-side record behind the response-time budgets, and the pilot's operational history (more than 40 releases, a single rolled-back deployment during a framework migration, and no data loss) is the evidence those targets were set to collect.

Conclusion

This chapter followed the construction of the platform theme by theme: the delivery foundations and the release train they carried, the dispatch model on comby, the contract domain with its lifecycle and its document links, the workflow and temporal automation built on reactors and a scheduler, the intelligence and signature subsystems, the composition of the internationalised web application, the identity and tenancy model enforced at dispatch, and the deployment and release operations that keep the pilot live at `clmpilot.com`. Each theme delivers a part of the designed architecture; together they close the 3 product phases of the release train.

The next chapter turns from mechanism to result. It walks the delivered platform as its users and its operators meet it, screen by screen, and closes with the performance measured on the running pilot.

Chapter V

The delivered platform

Introduction

A platform is judged on what it puts in front of the people who use it. This chapter walks the delivered CLMPilot platform as its users and its operators actually meet it, on the pilot running in production at <https://clmpilot.com>. The tour opens where a user does, at the sign-in and the dashboard that greets them, then goes down into the registry that holds the contracts and remembers every change made to them, the settings that shape the product around one organisation, and the work it then does on that organisation's behalf, before closing on the interface in 2 languages and the operator's own administration surface. One contract carried end to end draws the threads together, and the chapter ends with what the running system measured, so that the walkthrough answers to recorded values rather than to claims.

V.1 Access and onboarding

A user reaches CLMPilot at a single address and signs in through 2 steps: the credentials first, then a one-time code delivered by email. Figure V.1 shows the second of them. Only once the code is confirmed does the session begin, and it is renewed quietly before it expires, so a working day never ends in an unexpected sign-out.

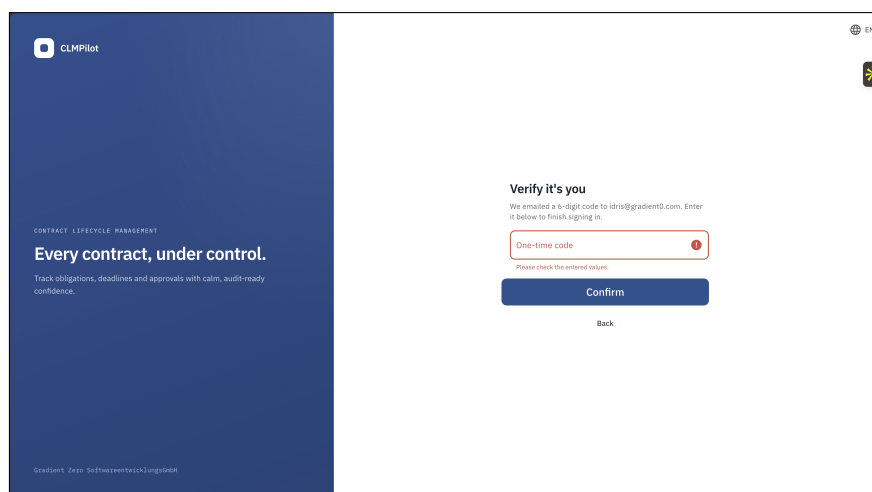


Figure V.1 – Two-step sign-in: the one-time-code confirmation that completes authentication

A new colleague arrives by invitation instead. The invitation mail is written in the language chosen by the person who sent it, and the page it opens asks for an explicit click before anything is consumed, so a corporate mail scanner cannot burn the single-use token on the recipient's behalf. Accepting it costs the new user exactly one code: registration and first sign-in complete together, and the user lands inside the tenant already authenticated.

V.2 Reporting and the dashboard

Signing in lands the user on the dashboard, and it is the clearest view of what the platform has been doing while nobody was watching. Figure V.2 shows it: the portfolio indicators, the contracts approaching expiry and the deadlines coming up. None of it is compiled by hand: every tile is a query over the read models the platform keeps up to date as events arrive.

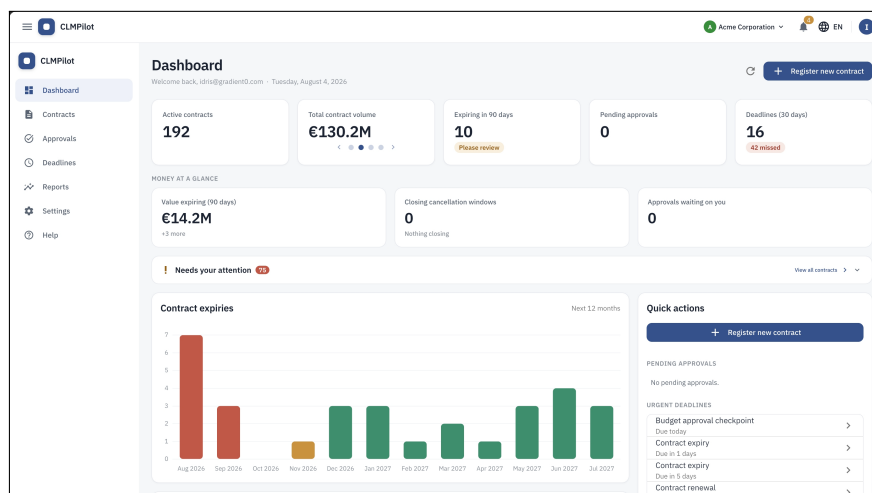


Figure V.2 – The dashboard, with the main metrics

The tiles answer the questions everybody asks; the questions one organisation asks are its own, so the user can also build the report. Figure V.3 shows the report builder: the user chooses the columns, the grouping and the chart, sees the result immediately, saves the configuration under their own name and, if the report is one they need regularly, schedules it as a recurring export delivered by email. The available measures include portfolio value, per-department and per-counterparty rollups, and a risk score.

V.3 The contract registry and its audit trail

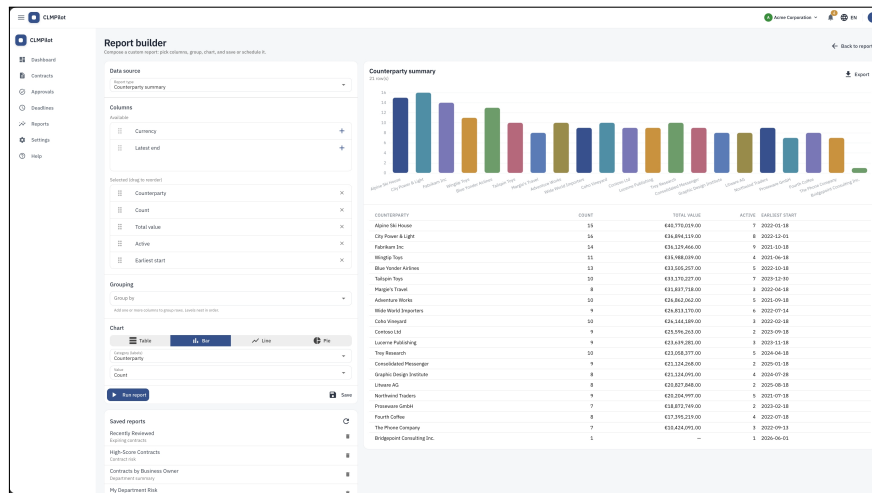


Figure V.3 – The report builder on the demo tenant

V.3 The contract registry and its audit trail

Every tile and every column of those 2 views resolves to the same place, and that is where the working day is actually spent: the registry that holds the contracts themselves. Figure V.4 shows it as the demo tenant's users see it: every contract of the organisation in one filterable listing, with a free-text search across the descriptive fields and filters on department, category, status and counterparty. Opening a record gives the business metadata, the status, the linked document, the deadlines derived from it and the people involved.

The screenshot shows the 'Contracts' registry in CLMPilot. At the top, there's a header with 'Contracts' and a '+ Register new contract' button. Below the header, there's a search bar and several filter dropdowns: Status, Type, Department, and End date (from - to). An 'Export' button is located on the right. The main content is a table of contracts with columns: URGENCY, CONTRACT NUMBER, TITLE, COUNTERPARTY, TYPE, DEPARTMENT, and STATUS. The table lists 11 contracts with various statuses like 'Draft', 'Active', 'Expanding', and 'Expired'.

URGENCY	CONTRACT NUMBER	TITLE	COUNTERPARTY	TYPE	DEPARTMENT	STATUS
Expires in 623d	ACME-00001	Master Services Agreement – Liware AG	Liware AG	Master Services Agreement	Human Resources	Draft
Cancel in 777d	ACME-00002	Service Agreement – Graphic Design Institute	Graphic Design Institute	Service Agreement	Procurement	Draft
Expires in 867d	ACME-00003	Supply Agreement – Fabrikam Inc	Fabrikam Inc	Supply Agreement	Sales	Active
Cancel in 746d	ACME-00004	SaaS Subscription – City Power & Light	City Power & Light	SaaS Subscription	Sales	Draft
Cancel in 288d	ACME-00005	SaaS Subscription – Adventure Works	Adventure Works	SaaS Subscription	Engineering	Draft
No action	ACME-00006	Employment Contract – Trey Research	Trey Research	Employment Contract	Operations	Expanding
Expired	ACME-00007	Reseller Agreement – Adventure Works	Adventure Works	Reseller Agreement	Finance	Expanding
Renews in 5d	ACME-00008	Service Agreement – Fourth Coffee	Fourth Coffee	Service Agreement	Finance	Expanding
No action	ACME-00009	Reseller Agreement – Wingtip Toys	Wingtip Toys	Reseller Agreement	Engineering	Expired
No action	ACME-00010	Reseller Agreement – Lucerne Publishing	Lucerne Publishing	Reseller Agreement	Operations	Archived
Expires in 440d	ACME-00011	Master Services Agreement – Alpine Ski House	Alpine Ski House	Master Services Agreement	Legal	Active

Figure V.4 – The contract registry

Every record carries its own history, and the platform shows it without interpretation. Figure V.5 shows the audit-log tab of a contract: each entry names the actor, the moment and

V.4 Fitting the platform to the organisation

what changed. Nothing in the interface can remove an entry, because nothing in the platform can; a deletion is recorded as one more event. The same history is available as an exportable file for an auditor who needs it outside the application, and it is served only to a user who may already read the contract itself.

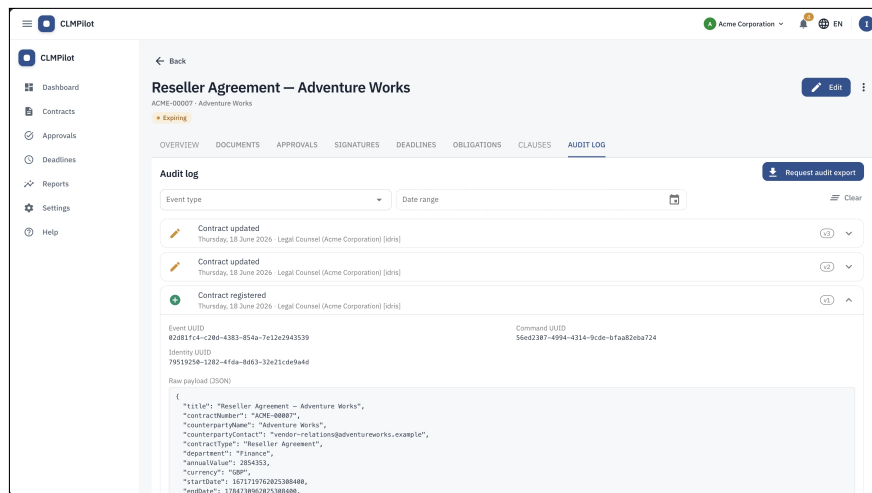


Figure V.5 – The audit-log view of a contract

V.4 Fitting the platform to the organisation

No 2 legal departments track the same attributes, so the contract form is only partly fixed. A tenant administrator defines the fields that matter to the organisation, choosing a type for each, and the form grows accordingly: Figure V.6 shows the fixed fields followed by the tenant's own, one widget per definition, with the date pickers, numeric inputs and selection lists that each type implies.

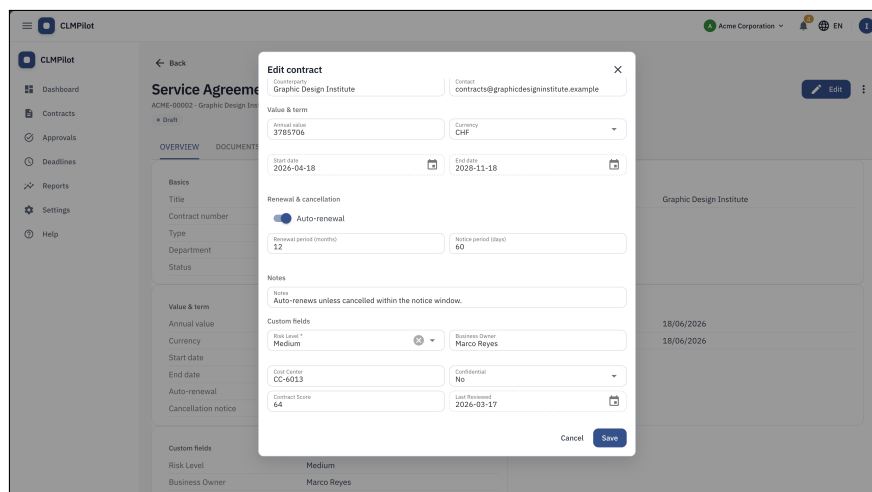


Figure V.6 – The contract form: fixed fields followed by the tenant's typed custom fields

V.5 Approvals, deadlines and obligations

The second piece of configuration connects the organisation's own document storage. Figure V.7 shows the integrations area, where an administrator authorises CLMPilot against Google Drive or SharePoint once and for the whole tenant.

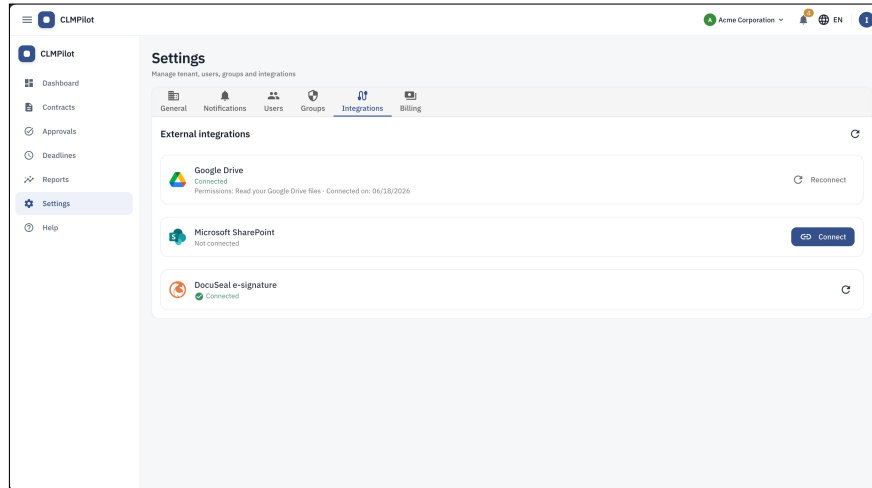


Figure V.7 – The integrations settings of a tenant administrator: the per-tenant OAuth connections behind the DocumentStore adapters.

V.5 Approvals, deadlines and obligations

A contract that needs review before it takes effect carries an approval chain, and each approver meets it as an inbox. Figure V.8 shows that inbox: the steps waiting on this user, in the order the chain defines. An approver acts only on their own step, and a single rejection ends the chain for everybody. When the last approver signs off, the contract becomes active without anyone having to remember to activate it.

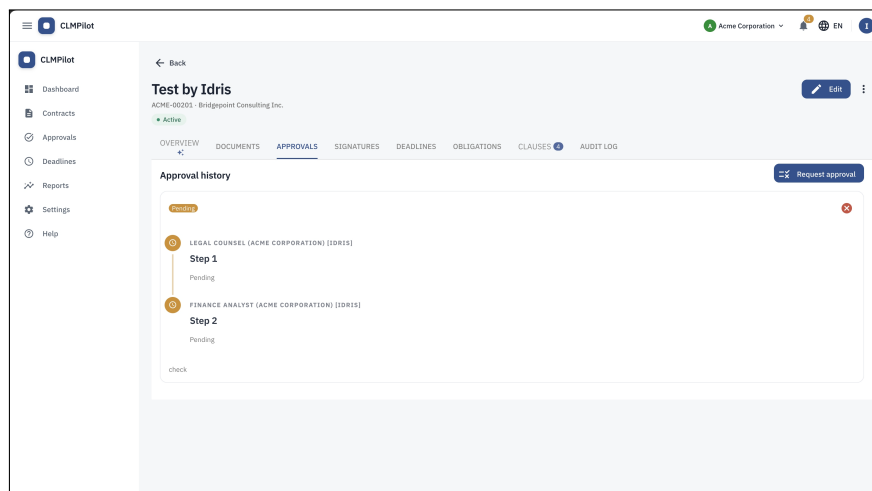


Figure V.8 – The approval inbox: ordered steps, each awaiting its designated approver.

V.5 Approvals, deadlines and obligations

Deadlines are the part of the product that works while nobody is watching. Registering a contract is enough to create them: the expiry, the cancellation window computed back from the notice period, and the renewal date when the contract renews itself, each with reminders at 90, 60 and 30 days. Figure V.9 shows the result: the horizon of what is coming, with the reminders that will announce it. A deadline a user has edited is left alone by the automation, and reminder mail is sent only to users who asked for it.

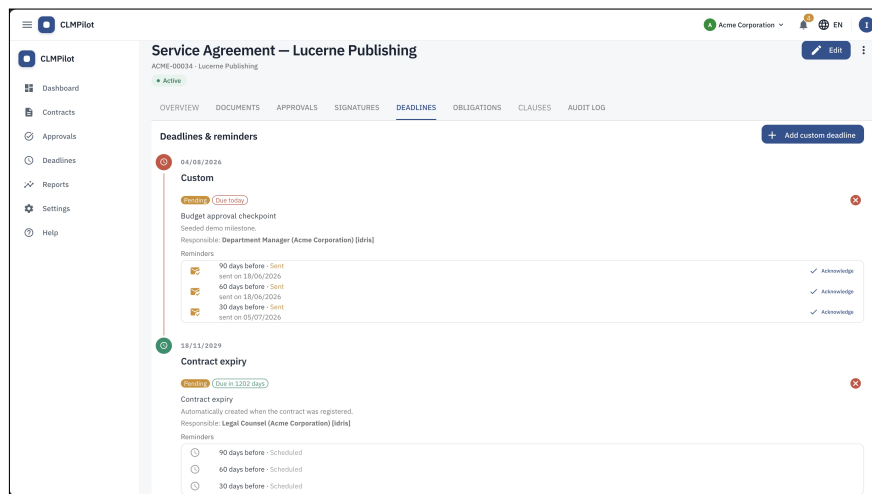


Figure V.9 – The deadline view: automation-created markers with their reminder

Obligations answer a different question: not what date is approaching, but what somebody has to do. Figure V.10 shows one recurring commitment with its occurrences. A user creates the obligation, owns it and completes it; completing a recurring one makes the next occurrence appear straight away, on the calendar day the original set, and never in the past. The reminders sit closer, at 14, 7 and 1 days, because operational tasks have shorter warning than contract dates.

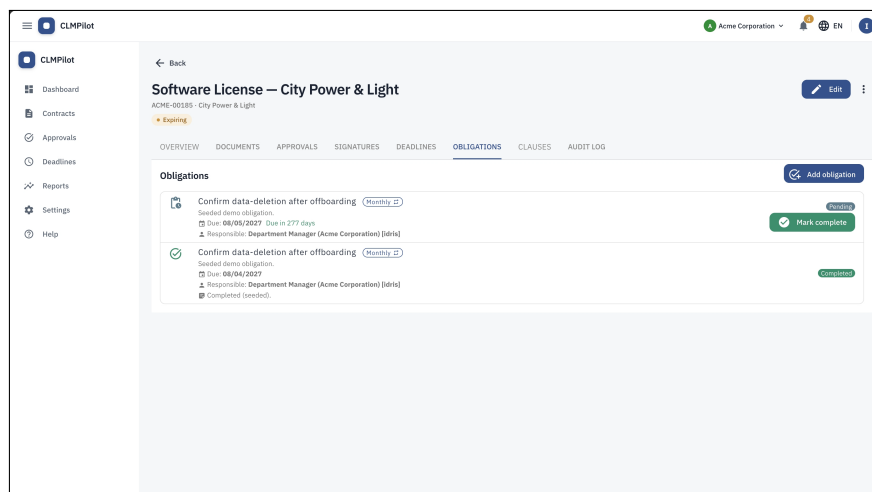


Figure V.10 – Obligation tracking: person-owned recurring tasks with the shorter 14/7/1-day reminder

V.6 Reading and signing the document

Linking a document changes what the record knows about itself. Within a few seconds of the link, the platform has read the document and proposes what it found: the counterparty, the dates, the value, the contract type and the clauses it could quote. Figure V.11 shows those suggestions on a contract overview.

The screenshot shows the CLMPilot interface for a contract titled 'Test by Idris' (ACME-00201 - Bridgepoint Consulting Inc.). The 'OVERVIEW' tab is active, displaying a table of enrichment suggestions. A blue banner at the top indicates 'AI-suggested values: 5 fields auto-populated from a linked document.' with 'Accept all' and 'Undo all' buttons. The table is organized into two columns: 'Basics' and 'Value & term'.

Basics		Counterparty	
Title	Test by Idris	Counterparty	Bridgepoint Consulting Inc. *
Contract number	ACME-00201		
Type	NDA *		
Department	—		
Status	Active		

Value & term		Metadata	
Annual value	—	Created	18/06/2026
Currency	EUR	Updated	04/08/2026
Start date	01/06/2026 *		
End date	01/06/2029 *		
Auto-renewal	No		
Cancellation notice	90 days *		

Figure V.11 – Enrichment suggestions on a contract of the demo tenant

The same document can be sent for signature without leaving the contract. Figure V.12 shows the signature panel of a request in progress: the signers in the order they will be asked, and where each one stands. The panel updates itself as the provider reports progress; nobody has to refresh a status by hand. When the last signer finishes, the executed document comes back and attaches itself to the contract as an ordinary link, so it enters the same audit trail as everything else, and a contract still in draft becomes active on the spot.

The screenshot shows the CLMPilot interface for a contract titled 'SaaS Subscription - Adventure Works' (ACME-00005 - Adventure Works). The 'APPROVALS' tab is active, and a 'Send for signature' button is visible. A modal window titled 'Send for signature' is open, displaying the request details and a list of signers.

Send for signature

Email subject: Signature requested: (ACME-00005)

Message to request: Please review and sign SaaS Subscription - Adventure Works (ACME-00005).

Signers

Name	Email
Signer 1	signer1@beta.com
Signer 2	signer2@alpha.com

+ Add signer

Cancel Send

Figure V.12 – The signature request on a contract

V.7 The product in 2 languages

The platform is delivered in English and German, and the choice follows the user rather than the browser: it is stored on the identity, so it holds across devices, and it travels with the notifications the platform sends, so each recipient reads their own language. Dates, numbers and currencies follow the same locale, including the day-first date ordering the European users of the English interface expect.

The user documentation is delivered the same way. It is not a separate site to be kept in step but a help area inside the application, built from the documentation sources at the same time as the code and shipped in both languages. Figure V.13 shows it in German. A user who upgrades therefore never reads documentation for a version they are not running.

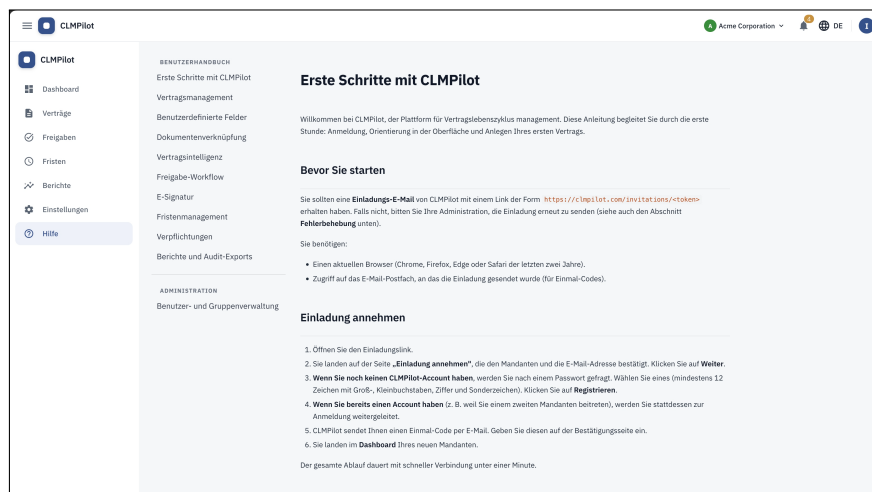


Figure V.13 – The bundled help area in German

V.8 The platform from the operator's side

One surface of the platform belongs to nobody's tenant. Figure V.14 shows the cross-tenant administration area, reachable only by the platform's own operators: the provisioned organisations, the action that creates a new one, and the archival that ends a subscription. Provisioning a tenant from here creates the organisation, waits for its role groups to be in place and sends the first administrator their invitation, after which the customer administers themselves. Archiving is not a deletion — the tenant's history stays intact and its users meet an explicit end-of-service screen.

V.9 Measured performance in production

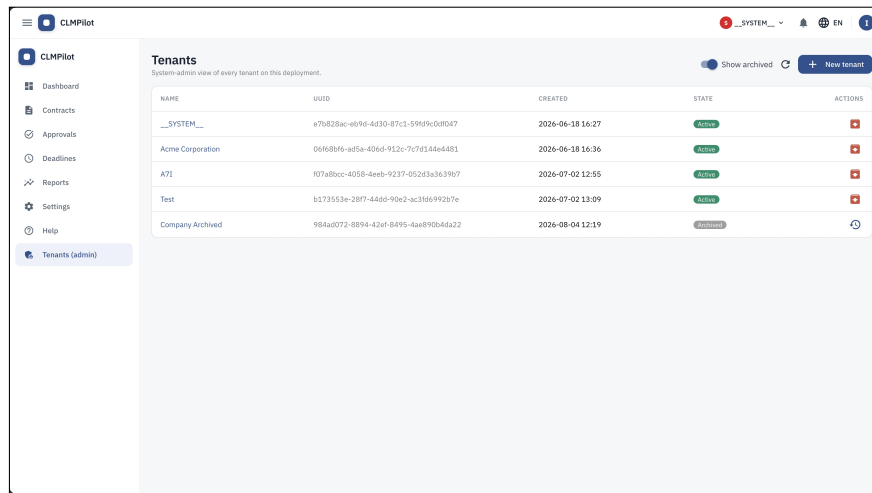


Figure V.14 – The cross-tenant administration area

V.9 Measured performance in production

The chapter closes with measurements from the running pilot: what the platform was asked to do over one week of production, how long it took and how that compares with the budget set for it.

V.9.1 The runtime instrumentation

Since every command, query and event goes through the same dispatch point, the framework can count them all: how many succeeded, failed or were rejected, how many arrive per second, and for each message type the shortest, average and longest handling time. It also records how long a message waits between its dispatch and the start of its handler. These counters live in the backend's memory and build up from the moment the process starts, so they always cover everything that happened since the last release, with no extra infrastructure to run.

The backend ships with an operator dashboard that shows these numbers live, as charts for the whole system and as a table per message type, next to the request traces and the logs. Figure V.15 shows it running against the pilot.

V.9 Measured performance in production

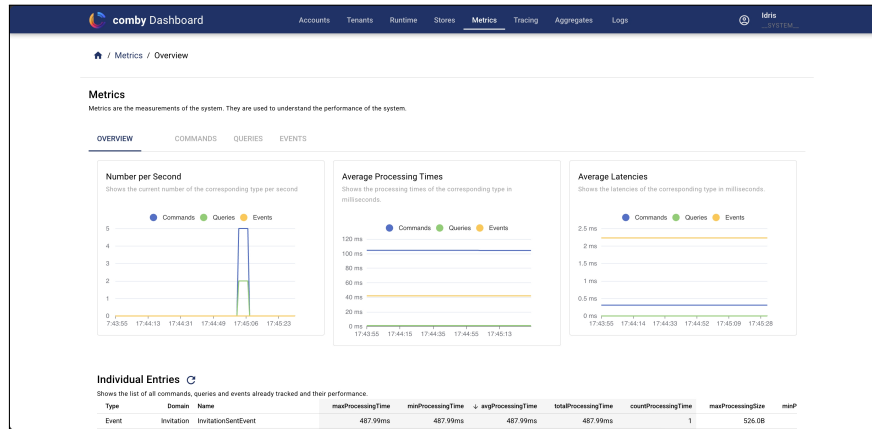


Figure V.15 – The runtime metrics dashboard of the backend

V.9.2 Traffic, response times and reliability

The numbers below come from that dashboard, after the pilot had run untouched for a little over a week. In that time the backend handled 5,672 messages: 3,046 queries of 39 kinds, 2,583 commands of 18 kinds and 43 domain events of 16 kinds. All but 22 of the commands came from the scheduler, whose hourly deadline, obligation and report checks run once per tenant whether anyone is signed in or not, so what users trigger directly is almost entirely reads. Table V.1 groups representative operations by the path they belong to.

Table V.1 – Representative dispatch measurements over roughly one week of production: runs, mean and maximum processing time each.

Operation	Kind	Runs	Mean	Max
<i>Interactive read path</i>				
Contract detail retrieval	Query	744	1.5 ms	49.9 ms
Contract registry listing	Query	21	21.7 ms	36.2 ms
Per-contract risk scoring	Query	11	2.8 ms	13.9 ms
Tenant resolution (per request)	Query	1,030	1.6 ms	10.3 ms
<i>Interactive write path</i>				
Contract update	Command	1	10.3 ms	10.3 ms
Role-group creation	Command	5	14.6 ms	18.5 ms
Sign-in credential verification	Command	2	183.6 ms	205.8 ms
<i>Scheduled automation (hourly / nightly, per tenant)</i>				
Deadline sweep	Command	833	152.7 ms	1,074.2 ms
Obligation sweep	Command	833	124.0 ms	721.7 ms
Scheduled-report tick	Command	829	32.4 ms	142.3 ms
<i>Asynchronous event handling</i>				
Obligation reminder delivered	Event	9	10.6 ms	19.6 ms
Invitation e-mail delivered	Event	1	488.0 ms	488.0 ms

The middle chart of the figure sums it up: queries stay near one millisecond while commands sit around a hundred. In detail, the 3,046 queries averaged 1.3 ms of processing, the heaviest list

V.9 Measured performance in production

query 31 ms, and the slowest single query of the whole week took 52 ms. That is a tenth of the 500 ms search budget set for the platform, before the network and the browser add their part. The heavier work runs where nobody is waiting for it. The scheduled checks average between 32 and 153 ms, with a worst case just over one second, and the slowest events, the invitation and approval notifications at about half a second, are e-mails sent through the external provider after the user's request has already returned. The one visible exception is sign-in, at about 184 ms; that is the cost of checking the password, and it is intentional. The dispatch layer itself is cheap: on average a command waits 0.3 ms between its dispatch and the start of its handler.

Reliability over the same week: no message failed, and the 4 rejections were correct behaviour, not bugs. They were list requests refused by the access rules. A quiet pilot says nothing about heavy load, which stays the job of the performance tests, but it does replace the response-time assumptions with measured values for every message type the platform uses.

V.9.3 Against the non-functional requirements

Table V.2 sets the delivery against each of them.

Table V.2 – The delivery against each non-functional requirement.

Requirement	Asked for	Delivered	Verdict
Security	Transport encryption, data encrypted at rest, every message authorised, a second factor.	TLS 1.2 and 1.3, 2-step sign-in, authorisation on every dispatch; the event store is not encrypted at rest.	Partly met
Compliance and residency	EU processing, 10-year immutable retention, erasure of personal data.	EU host and append-only history; no retention policy configured, and erasure from the log unresolved.	Partly met
Availability and recovery	99.5% in business hours, 24-hour recovery point, 4-hour recovery time.	Health probe every 5 minutes; a production restore drill finished in under a minute. No availability figure computed yet.	Partly met
Performance	Page loads under 1 s, search under 500 ms, document-link resolution under 2 s.	2 orders of magnitude inside the budgets on the server; the browser's share and the store round-trip unmeasured.	Met, server side
Auditability	Immutable attributed events, no destructive delete, an export, the state as of a past date.	Events, the audit tab and the per-contract export, under the record's own access rules; no as-of-date view.	Partly met
Internationalisation	English and German, locale on the user, time-stamps shown in the user's zone.	Both languages in the interface, the notifications and the help; locale on the identity.	Met
Maintainability	Narrow stack, conventional commits, 3 test layers, documentation beside the code.	40 releases with one rollback; lint, tests, build and image on every push, 198 unit tests; help built from the docs.	Met

3 gaps carry the most weight: the event store is not encrypted at rest, the 10-year retention

follows from an append-only log rather than from a configured policy, and the probe's record has not yet been turned into an availability figure.

Conclusion

This chapter showed the delivered platform rather than its construction: the sign-in that admits a user, the dashboard and the reports that put the whole portfolio on one screen, the registry and the audit trail behind every record, the configuration that fits the product to an organisation, the approvals, deadlines and obligations it then watches on that organisation's behalf, the suggestions and the signature requests raised on a linked document, the interface in 2 languages with its documentation shipped alongside, and the operator's own administration surface. The closing measurements complete the picture from the other end, replacing the platform's response-time assumptions with values recorded for every message type it dispatches, and a closing review sets the delivery against each non-functional requirement, the unmet ones included.

With the platform realised across 3 phases and shown here as it runs, the contribution of this report is complete. The general conclusion that follows recaps that contribution and sets out the perspectives for the evolution of the event-sourced system beneath the platform.

Conclusion and Perspectives

This report presented **CLMPilot**, an integration-first contract lifecycle management platform for the DACH *Mittelstand*, built during an end-of-studies internship at Gradient Zero and live in production at `clmpilot.com`. The platform manages the lifecycle of a contract (its metadata, deadlines, approvals and audit trail) while the document itself stays in the customer's own storage, so adoption carries no migration cost. It is built on the `comby` framework, which brings CQRS and Event Sourcing in Go, and that choice is what makes the compliance-grade audit trail a property of the storage model itself: every change to a contract is an immutable event, traced to an actor and a moment, and any past state can be reconstructed on demand.

The pilot was delivered as a release train of 3 product phases, all realised and shipped. Phase 0 laid the engineering foundations: repositories, continuous integration and delivery, and the roadmap that served as the single source of truth for the whole internship, ordering both the work and the structure of this report. Phase 1 delivered the full-stack MVP: the 4 core domains (Contract, Approval, Deadline and DocumentLink) behind a generated API, an internationalised web application, and a monitored, backed-up production deployment on EU-hosted infrastructure. Phase 2 added the enterprise tier: intelligence-assisted metadata and clause extraction, electronic signature, per-tenant custom fields, obligation tracking and advanced reporting. The work ran under Extreme Programming adapted to a solo developer, with weekly supervision and release gates in place of team practices. The main difficulties were operational: delivering alone against fixed phase milestones, and keeping the releases of 3 separately versioned services reproducible as one unit.

The closing weeks of the internship went into an engineering audit of the event-sourced framework beneath the platform, asking of each guarantee the architecture depends on where it is enforced and where that enforcement stops. Most of the perspectives below come from its findings.

On consistency, the audit found that the framework enforces its strongest guarantees in the running process and not in the store: optimistic concurrency is checked against a version cache in memory, and the events of a command are written one by one. A single instance behaves correctly, but a second concurrent writer, even a brief overlap during a deployment, could write conflicting versions into a stream without any error, and a crash between two writes leaves an operation half-recorded. The fix is to enforce these rules in the event store: a uniqueness constraint on aggregate and version, and one transaction per command. A corrupted stream then becomes a conflict the dispatcher can catch and retry.

Operating at scale raises the same kind of question. Start-up rebuilds every read model by replaying the whole log, and the snapshot mechanism meant to bound that has defects to correct before it can be enabled. Under load, nothing bounds concurrent work, ordering per aggregate

is not guaranteed, and shutdown can drop accepted commands. Assigning each aggregate to one worker in a sharded execution model would give ordering, backpressure and a clean shutdown at once. Read models also need a recovery story: a global order over the log, checkpointed projections and a dead-letter queue, so a failing projection can be repaired and replayed instead of silently skipping events, and replicas that catch up from the store when broker messages are missed. This is what it would take to run the multi-instance topology the deployment design already anticipates.

Isolation between tenants is solid (a tenant context on every command and event, with row-level security behind it), but growth adds problems isolation does not solve. Nothing bounds one tenant's load, so quotas and metering are needed on the shared dispatch path. Partitioning the event store by tenant would give large or compliance-sensitive customers their own storage, backup and deletion boundaries. Corporate groups need consolidation across their tenants, which has no model yet. And the immutable log conflicts with GDPR erasure; per-tenant encryption keys would settle that conflict, since destroying a key erases the personal data while the stream stays intact.

On the product side, the extraction pipeline is worth extending. It is a stateless boundary (fetch a document's bytes, extract metadata and clauses, event-source the result) and nothing in it is specific to contract documents; the broker communications around a contract, which the platform does not touch today, could be fed through the same path. Deeper connections to the storage and ERP systems the *Mittelstand* already runs would serve the same goal: adoption that asks no migration of the customer.

The pilot showed that a compliance-grade contract registry can be delivered at mid-market cost when auditability is built into the storage model. The work sketched above is what would turn that pilot into a durable product.

References

- [1] GREG YOUNG. CQRS Documents. Whitepaper, (2010). 4, 27, 28
- [2] MARTIN FOWLER. CQRS. <https://martinfowler.com/bliki/CQRS.html>, (2011). Accessed: 20 May 2026. 27, 28
- [3] MARTIN FOWLER. Event Sourcing. <https://martinfowler.com/eaDev/EventSourcing.html>, (2005). Accessed: 20 May 2026. 4, 27, 28
- [4] INSTITUT FÜR MITTELSTANDSFORSCHUNG BONN. Mittelstandsdefinition des IfM Bonn. <https://www.ifm-bonn.org/definitionen/mittelstandsdefinition-des-ifm-bonn>. Accessed: 22 May 2026. 5
- [5] EUROPEAN COMMISSION. Commission Recommendation 2003/361/EC concerning the definition of micro, small and medium-sized enterprises. https://single-market-economy.ec.europa.eu/smes/sme-definition_en, (2003). Accessed: 22 May 2026. 5
- [6] FORTUNE BUSINESS INSIGHTS. Contract Lifecycle Management (CLM) Solution Market Size, Share & COVID-19 Impact Analysis, and Regional Forecast, 2026–2034. <https://www.fortunebusinessinsights.com/contract-lifecycle-management-clm-solution-market-106472>, (2026). Report ID FBI106472. Accessed: 22 May 2026. 5
- [7] KENT BECK AND CYNTHIA ANDRES. *Extreme Programming Explained: Embrace Change*. Addison-Wesley Professional, second edition (2004). 8, 9
- [8] PORTLAND PATTERN REPOSITORY. Extreme Programming For One. <https://wiki.c2.com/?ExtremeProgrammingForOne>. Accessed: 6 July 2026. 8, 10
- [9] MARTIN FOWLER. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional (2003). 27, 28
- [10] MICROSOFT. CQRS Pattern. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs>, (2025). Accessed: 1 July 2026. 28
- [11] MICROSOFT. Event Sourcing Pattern. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing>, (2026). Accessed: 1 July 2026. 28
- [12] ERIC EVANS. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional (2003). 28

- [13] VAUGHN VERNON. *Implementing Domain-Driven Design*. Addison-Wesley Professional (2013). 28
- [14] KURRENT, INC. KurrentDB (formerly EventStoreDB) Documentation. <https://docs.kurrent.io/>. Accessed: 31 July 2026. 33
- [15] AXONIQ. Axon Framework Documentation. <https://docs.axoniq.io/>. Accessed: 31 July 2026. 33
- [16] EVENTFLOW CONTRIBUTORS. EventFlow: CQRS+ES Framework for .NET. <https://geteventflow.net/>. Accessed: 31 July 2026. 33
- [17] ANTHROPIC. Claude API Documentation. <https://docs.anthropic.com/>. Accessed: 21 July 2026. 51
- [18] DOCUSEAL. DocuSeal: Open-Source Document Signing. <https://www.docuseal.com/>. Accessed: 21 July 2026. 53
- [19] EUROPEAN PARLIAMENT AND COUNCIL. Regulation (EU) No 910/2014 on electronic identification and trust services for electronic transactions in the internal market (eIDAS). <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32014R0910>, (2014). Accessed: 21 July 2026. 53
- [20] TWILIO SENDGRID. SendGrid Web API Documentation. <https://www.twilio.com/docs/sendgrid>. Accessed: 21 July 2026. 54
- [21] VUE.JS TEAM. Vue.js Guide. <https://vuejs.org/guide/>. Accessed: 21 July 2026. 54
- [22] QUASAR FRAMEWORK. Quasar Framework Documentation. <https://quasar.dev/docs>. Accessed: 21 July 2026. 54
- [23] OPENAPI INITIATIVE. OpenAPI Specification. <https://spec.openapis.org/oas/latest.html>. Accessed: 21 July 2026. 55
- [24] DOCKER INC. Docker Compose Documentation. <https://docs.docker.com/compose/>. Accessed: 21 July 2026. 58

Appendices

A.1 Glossary

- CLM** Contract Lifecycle Management: the discipline and tooling that track a contract from drafting through approval, signature, obligations and renewal to expiry.
- CQRS** Command Query Responsibility Segregation: an architecture that separates the write model (commands validated by aggregates) from the read models (projections serving queries).
- ES** Event Sourcing: persisting every state change as an immutable event; the current state is rebuilt by replaying the log.
- DDD** Domain-Driven Design: modelling software around the business domain, organised in bounded contexts and aggregates.
- Aggregate** The DDD consistency unit: an object cluster that guards its own invariants and is modified only through its intentions (commands).
- Reactor** In comby, a component that reacts to a domain event by dispatching a follow-up command, connecting 2 contexts without coupling them.
- RBAC** Role-Based Access Control: permissions are granted to roles (groups), and users acquire them through role membership.
- MFA / OTP** Multi-Factor Authentication / One-Time Password: a second authentication step, realised in CLMPilot as an emailed one-time code.
- SaaS** Software as a Service: software delivered as a hosted, subscription-based service.
- SPA** Single-Page Application: a web client that loads once and renders every view itself, talking to the backend through an API.
- MVP** Minimum Viable Product: the smallest product slice that delivers verifiable customer value.
- LLM** Large Language Model: the hosted model class behind the intelligence service's extraction, classification and clause detection.
- OCR** Optical Character Recognition: extracting a text layer from scanned images; the prerequisite for clause detection on scans.
- eIDAS** EU Regulation 910/2014 on electronic identification and trust services; its Article 25 gives simple electronic signatures legal effect.
- GDPR** The EU General Data Protection Regulation, the privacy baseline for processing personal data.
- GoBD** German principles for the proper keeping of books and records in electronic form; the driver behind the immutable audit trail.
- OAuth 2.0** The delegation protocol used to connect a tenant's document storage (Google Drive, SharePoint) without holding user passwords.
- ERP** Enterprise Resource Planning: the business-management systems (finance, procurement) a CLM platform typically borders on.
- SLA** Service-Level Agreement: the availability and support commitments a provider makes to its customers.
- Semver** Semantic Versioning: `major.minor.patch` version numbers whose increments encode compatibility.
- CI/CD** Continuous Integration / Continuous Delivery: the automated lint-test-build-publish pipeline every repository runs.

A.2 Comby framework reference

Every state change in the platform is a command dispatched on the comby bus, and every read is a query; the permission checked at dispatch is derived mechanically from the context name and the message type:

```
comby.NewCommand("Contract", &command.ContractCommandRegister{...})
    -> checked permission "Contract.ContractCommandRegister"
```

A.3 Configuration and delivery pipeline

A caller must hold that permission through one of its role groups; members of the framework-seeded system-administrator group bypass the check, which is what the *system dispatch* convention of Section IV.2 relies on: boot-time seeding, reactors and scheduler jobs dispatch with system authority, while every interactive call is checked against the 5 seeded role groups (administrator, legal, finance, manager, viewer).

Alongside CLMPilot’s own contexts, the framework registers its default domains — tenant, account, identity, group, invitation, asset and webhook — which bring their own commands and queries (account registration and login, invitation issue and confirmation, group and identity administration). Table A.1 catalogues the CLMPilot-specific contexts as realised at the close of Phase 2. Event names mirror their commands (a `ContractCommandRegister` that succeeds appends a `ContractRegisteredEvent`, and so on), so the event catalogue follows from the command column.

Table A.1 – Command and query catalogue of the CLMPilot contexts.

Context	Commands (intentions)	Queries
Contract	Register, Update, Terminate, Archive, Delete (soft), Restore, RequestAuditExport	Get, List, Expiring, Search, Volume, DepartmentSummary, CounterpartySummary, Risk, Events (audit trail)
Approval	Request, ApproveStep, RejectStep, Cancel	Get, List, PendingForIdentity, HistoryForContract
Deadline	Set, Update, AcknowledgeReminder, Cancel, CheckDeadlines (batch), MarkMissed (batch)	Get, List, Upcoming, Missed
DocumentLink	Link, Unlink, Refresh (metadata)	ListByContract, IntegrationAccess, PickerAccess
CustomField	Create, Update, Delete	List (definitions per tenant)
Signature	CreateAndSend, ApplyStatus (webhook, system dispatch), Void	ForContract
Obligation	Create, Update, Complete, CheckObligations (batch), MarkMissed (batch)	Upcoming, Report
ReportConfig	Create, Update, Delete, RunScheduled (scheduler, system dispatch)	List

The batch commands (`CheckDeadlines`, `MarkMissed`, `CheckObligations`, `RunScheduled`) are dispatched by the in-process scheduler once per tenant with system authority; they are the only commands never issued by an interactive caller. The exported OpenAPI contract covering this surface counts 165 paths at the close of Phase 2 and is regenerated from the backend whenever the surface changes; the frontend’s typed client is generated from the same document.

A.3 Configuration and delivery pipeline

All excerpts in this section are the real files from the `clmpilot-deployment` repository, trimmed to their structure; secrets and credentials are replaced by `$(VAR)` environment references resolved from an `.env` file that never leaves the host.

A.3.1 Production Docker Compose stack

The production stack is one Compose project: 3 application services exposed on loopback only, 4 backing stores on an internal network with no internet access, and an explicit memory limit on every container so a runaway process cannot exhaust the 4 GB host.

```
services:
  frontend:                                     # SPA behind host nginx
    image: ${REGISTRY}/clmpilot-frontend:${FRONTEND_VERSION}
    mem_limit: 64m
    ports: ["127.0.0.1:18081:80"]
    networks: [web]

  backend:                                       # Go service on comby
    image: ${REGISTRY}/clmpilot-backend:${BACKEND_VERSION}
    mem_limit: 256m
    ports: ["127.0.0.1:18080:8080"]
    environment:                                # ~60 keys, grouped:
      # PostgreSQL event + command stores, Redis, NATS, MinIO
      # OAuth (Google/SharePoint) client ids + callback base URL
      # DocuSeal base URL / API key / webhook secret
      # SendGrid key + sender; SMTP fallback (dev mail catcher)
      # comby defaults: session cookies, registration mode,
      #                               system-tenant administrator seeding
    depends_on: [postgres, redis, nats, minio] # all health-gated
    networks: [web, internal]

  intelligence:                                 # stateless Python extraction service
    image: ${REGISTRY}/clmpilot-intelligence:${INTELLIGENCE_VERSION}
    mem_limit: 384m
    environment:                                # LLM provider/model, Redis cache,
      # backend service token
    networks: [web, internal]                  # web only for LLM egress

  postgres:                                    # event store + command store
    image: postgres:16-alpine
    mem_limit: 1g
    command: postgres -c shared_buffers=256MB -c max_connections=50
      -c effective_cache_size=1GB -c work_mem=8MB
    networks: [internal]

  redis:    { image: redis:8-alpine, mem_limit: 96m,
              command: --maxmemory 64mb --maxmemory-policy allkeys-lru }
  nats:     { image: nats:2-alpine, mem_limit: 64m } # JetStream on
  minio:    { image: minio/minio, mem_limit: 256m }
```

A.3 Configuration and delivery pipeline

```
networks:
  web:      { driver: bridge }           # host nginx reaches these
  internal: { driver: bridge, internal: true } # no internet
```

2 details are deliberate. The backend's runtime image is distroless (the Go binary and nothing else, no shell), so it carries no container-level health check; liveness is probed from outside instead (Section IV.9). And the self-hosted DocuSeal service is declared behind an opt-in Compose profile: the pilot signs against the provider's hosted sandbox, and the self-hosted container starts only when an operator selects that profile.

A.3.2 Host nginx reverse proxy

TLS terminates on the host, not in a container. One site file proxies the apex domain to the loopback ports; certificates are issued and renewed by certbot over the HTTP-01 challenge.

```
upstream clmpilot_backend { server 127.0.0.1:18080; keepalive 32; }
upstream clmpilot_frontend { server 127.0.0.1:18081; keepalive 16; }

server {                                     # port 80: ACME + redirect
  listen 80; server_name clmpilot.com;
  location ~ /.well-known/acme-challenge/ { root /usr/share/nginx/html; }
  location / { return 301 https://$host$request_uri; }
}

server {                                     # port 443: TLS + proxy
  listen 443 ssl http2; server_name clmpilot.com;
  ssl_certificate /etc/letsencrypt/live/clmpilot.com/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/clmpilot.com/privkey.pem;
  ssl_protocols TLSv1.2 TLSv1.3;           # Mozilla intermediate profile

  add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
    always;
  add_header X-Content-Type-Options "nosniff" always;
  add_header X-Frame-Options "DENY" always;
  add_header Referrer-Policy "strict-origin-when-cross-origin" always;

  client_max_body_size 25m;                # contract attachments are small

  location /api/ { proxy_pass http://clmpilot_backend; }
  location /webhooks/ { proxy_pass http://clmpilot_backend; }
  location / { proxy_pass http://clmpilot_frontend; }
}
```

A.3.3 systemd service unit

The Compose stack runs under a systemd unit so it survives host reboots and restarts on failure; this is the complete unit.

```
[Unit]
Description=clmpilot deployment service
Requires=docker.service
After=docker.service network-online.target

[Service]
Type=simple
WorkingDirectory=/home/user/clmpilot
ExecStartPre=/usr/bin/docker compose -f docker-compose.yml \
    -f docker-compose.prod.yml down --remove-orphans
ExecStart=/usr/bin/docker compose -f docker-compose.yml \
    -f docker-compose.prod.yml up
ExecStop=/usr/bin/docker compose -f docker-compose.yml \
    -f docker-compose.prod.yml down
TimeoutStartSec=300
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
```

A.3.4 Continuous-integration pipeline

Each repository carries its own GitHub Actions workflow with the same shape; the backend's is the largest and stands for the pattern. It guards main and develop on every push and pull request, and a semantic-version tag additionally publishes the image.

```
on:
  push:          { branches: [main, develop], tags: ["v*.*.*"] }
  pull_request: { branches: [main, develop] }

jobs:
  lint:          # golangci-lint over the module (comby pinned at v3.0.2)
  test:          # go test suites against disposable PostgreSQL, Redis
                  # and NATS service containers
  build:         # compile the binary; regenerate + diff the OpenAPI spec
  image:         # on tags: build the distroless image and push it to the
                  # registry, tagged with commit, branch and release version
```

The frontend workflow lints, runs the 198-test unit suite and builds the production bundle; the intelligence workflow runs its linter, its tests and the extraction-evaluation harness; the deployment workflow validates the Compose files and shell scripts it ships. Dependency updates arrive as weekly automated pull requests and pass the same gates.

A.3.5 Release script and the lockstep bypass

A release is executed on the host by `deploy.sh`, which refuses unpinned versions, updates the pinned tags, restarts the unit and polls the health endpoint before reporting success.

```
sudo ./scripts/deploy.sh production v1.2.3

# 1. validate: env known, version is a semver ('latest' rejected),
#    .env present
# 2. pin BACKEND_VERSION / FRONTEND_VERSION / INTELLIGENCE_VERSION
#    to the given tag in .env (<-- the lockstep)
# 3. docker compose pull
# 4. systemctl restart clmpilot.service
# 5. poll http://127.0.0.1:18080/healthz, 60 x 2 s; on timeout,
#    print service status + last backend logs and exit non-zero
# 6. docker image prune -f
# Rollback: re-run with the previous version number.
```

The script pins all 3 service versions in lockstep (step 2), which keeps a full release reproducible but makes a single-service hotfix awkward. The bypass used in practice edits the one pinned variable in `.env` directly and restarts the unit:

```
sudo sed -i 's/^FRONTEND_VERSION=.* /FRONTEND_VERSION=v0.3.1/' .env
sudo systemctl restart clmpilot.service
```

The services were re-unified onto one version at the close of Phase 2 so the lockstep procedure and its rollback apply again.

A.3.6 Packaging caveat: the bundled help documentation

The user documentation is bundled into the SPA at build time from the repository's `docs/` tree (Section IV.7). The frontend's `.dockerignore`, written to keep the build context lean, originally excluded that tree — so the CI-built image shipped with an empty help area while every local build, which does not pass through Docker, had the content. The gap surfaced only in production and was corrected by a hotfix release that re-included `docs/` in the build context. The lesson is recorded here because it generalises: whatever a build bundles from the repository must be present in the image build context, and a bundle that can silently be empty deserves a CI assertion.

CLMPilot: an integration-first Contract Lifecycle Management platform, built on a CQRS and Event Sourcing Architecture

Abstract

Mid-sized DACH enterprises manage contracts by hand: deadlines pass unnoticed, approvals leave no auditable record, and GoBD, GDPR and ISO 27001 demand immutable records. **CLMPilot** is an integration-first CLM platform for the DACH *Mittelstand*: it owns metadata, deadlines, approvals and audit trail; the document stays in the customer's storage. Built on comby, an in-house Go CQRS and Event Sourcing framework, its audit trail is architectural.

Keywords: Contract Lifecycle Management, CQRS, Event Sourcing, Audit Trail.

Résumé

Les moyennes entreprises du DACH gèrent leurs contrats à la main : les échéances passent inaperçues, les validations ne laissent nulle trace, quand le GoBD, le RGPD et l'ISO 27001 imposent l'immuabilité. **CLMPilot** est une plateforme CLM orientée intégration pour le *Mittelstand* : elle gère métadonnées, échéances, validations et audit ; le document reste chez le client. Bâtie sur comby, framework Go interne CQRS/*Event Sourcing*, l'audit est architectural.

Mots-clés : gestion du cycle de vie des contrats, CQRS, Event Sourcing, piste d'audit.

ملخص

تدير الشركات المتوسطة في DACH عقودها يدويًا، فتفوت مواعيد التجديد ولا تترك الموافقات أثرًا قابلاً للتدقيق، بينما تشترط GoBD والألحقة الأوروبية لحماية البيانات ومعياري ISO 27001 سجلات غير قابلة للتعديل ومنسوبة. و CLMPilot منصة لإدارة دورة حياة العقود قائمة على أولوية الدمج، تدير البيانات الوصفية والآجال والموافقات وسجل التدقيق مع بقاء الوثيقة لدى العميل، وتعتمد على comby ، وهو إطار داخلي بلغة Go يطبق CQRS ومصادر الأحداث، فيغذو سجل التدقيق خاصية أصيلة في النظام.

الكلمات المفتاحية: إدارة دورة حياة العقود، CQRS ، Event Sourcing ، سجل التدقيق.

